# Final Project - Modeling GDP to Predict Recession

Harrison Halesworth

2026-03-23

## 1.) Setup

### Package Installation and Setup

```
# install.packages("quantmod")
# install.packages("cli")
# install.packages(c("readr", "pracma", "devtools"))
# devtools::install_github("cykbennie/fbi")
library(fbi)
library(lmtest)
```

```
## Warning: package 'lmtest' was built under R version 4.4.3
```

```
## Loading required package: zoo
```

```
## Warning: package 'zoo' was built under R version 4.4.3
```

```
##
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

```
library(sandwich)
```

```
## Warning: package 'sandwich' was built under R version 4.4.3
```

```
library(runjags)
```

```
## Warning: package 'runjags' was built under R version 4.4.3
```

```
library(coda)
```

```
## Warning: package 'coda' was built under R version 4.4.3
```

### Load in Data with fbi Package

```r
# Load in data from 1959-2025
qd <- fredqd(
  file = "./current.csv",
  date_start = as.Date("1959-12-01"),
  date_end   = as.Date("2025-12-01"),
  transform  = FALSE
)

# Keep variables mentioned in proposal
# date - quarter date of observation (YYYY-MM-DD)
# GDPC1 - Real Gross Domestic Product ($B)
# UNRATE - Civilian Unemployment Rate (%)
# CPIAUCSL - Consumer Price Index for all Urban Consumers (1982-1984=100)
# GS10 - 10-Year Treasury Constant Maturity Rate (%)
# TB3MS -  3-Month Treasury Bill Secondary Market Rate (%)
vars <- c("date", "GDPC1", "UNRATE", "CPIAUCSL", "INDPRO", "GS10", "TB3MS")
df <- qd[, vars]

# Rename variables after log transformations
df$gdp_growth <- c(NA, diff(log(df$GDPC1)))
df$inflation  <- c(NA, diff(log(df$CPIAUCSL)))
df$ip_growth  <- c(NA, diff(log(df$INDPRO)))

# Calculate treasury yield spread
df$spread     <-df$GS10 - df$TB3MS

# Calculate lag variables (values from previous quarter)
df$lag_gdp_growth <- c(NA, head(df$gdp_growth, -1))
df$lag_unrate     <- c(NA, head(df$UNRATE, -1))
df$lag_inflation  <- c(NA, head(df$inflation, -1))
df$lag_ip_growth  <- c(NA, head(df$ip_growth, -1))
df$lag_spread     <- c(NA, head(df$spread, -1))

df_final <- na.omit(df[, c(
  "date", "gdp_growth", "lag_gdp_growth", "lag_unrate",
  "lag_inflation", "lag_ip_growth", "lag_spread"
)])

# Inspect data
head(df_final)
```

```
##          date   gdp_growth lag_gdp_growth lag_unrate lag_inflation
## 6  1960-06-01 -0.005398116    0.022237184     5.1333  0.0009086779
## 7  1960-09-01  0.004887350   -0.005398116     5.2333  0.0059895041
## 8  1960-12-01 -0.012914344    0.004887350     5.5333  0.0005645392
## 9  1961-03-01  0.006727499   -0.012914344     6.2667  0.0064005608
## 10 1961-06-01  0.016836022    0.006727499     6.8000  0.0020127481
## 11 1961-09-01  0.019021595    0.016836022     7.0000 -0.0003351768
##    lag_ip_growth lag_spread
## 6     0.05854462     0.6134
## 7    -0.02193741     1.2667
## 8    -0.01657766     1.4733
## 9    -0.02440092     1.5800
```

```
## 10   -0.01525600     1.4367
## 11    0.03887692     1.4867
```

```
summary(df_final)
```

```
##       date              gdp_growth        lag_gdp_growth        lag_unrate
##  Min.   :1960-06-01   Min.   :-0.082043   Min.   :-0.082043   Min.   : 3.400
##  1st Qu.:1976-10-16   1st Qu.: 0.003493   1st Qu.: 0.003493   1st Qu.: 4.633
##  Median :1993-03-01   Median : 0.007602   Median : 0.007681   Median : 5.567
##  Mean   :1993-03-01   Mean   : 0.007320   Mean   : 0.007391   Mean   : 5.869
##  3rd Qu.:2009-07-17   3rd Qu.: 0.011239   3rd Qu.: 0.011331   3rd Qu.: 6.917
##  Max.   :2025-12-01   Max.   : 0.074758   Max.   : 0.074758   Max.   :13.000
##  lag_inflation       lag_ip_growth          lag_spread
##  Min.   :-0.023168   Min.   :-0.1343803   Min.   :-1.480
##  1st Qu.: 0.005033   1st Qu.:-0.0003963   1st Qu.: 0.435
##  Median : 0.007831   Median : 0.0067439   Median : 1.473
##  Mean   : 0.009119   Mean   : 0.0057158   Mean   : 1.399
##  3rd Qu.: 0.011336   3rd Qu.: 0.0142108   3rd Qu.: 2.502
##  Max.   : 0.038698   Max.   : 0.0905126   Max.   : 3.800
```

```
nrow(df_final)
```

```
## [1] 263
```

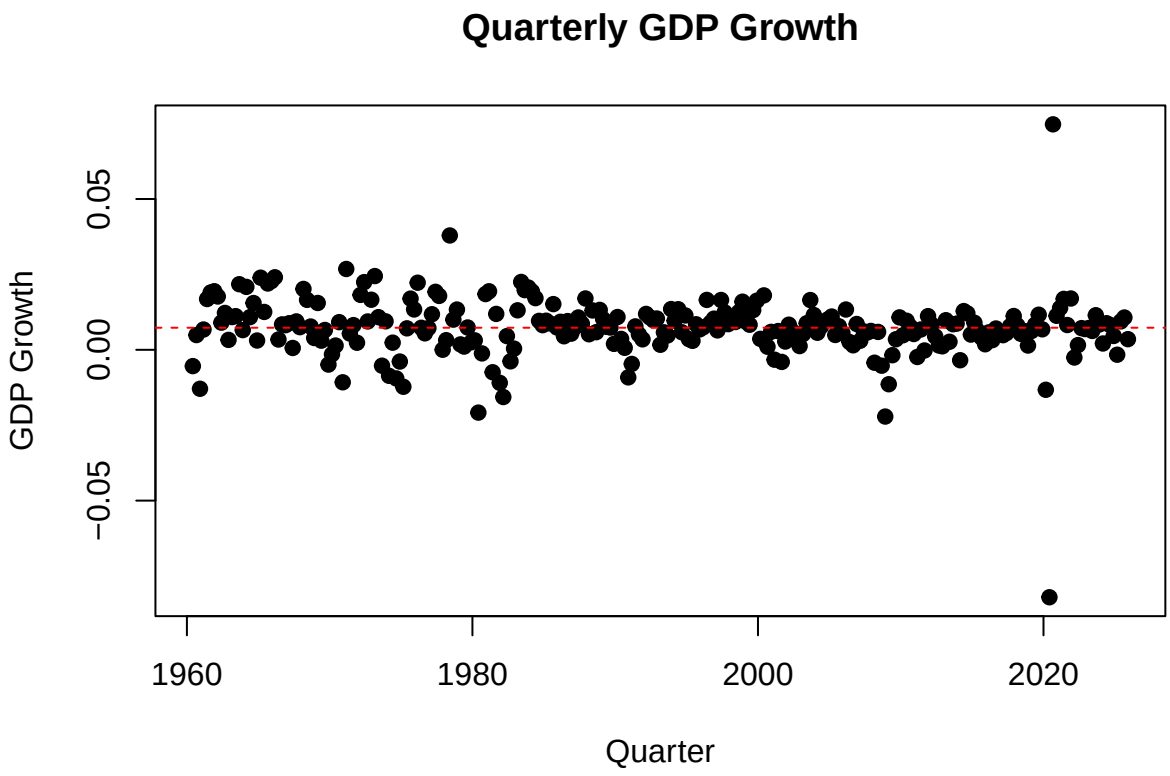
# 2.) EDA

## Plots of variables

### Quarterly GDP Growth

```
# Plot quarterly GDP growth
plot(df_final$date, df_final$gdp_growth,
     main="Quarterly GDP Growth",
     xlab="Quarter",
     ylab="GDP Growth",
     pch=19)

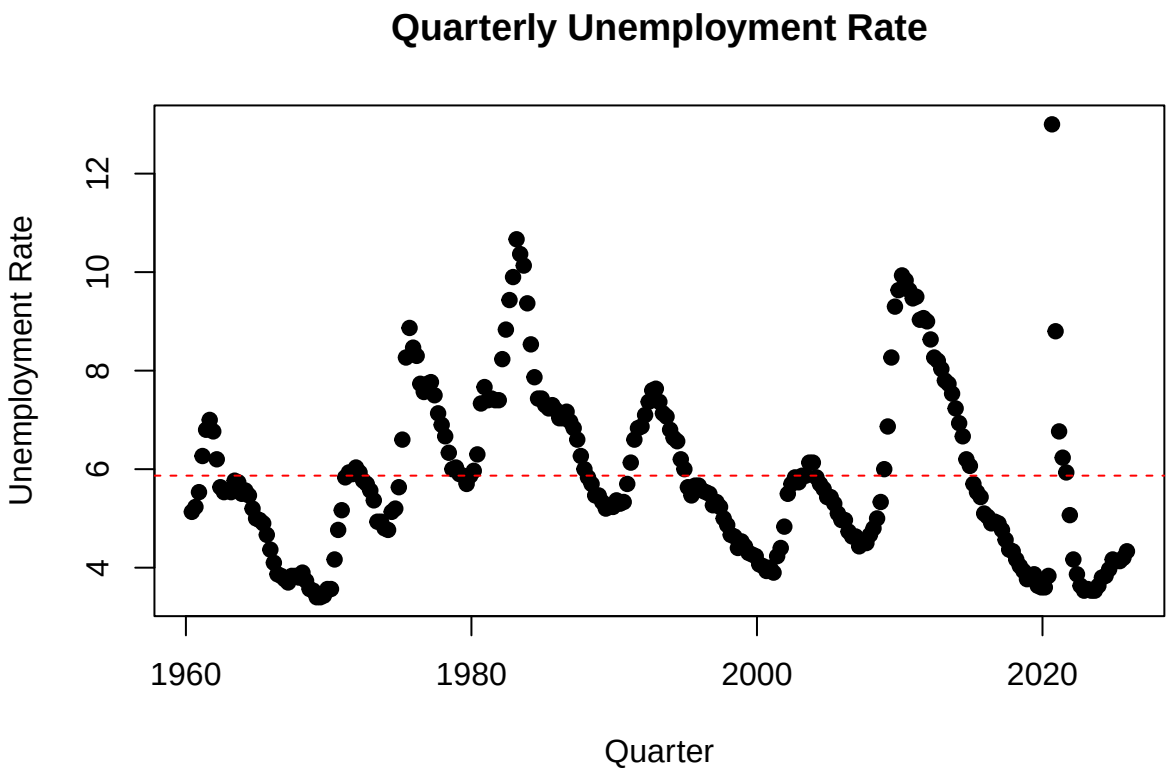
# Add average line
abline(h=mean(df_final$gdp_growth),
       col="red",
       lty=2)
```




### Quarterly Unemployment Rate (lagged)

```
# Plot quarterly Unemployment Rate
plot(df_final$date, df_final$lag_unrate,
     main="Quarterly Unemployment Rate",
     xlab="Quarter",
     ylab="Unemployment Rate",
     pch=19)

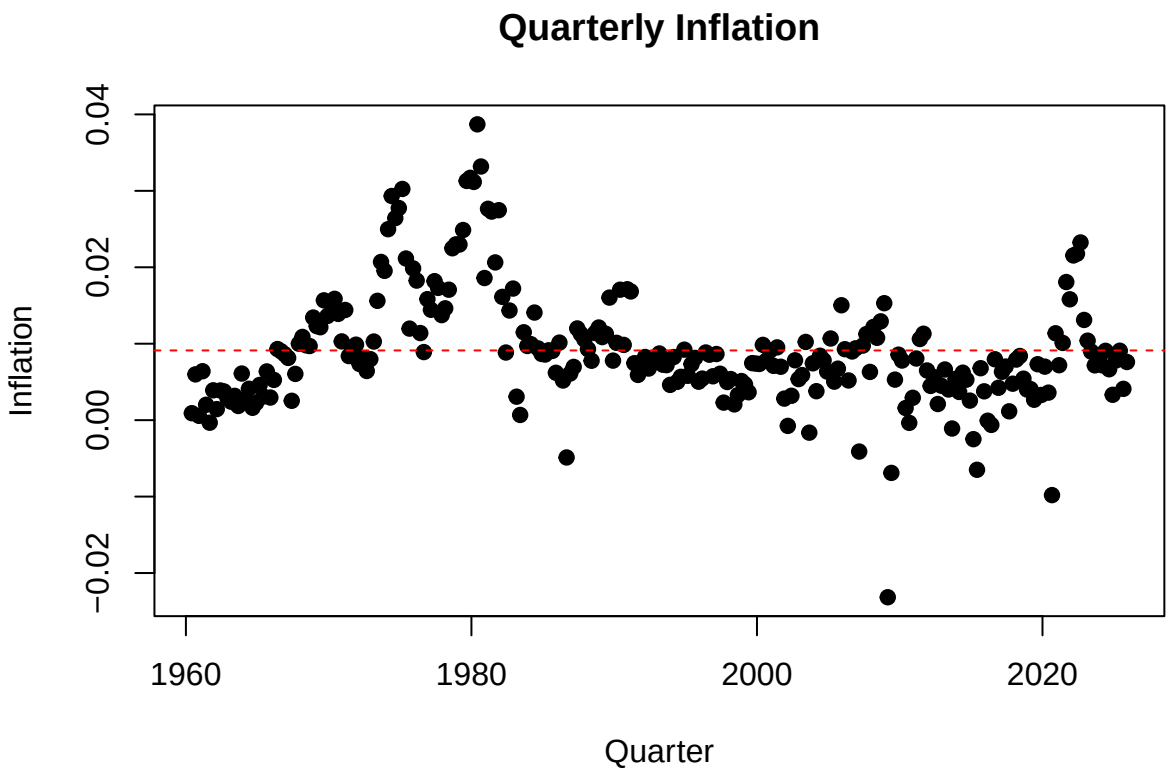
# Add average line
abline(h=mean(df_final$lag_unrate),
       col="red",
       lty=2)
```




### Quarterly Inflation (lagged)

```
# Plot quarterly Inflation
plot(df_final$date, df_final$lag_inflation,
     main="Quarterly Inflation",
     xlab="Quarter",
     ylab="Inflation",
     pch=19)

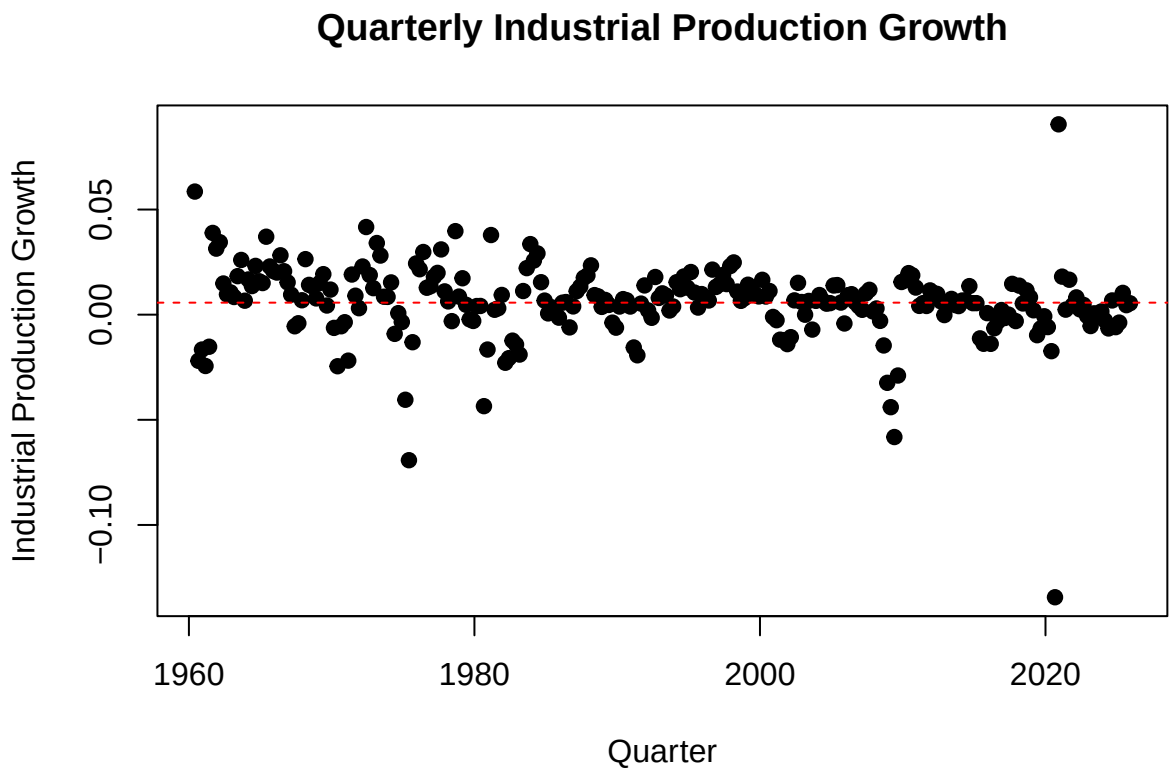
# Add average line
abline(h=mean(df_final$lag_inflation),
       col="red",
       lty=2)
```




### Quarterly Industrial Production Growth (lagged)

```
# Plot quarterly Industrial Production Growth
plot(df_final$date, df_final$lag_ip_growth,
     main="Quarterly Industrial Production Growth",
     xlab="Quarter",
     ylab="Industrial Production Growth",
     pch=19)

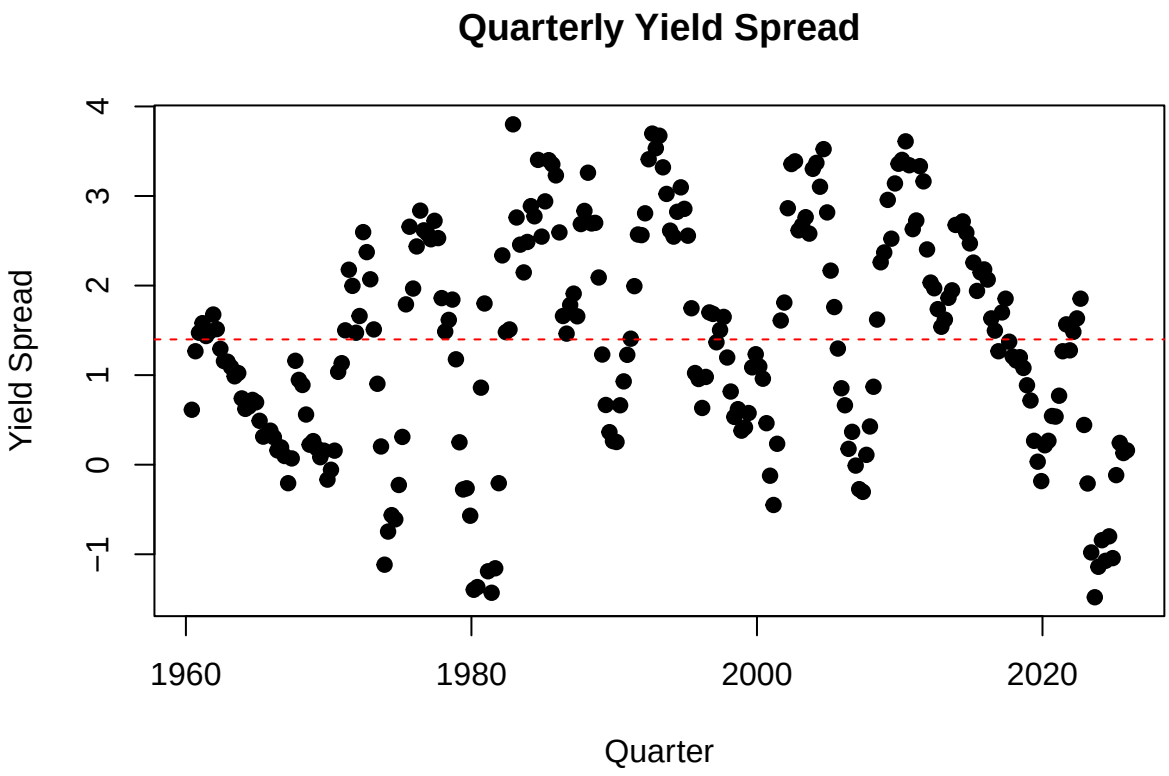
# Add average line
abline(h=mean(df_final$lag_ip_growth),
       col="red",
       lty=2)
```




### Quarterly Yield Spread (lagged)

```
# Plot quarterly Yield Spread
plot(df_final$date, df_final$lag_spread,
     main="Quarterly Yield Spread",
     xlab="Quarter",
     ylab="Yield Spread",
     pch=19)

# Add average line
abline(h=mean(df_final$lag_spread),
       col="red",
       lty=2)
```




### Summary Stats

```
# Summary stats for variables
summary(df_final)
```

```
##       date              gdp_growth        lag_gdp_growth        lag_unrate
##  Min.   :1960-06-01   Min.   :-0.082043   Min.   :-0.082043   Min.   : 3.400
##  1st Qu.:1976-10-16   1st Qu.: 0.003493   1st Qu.: 0.003493   1st Qu.: 4.633
##  Median :1993-03-01   Median : 0.007602   Median : 0.007681   Median : 5.567
##  Mean   :1993-03-01   Mean   : 0.007320   Mean   : 0.007391   Mean   : 5.869
##  3rd Qu.:2009-07-17   3rd Qu.: 0.011239   3rd Qu.: 0.011331   3rd Qu.: 6.917
##  Max.   :2025-12-01   Max.   : 0.074758   Max.   : 0.074758   Max.   :13.000
##  lag_inflation       lag_ip_growth         lag_spread
##  Min.   :-0.023168   Min.   :-0.1343803   Min.   :-1.480
##  1st Qu.: 0.005033   1st Qu.:-0.0003963   1st Qu.: 0.435
##  Median : 0.007831   Median : 0.0067439   Median : 1.473
##  Mean   : 0.009119   Mean   : 0.0057158   Mean   : 1.399
##  3rd Qu.: 0.011336   3rd Qu.: 0.0142108   3rd Qu.: 2.502
##  Max.   : 0.038698   Max.   : 0.0905126   Max.   : 3.800
```

# 3.) Frequentist Model

## Fit OLS Model

```
# Extract variable names
X_names <- colnames(df_final)
X_names <- X_names[3:ncol(df_final)]
formula <- as.formula(paste("gdp_growth ~", paste(X_names, collapse = " + ")))

# Scale data for regression
df_final_scaled <- as.data.frame(scale(df_final[,-1]))

# Use lm() to fit OLS model for gdp growth
freq_mod <- lm(formula, data=df_final_scaled)

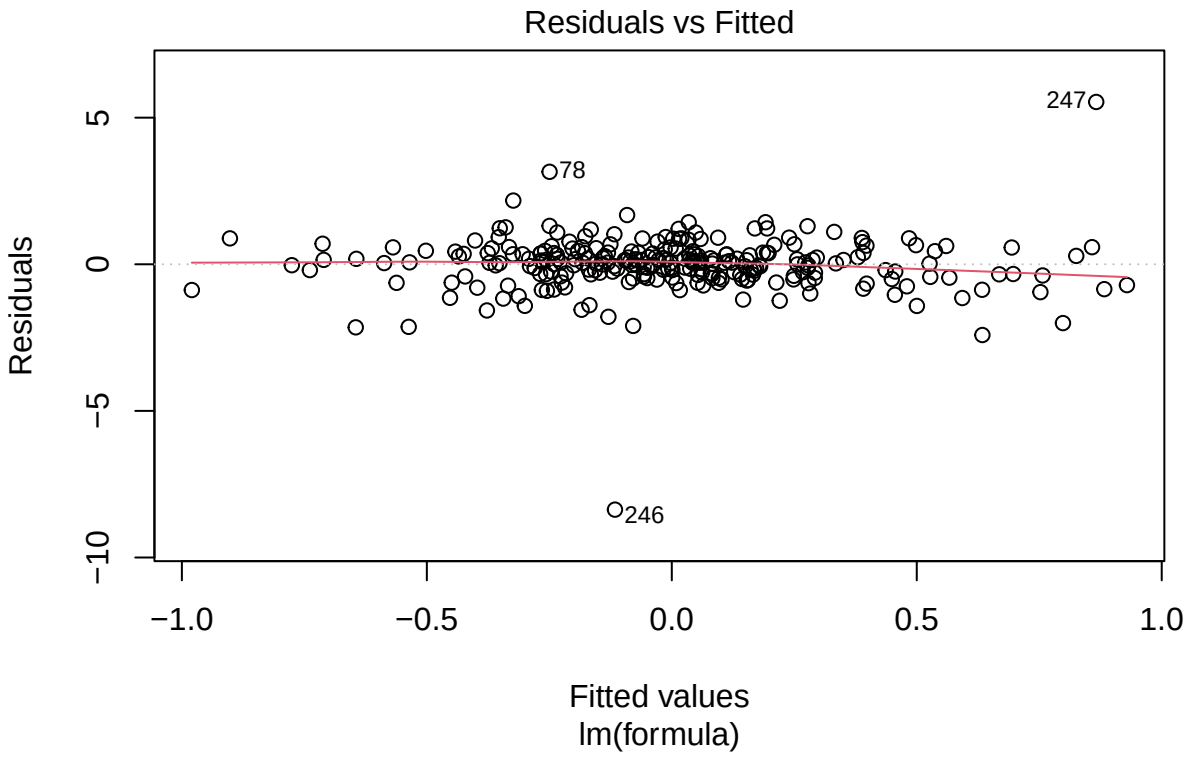
# Summary of OLS model
summary(freq_mod)
```

```
##
## Call:
## lm(formula = formula, data = df_final_scaled)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.3669 -0.3596  0.0522  0.3877  5.5354
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -1.267e-16  5.887e-02   0.000 1.000000
## lag_gdp_growth -1.662e-01  1.052e-01  -1.580 0.115343
## lag_unrate      2.611e-01  7.545e-02   3.461 0.000630 ***
## lag_inflation  -2.160e-01  6.337e-02  -3.408 0.000758 ***
## lag_ip_growth   2.941e-01  1.051e-01   2.800 0.005505 **
## lag_spread     -9.243e-02  7.835e-02  -1.180 0.239173
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9547 on 257 degrees of freedom
## Multiple R-squared:  0.106,  Adjusted R-squared:  0.0886
## F-statistic: 6.094 on 5 and 257 DF,  p-value: 2.359e-05
```

## OLS Model Diagnostics

### Heteroskedasticity

```
# Plot residuals
plot(freq_mod, which = 1)
```

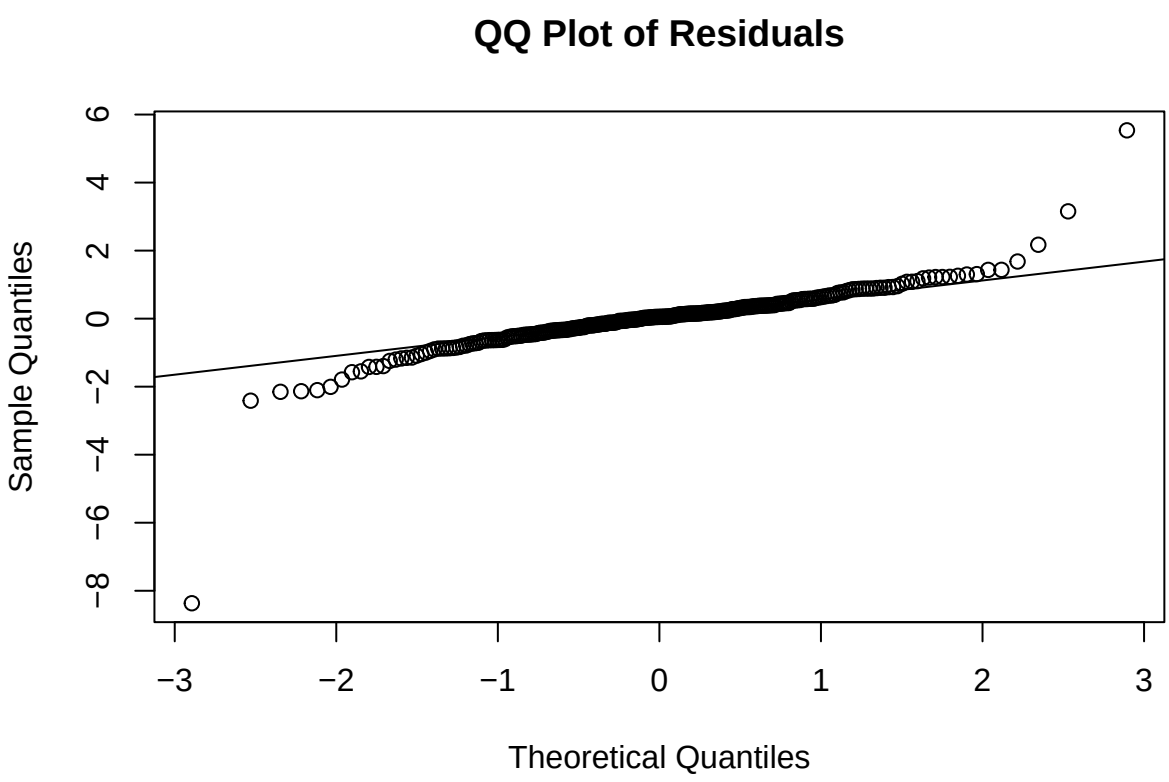



```
# Perform BP-Test on model
bptest(freq_mod)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  freq_mod
## BP = 34.578, df = 5, p-value = 1.827e-06
```

**Normality of Residuals**

```
# Inspect normality of residuals
qqnorm(freq_mod$residuals, main = "QQ Plot of Residuals")
qqline(freq_mod$residuals)
```




```
# Inspect normality of residuals with Shapiro-Wilk Test
shapiro.test(resid(freq_mod))
```

```
##
##  Shapiro-Wilk normality test
##
## data:  resid(freq_mod)
## W = 0.7812, p-value < 2.2e-16
```

### Remarks

Residual diagnostics suggest mild deviations from normality, particularly in the tails, and evidence of heteroskedasticity. Given the nature of macroeconomic data, these findings are expected. To address potential heteroskedasticity, robust standard errors were employed in inference.

### Robust Standard Errors

```
# Robust Standard Errors to deal with heteroskedasticity
coeftest(freq_mod, vcov = vcovHC(freq_mod, type = "HC1"))
```

```
##
## t test of coefficients:
```

```
##
##                   Estimate  Std. Error t value Pr(>|t|)
## (Intercept)    -1.2670e-16  5.8868e-02  0.0000 1.000000
## lag_gdp_growth -1.6617e-01  1.8132e-01 -0.9165 0.360283
## lag_unrate      2.6113e-01  1.4643e-01  1.7833 0.075715 .
## lag_inflation  -2.1601e-01  1.0789e-01 -2.0022 0.046314 *
## lag_ip_growth   2.9413e-01  1.0667e-01  2.7575 0.006243 **
## lag_spread     -9.2432e-02  1.2030e-01 -0.7684 0.442967
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

### Probability of Negative GDP Growth

For this part, we want to compute:

$$P(Y_t < 0 \mid X_t)$$

Where $Y_t$ is GDP growth. We will do this with unscaled data.

Our model implies:

$$Y_t \approx N(\hat{\mu}_t, \hat{\sigma}^2)$$

Where $\hat{\mu}_t$ is the fitted regression value, and $\hat{\sigma}$ is the residual standard error.

From this, we have:

$$P(Y_t < 0) = \Phi\left(\frac{0 - \hat{\mu}_t}{\hat{\sigma}}\right)$$

Which we can use pnorm() to calculate.

```r
# Fit unscaled model
freq_mod_unscaled <- lm(formula, data = df_final)

# Make predictions with uncertainty
pred <- predict(freq_mod_unscaled, newdata = df_final, se.fit = TRUE)

# Store values to use for pnorm()
mu_hat <- pred$fit
se_mu <- pred$se.fit
sigma_hat <- sigma(freq_mod_unscaled)

# Calculate predictive SD for each observation
pred_sd <- sqrt(se_mu^2 + sigma_hat^2)

# Calculate probability of negative GDP growth
recession_prob <- pnorm(0, mean = mu_hat, sd = pred_sd)

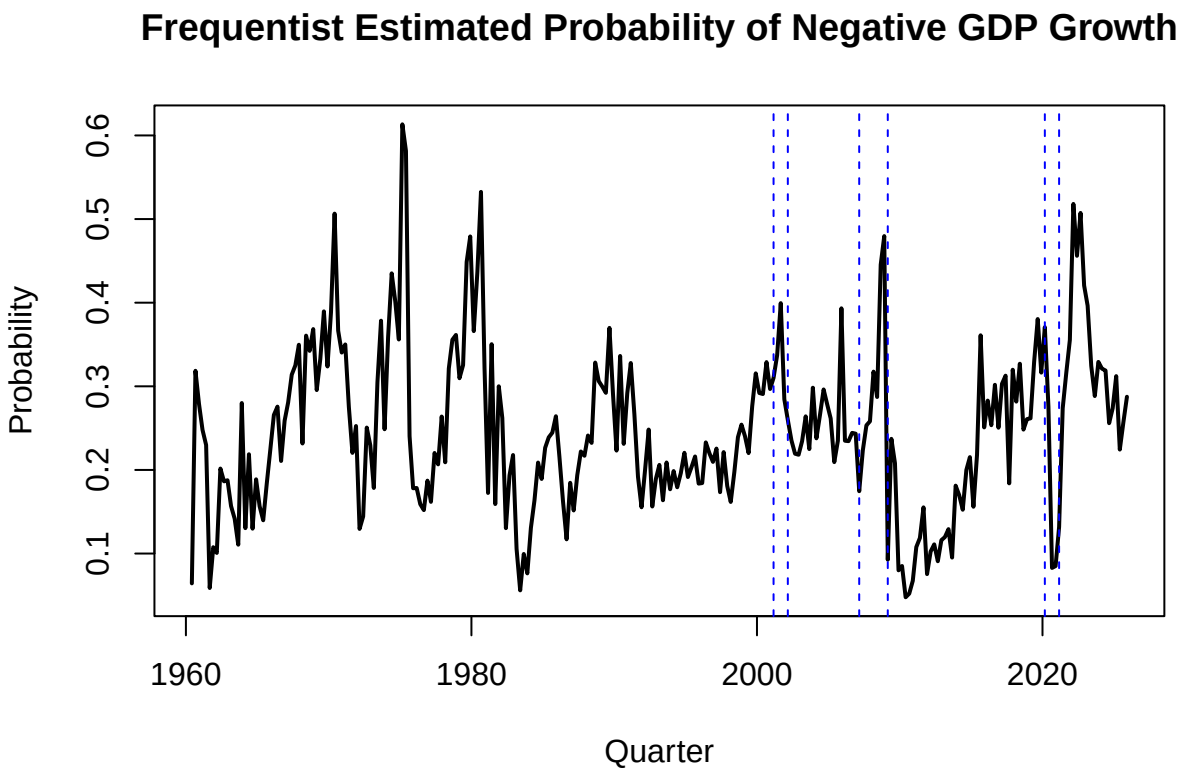
# Attach results to df
df_final$pred_gdp_growth <- mu_hat
df_final$pred_sd <- pred_sd
df_final$recession_prob <- recession_prob
```

```r
# Inspect results
head(df_final[, c("date", "gdp_growth", "pred_gdp_growth", "pred_sd", "recession_prob")])
```

```
##          date   gdp_growth pred_gdp_growth    pred_sd recession_prob
## 6  1960-06-01 -0.005398116     0.015730494 0.01035534     0.06437237
## 7  1960-09-01  0.004887350     0.004778231 0.01012910     0.31855869
## 8  1960-12-01 -0.012914344     0.005956220 0.01016284     0.27891138
## 9  1961-03-01  0.006727499     0.006913016 0.01014915     0.24789079
## 10 1961-06-01  0.016836022     0.007524789 0.01017526     0.22979612
## 11 1961-09-01  0.019021595     0.016016132 0.01023518     0.05881345
```

```r
# Plot probability over time
plot(df_final$date, df_final$recession_prob,
     type = "l",
     lwd = 2,
     main = "Frequentist Estimated Probability of Negative GDP Growth",
     xlab = "Quarter",
     ylab = "Probability")

# Add vertical lines for key quarters
abline(v = as.Date(c(
  "2001-03-01",
  "2002-03-01",
  "2007-03-01",
  "2009-03-01",
  "2020-03-01",
  "2021-03-01"
)), col = "blue", lty = 2)
```




# 4.) Bayesian Model

## Model Definition

We define our Bayesian model as such:

$$y_t \sim N(\mu_t, \sigma)$$

$$\mu_t = \beta_0 + \beta_1 X_{t-1}$$

We have weakly informative priors of:

$$\beta_j \sim N(0, 5)$$

## JAGS on Unscaled Data

### JAGS Setup

```
# Model String for Bayesian MLR
modelString <- "
model {
```

```
## sampling
for (i in 1:N) {
    mu[i] <- beta_0 +
             beta_gdp * lag_gdp[i] +
             beta_unrate * lag_unrate[i] +
             beta_inflation * lag_inflation[i] +
             beta_ip * lag_ip[i] +
             beta_spread * lag_spread[i]

  y[i] ~ dnorm(mu[i], tau)
}

## priors
beta_0          ~ dnorm(mu0, g0)
beta_gdp        ~ dnorm(mu1, g1)
beta_unrate     ~ dnorm(mu2, g2)
beta_inflation ~ dnorm(mu3, g3)
beta_ip         ~ dnorm(mu4, g4)
beta_spread     ~ dnorm(mu5, g5)
sigma ~ dt(mu_sigma, tau_sigma, nu_sigma) T(0, upper_sigma)
tau <- pow(sigma, -2)
}
"

# Pass data and hyperparameter values
y <- as.vector(df_final$gdp_growth)
lag_gdp <- as.vector(df_final$lag_gdp)
lag_unrate <- as.vector(df_final$lag_unrate)
lag_inflation <- as.vector(df_final$lag_inflation)
lag_ip <- as.vector(df_final$lag_ip)
lag_spread <- as.vector(df_final$lag_spread)

N <- length(y)

mu_sigma <- 0
nu_sigma <- 1
upper_sigma <- 1000000

the_data <- list(
  y = y,
  lag_gdp = lag_gdp,
  lag_unrate = lag_unrate,
  lag_inflation = lag_inflation,
  lag_ip = lag_ip,
  lag_spread = lag_spread,
  N = N,
  mu0 = 0, g0 = 1/25,
  mu1 = 0, g1 = 1/25,
  mu2 = 0, g2 = 1/25,
  mu3 = 0, g3 = 1/25,
  mu4 = 0, g4 = 1/25,
  mu5 = 0, g5 = 1/25,
  mu_sigma = 0,
```

```
  tau_sigma = 1/25,
  nu_sigma = 1,
  upper_sigma = 1e6
)

initsfunction <- function(chain){
  return(list(
    .RNG.seed = 100 + chain,
    .RNG.name = "base::Mersenne-Twister"
  ))
}

posterior_MLR <- run.jags(modelString,
                      n.chains = 3,
                      data = the_data,
                      monitor = c("beta_0", "beta_gdp", "beta_unrate",
                                  "beta_inflation", "beta_ip", "beta_spread",
                                  "sigma"),
                      adapt = 2000,
                      burnin = 10000,
                      sample = 10000,
                      thin = 50,
                      inits = initsfunction)
```

```
## Loading required namespace: rjags
```

```
## Compiling rjags model...
## Calling the simulation using the rjags method...
## Adapting the model for 2000 iterations...
## Burning in the model for 10000 iterations...
## Running the model for 500000 iterations...
## Simulation complete
## Calculating summary statistics...
## Calculating the Gelman-Rubin statistic for 7 variables....
## Finished running the simulation
```

```
# Print summary
summary(posterior_MLR)
```

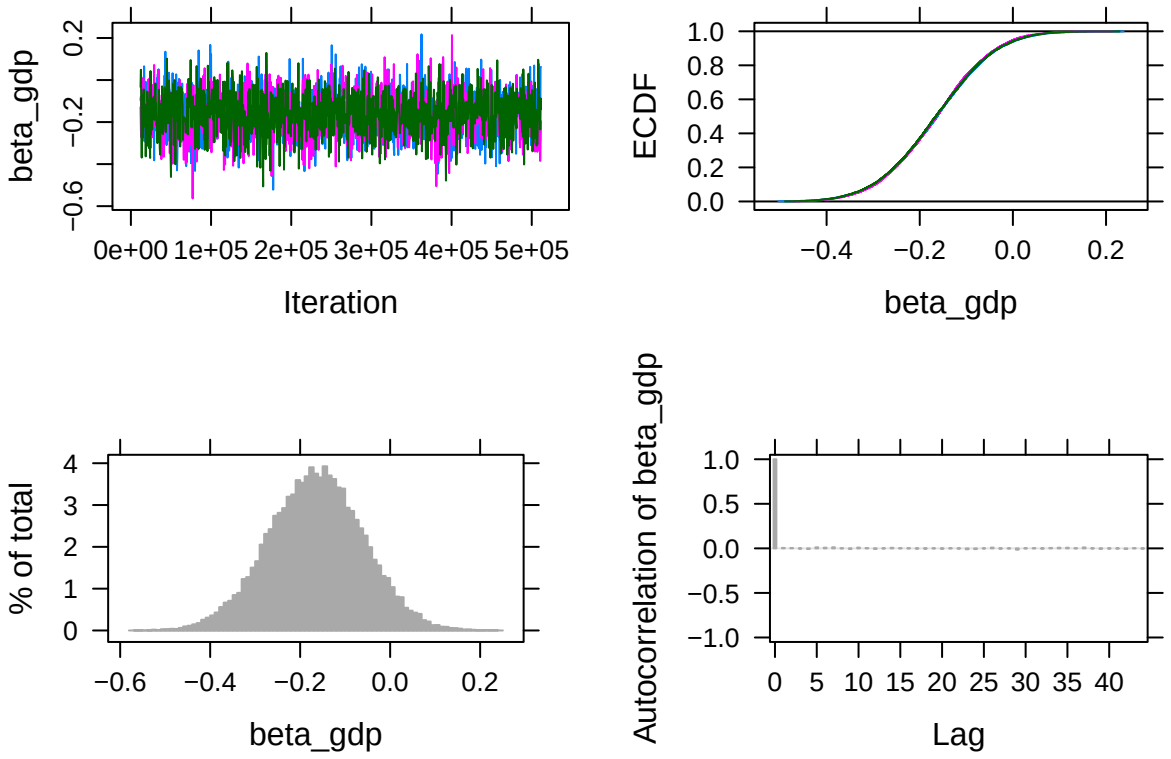
```
##                      Lower95        Median       Upper95          Mean
## beta_0         -0.0030075948  0.0018947461  0.0069432100  0.0018984567
## beta_gdp       -0.3788082239 -0.1642431346  0.0323771641 -0.1651323771
## beta_unrate     0.0007011709  0.0016266460  0.0025558191  0.0016280871
## beta_inflation -0.4793794474 -0.3034520678 -0.1289454552 -0.3032732321
## beta_ip         0.0529673316  0.1691843651  0.2902355924  0.1693454404
## beta_spread    -0.0021851241 -0.0007896818  0.0005048451 -0.0007944933
## sigma           0.0092423346  0.0100852346  0.0109945105  0.0101003806
##                          SD Mode        MCerr MC%ofSD SSeff         AC.500
## beta_0         0.0025482587   NA 1.519506e-05     0.6 28124  0.0043416843
## beta_gdp       0.1047466839   NA 6.047553e-04     0.6 30000  0.0085732302
## beta_unrate    0.0004739368   NA 2.827247e-06     0.6 28100 -0.0025035787
## beta_inflation 0.0898883346   NA 5.319438e-04     0.6 28555  0.0006299422
```

```
## beta_ip         0.0603542083   NA 3.461020e-04      0.6 30409  0.0066018292
## beta_spread     0.0006806333   NA 4.031468e-06      0.6 28504 -0.0009800786
## sigma           0.0004493674   NA 2.628875e-06      0.6 29219 -0.0131518580
##                      psrf
## beta_0          0.9999765
## beta_gdp        1.0002627
## beta_unrate     0.9999888
## beta_inflation 0.9999887
## beta_ip         1.0000742
## beta_spread     1.0001430
## sigma           1.0001074
```

### Diagnostic Plots

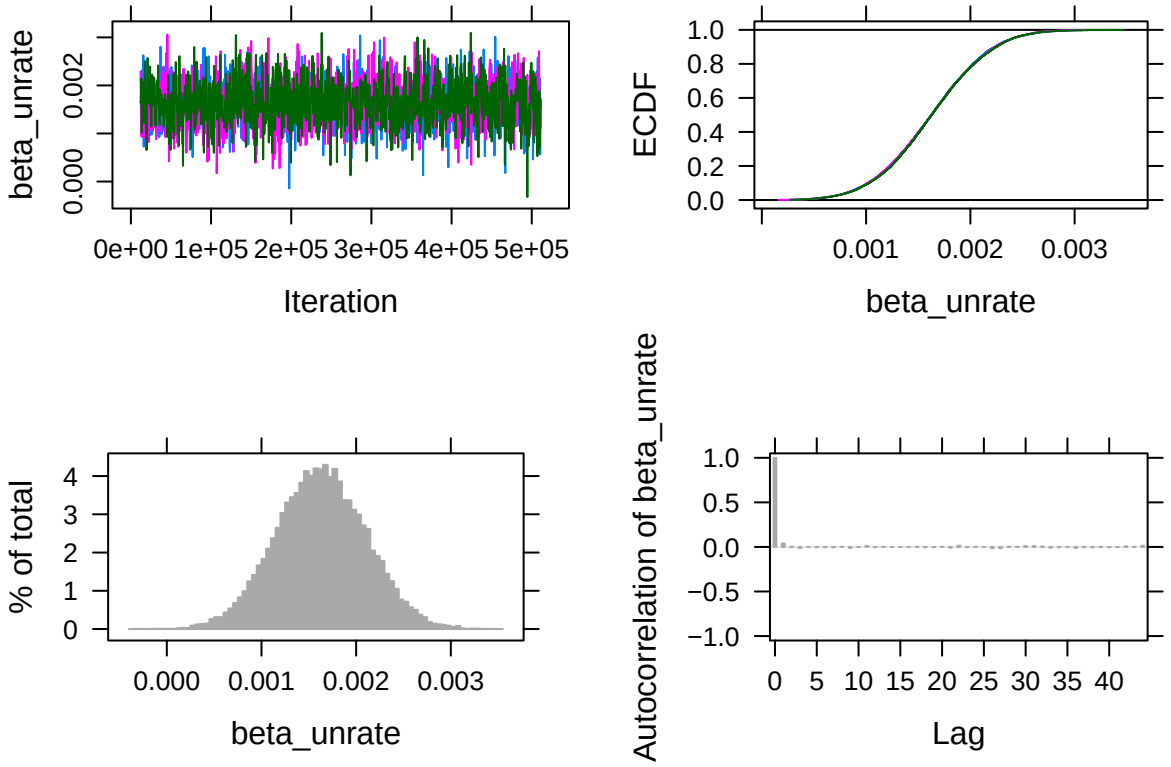
```
# Diagnostic plots for beta_gdp
plot(posterior_MLR, vars = "beta_gdp")
```

```
## Generating plots...
```



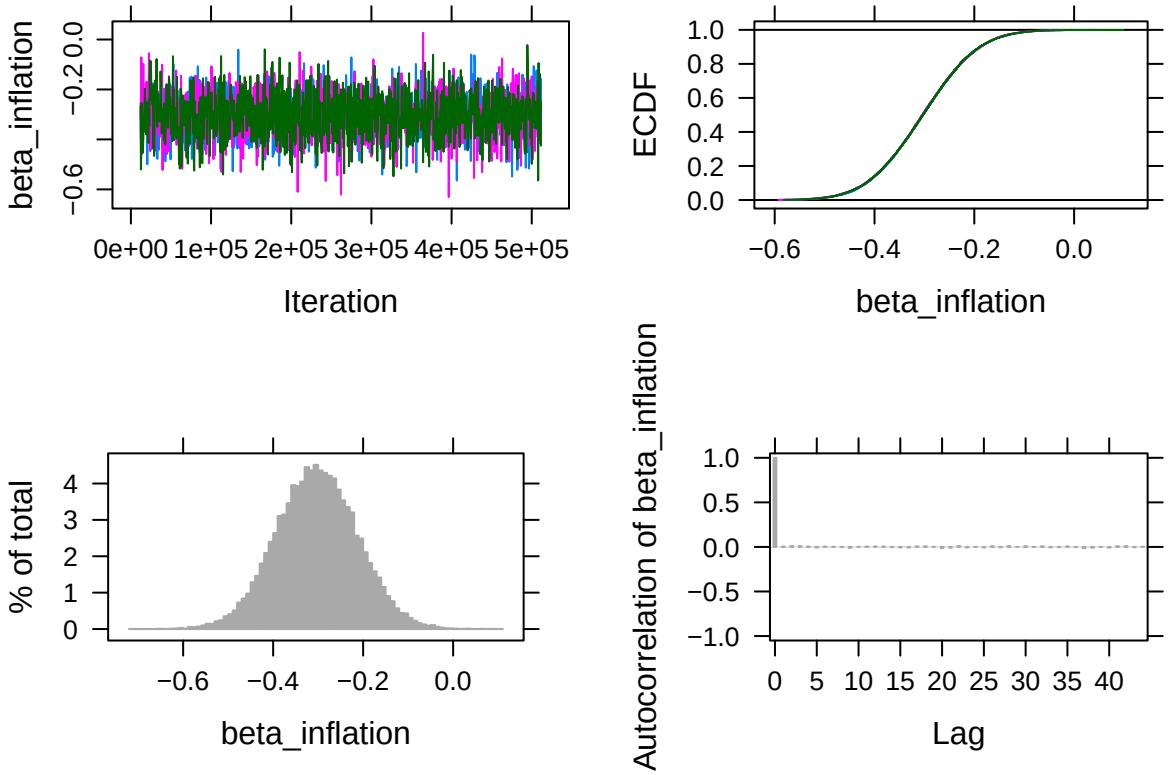

```
# Diagnostic plots for beta_unrate
plot(posterior_MLR, vars = "beta_unrate")
```

```
## Generating plots...
```



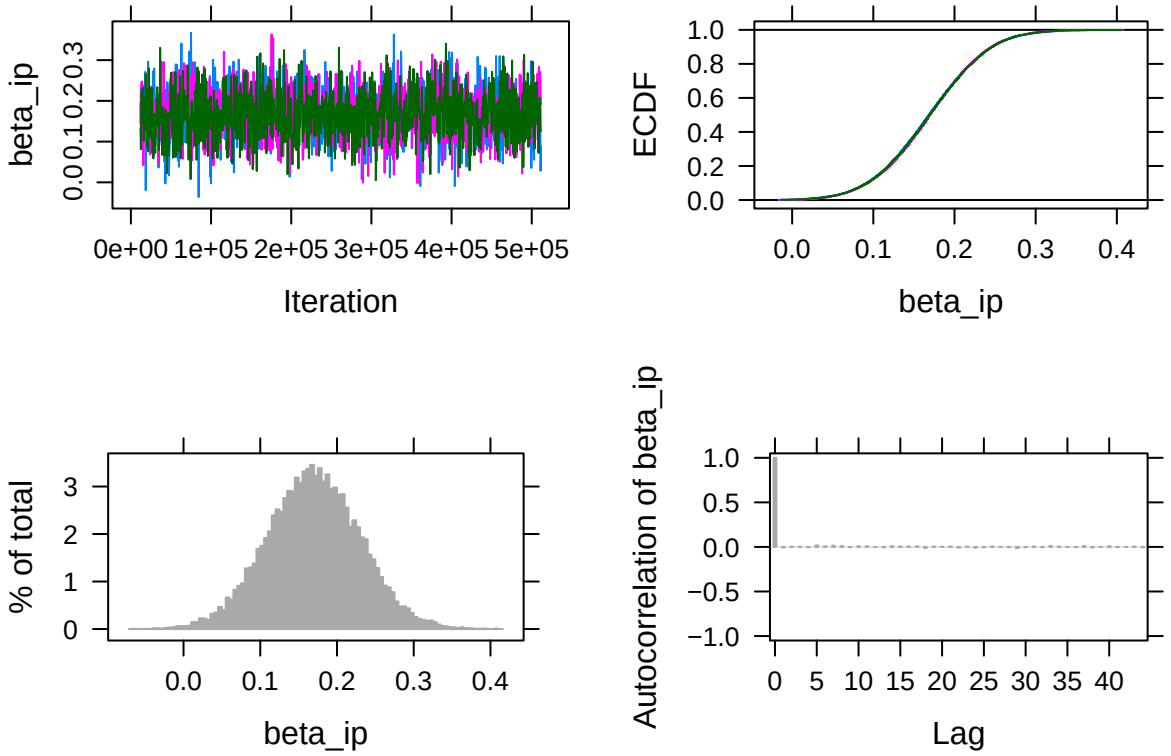

```
# Diagnostic plots for beta_inflation
plot(posterior_MLR, vars = "beta_inflation")
```

```
## Generating plots...
```



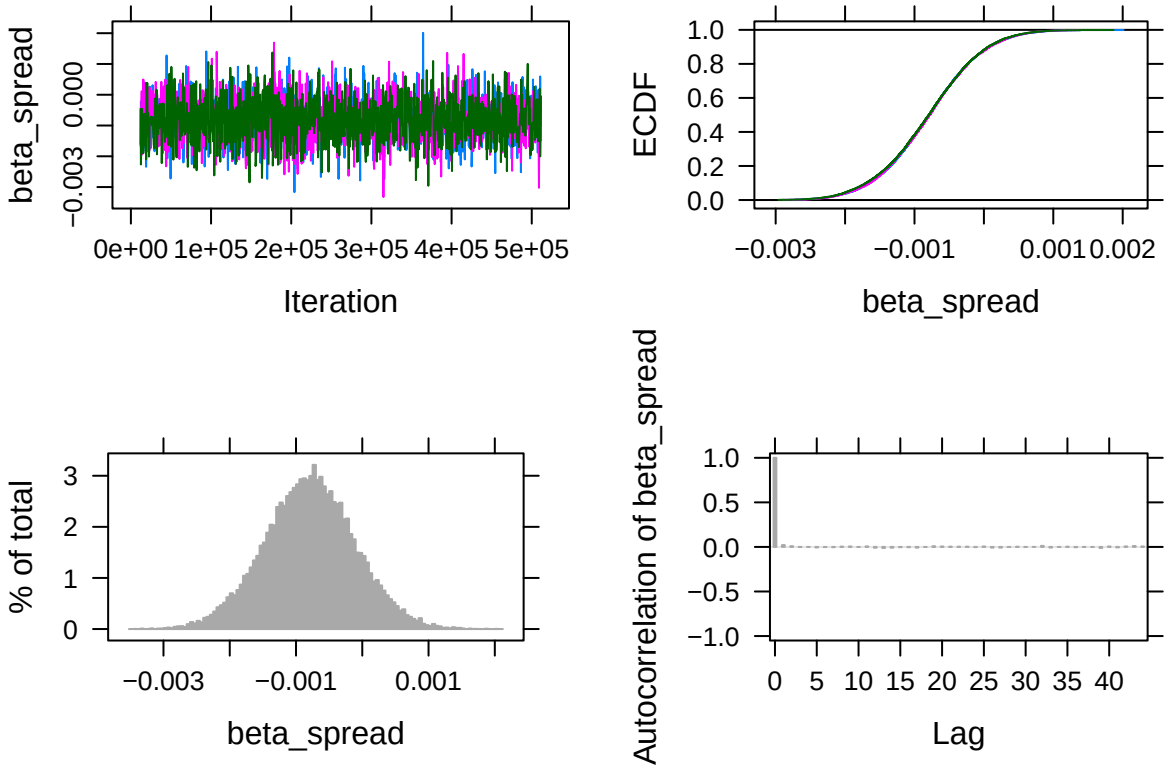

```
# Diagnostic plots for beta_ip
plot(posterior_MLR, vars = "beta_ip")
```

```
## Generating plots...
```



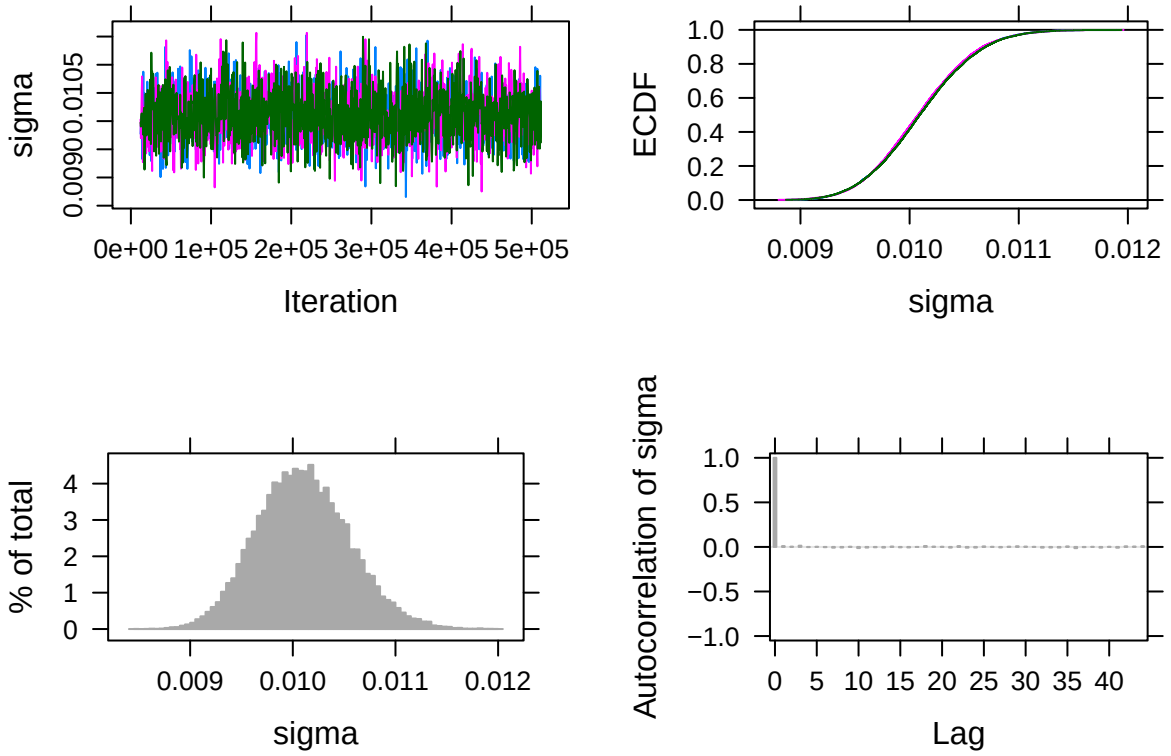

```
# Diagnostic plot for beta_spread
plot(posterior_MLR, vars = "beta_spread")
```

```
## Generating plots...
```




```
# Diagnostic plot for sigma
plot(posterior_MLR, vars = "sigma")
```

```
## Generating plots...
```




### Intereпretation and Inference

## Jags on Scaled Data

### JAGS Setup

```
# Model String for Bayesian MLR
modelStringScaled <- "
model {
## sampling
for (i in 1:N) {
    mu[i] <- beta_0 +
             beta_gdp * lag_gdp[i] +
             beta_unrate * lag_unrate[i] +
             beta_inflation * lag_inflation[i] +
             beta_ip * lag_ip[i] +
             beta_spread * lag_spread[i]

  y[i] ~ dnorm(mu[i], tau)
}

## priors
beta_0          ~ dnorm(mu0, g0)
beta_gdp       ~ dnorm(mu1, g1)
beta_unrate    ~ dnorm(mu2, g2)
```

```
beta_inflation ~ dnorm(mu3, g3)
beta_ip        ~ dnorm(mu4, g4)
beta_spread    ~ dnorm(mu5, g5)
sigma ~ dt(mu_sigma, tau_sigma, nu_sigma) T(0, upper_sigma)
tau <- pow(sigma, -2)
}
"

# Pass data and hyperparameter values
y <- as.vector(df_final_scaled$gdp_growth)
lag_gdp <- as.vector(df_final_scaled$lag_gdp)
lag_unrate <- as.vector(df_final_scaled$lag_unrate)
lag_inflation <- as.vector(df_final_scaled$lag_inflation)
lag_ip <- as.vector(df_final_scaled$lag_ip)
lag_spread <- as.vector(df_final_scaled$lag_spread)

N <- length(y)

mu_sigma <- 0
nu_sigma <- 1
upper_sigma <- 1000000

the_data <- list(
  y = y,
  lag_gdp = lag_gdp,
  lag_unrate = lag_unrate,
  lag_inflation = lag_inflation,
  lag_ip = lag_ip,
  lag_spread = lag_spread,
  N = N,
  mu0 = 0, g0 = 1,
  mu1 = 0, g1 = 1,
  mu2 = 0, g2 = 1,
  mu3 = 0, g3 = 1,
  mu4 = 0, g4 = 1,
  mu5 = 0, g5 = 1,
  mu_sigma = 0,
  tau_sigma = 1,
  nu_sigma = 1,
  upper_sigma = 1e6
)

initsfunction <- function(chain){
  return(list(
    .RNG.seed = 100 + chain,
    .RNG.name = "base::Mersenne-Twister"
  ))
}

posterior_MLR_scaled <- run.jags(modelStringScaled,
                      n.chains = 3,
                      data = the_data,
                      monitor = c("beta_0", "beta_gdp", "beta_unrate",
```

```
                                  "beta_inflation", "beta_ip", "beta_spread",
                                  "sigma"),
                      adapt = 2000,
                      burnin = 10000,
                      sample = 10000,
                      thin = 10,
                      inits = initsfunction)
```

```
## Compiling rjags model...
## Calling the simulation using the rjags method...
## Adapting the model for 2000 iterations...
## Burning in the model for 10000 iterations...
## Running the model for 100000 iterations...
## Simulation complete
## Calculating summary statistics...
## Calculating the Gelman-Rubin statistic for 7 variables....
## Finished running the simulation
```

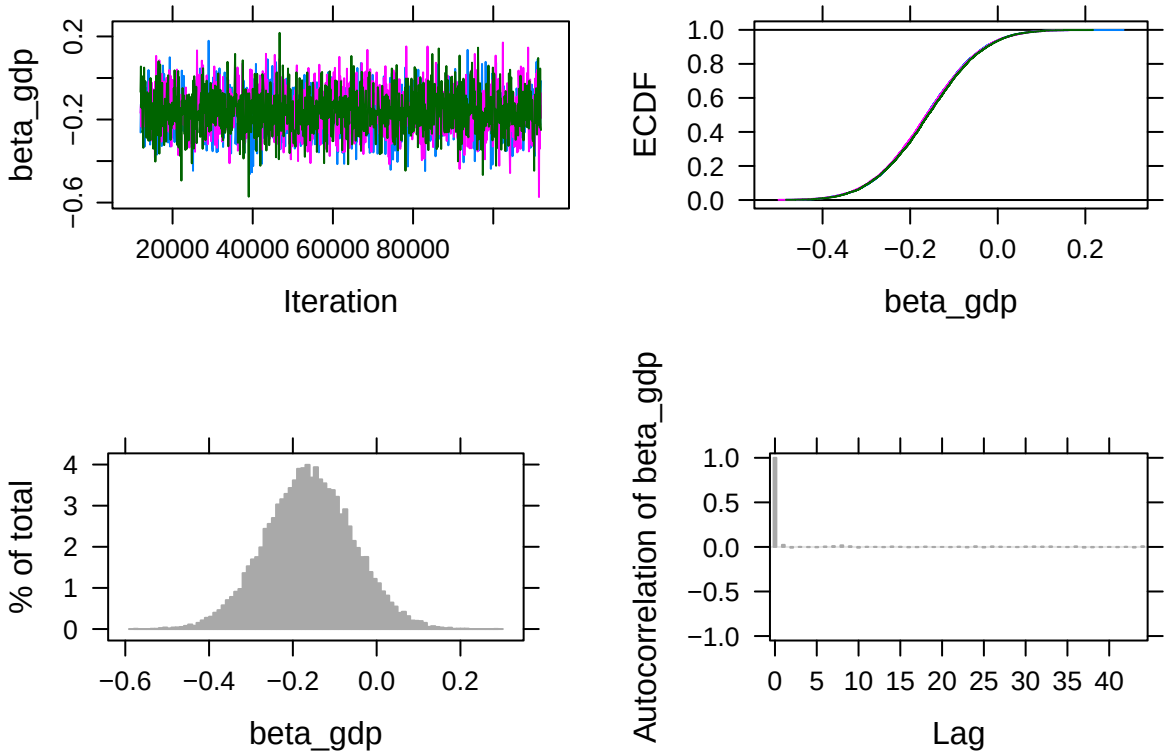
```
# Print summary
summary(posterior_MLR_scaled)
```

```
##                    Lower95        Median     Upper95          Mean         SD
## beta_0         -0.11584311  0.0004792363  0.11285329 -0.0001272755 0.05885374
## beta_gdp       -0.36829613 -0.1613105992  0.04169042 -0.1608660732 0.10420397
## beta_unrate     0.11250593  0.2593633718  0.40906461  0.2589374590 0.07566253
## beta_inflation -0.33925733 -0.2141718180 -0.09195658 -0.2143186959 0.06343057
## beta_ip         0.08074376  0.2887935926  0.49074473  0.2883822089 0.10449362
## beta_spread    -0.24003538 -0.0900833721  0.06377852 -0.0902198396 0.07770972
## sigma           0.87584635  0.9563171954  1.04027062  0.9575355010 0.04211723
##                   Mode        MCerr MC%ofSD SSeff        AC.100      psrf
## beta_0              NA 0.0003403555     0.6 29901 -0.0029261428 0.9999544
## beta_gdp            NA 0.0006132209     0.6 28876 -0.0088122083 1.0001245
## beta_unrate         NA 0.0004368378     0.6 30000  0.0009449811 1.0001659
## beta_inflation      NA 0.0003662166     0.6 30000  0.0018484662 0.9999994
## beta_ip             NA 0.0006082232     0.6 29516  0.0008222177 1.0000818
## beta_spread         NA 0.0004464469     0.6 30298  0.0014037538 1.0000228
## sigma               NA 0.0002466825     0.6 29150  0.0001320699 0.9999700
```

### Diagnostic Plots

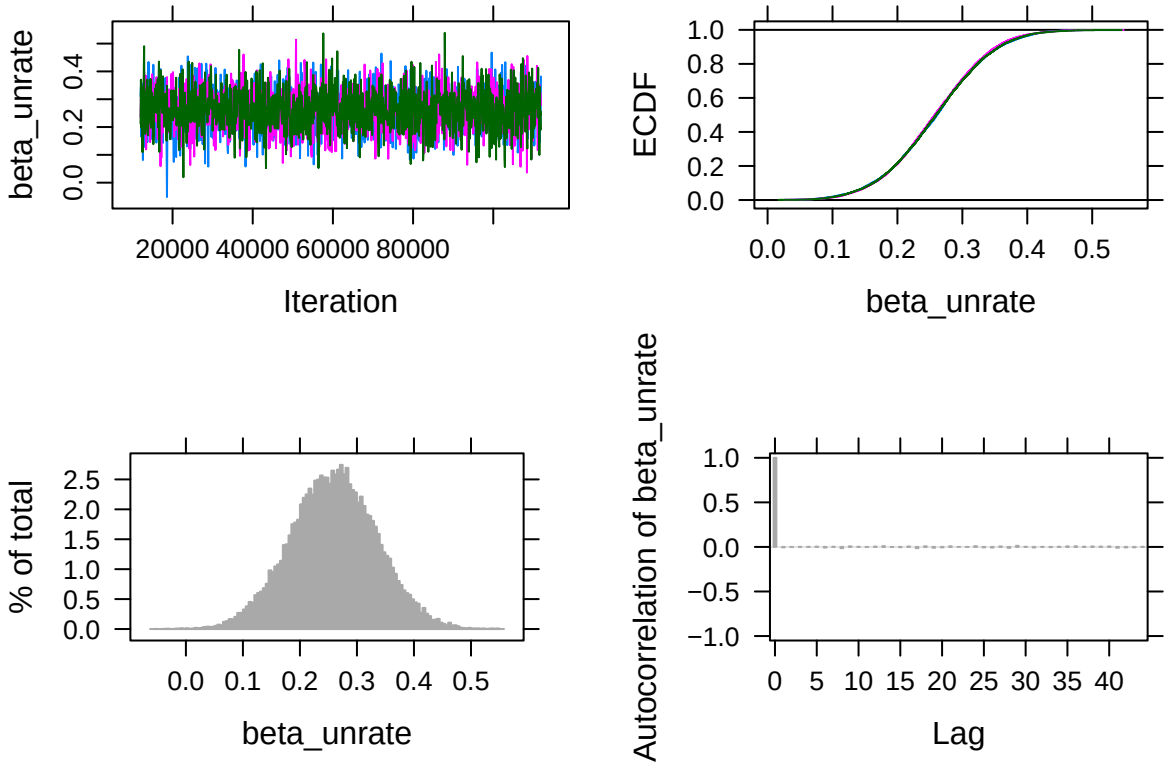
```
# Diagnostic plots for beta_gdp
plot(posterior_MLR_scaled, vars = "beta_gdp")
```

```
## Generating plots...
```



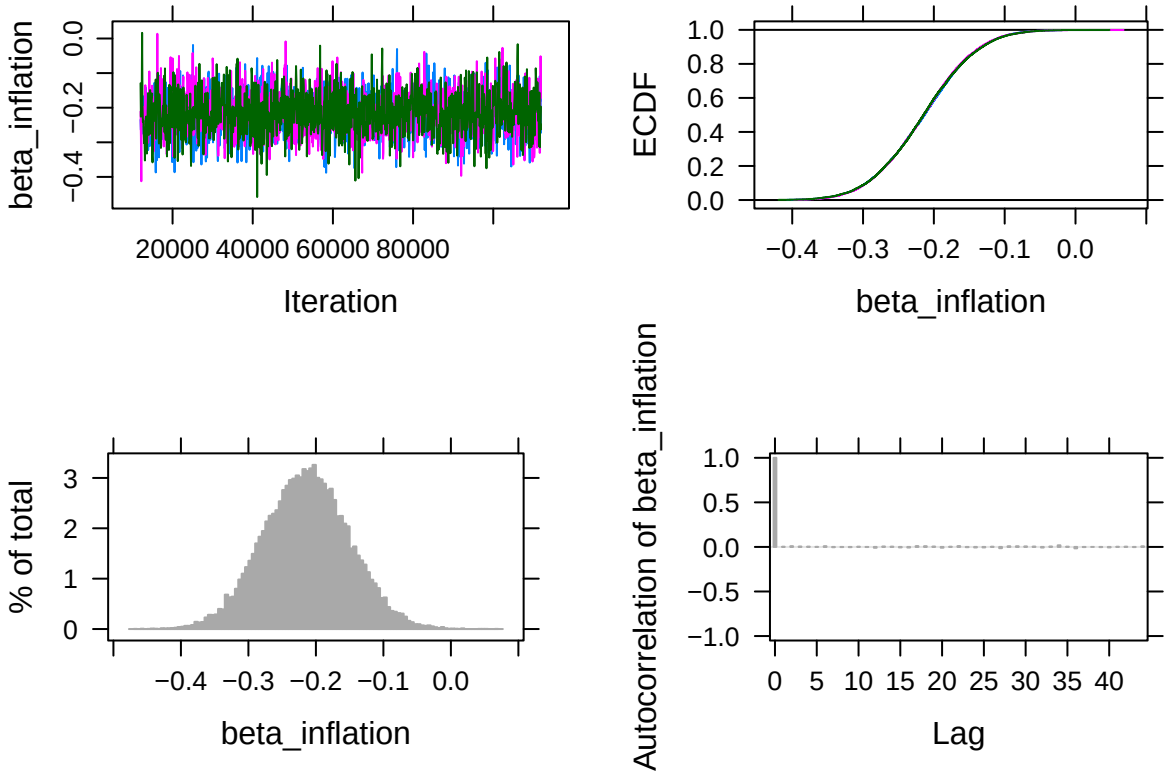

```
# Diagnostic plots for beta_unrate
plot(posterior_MLR_scaled, vars = "beta_unrate")
```

```
## Generating plots...
```



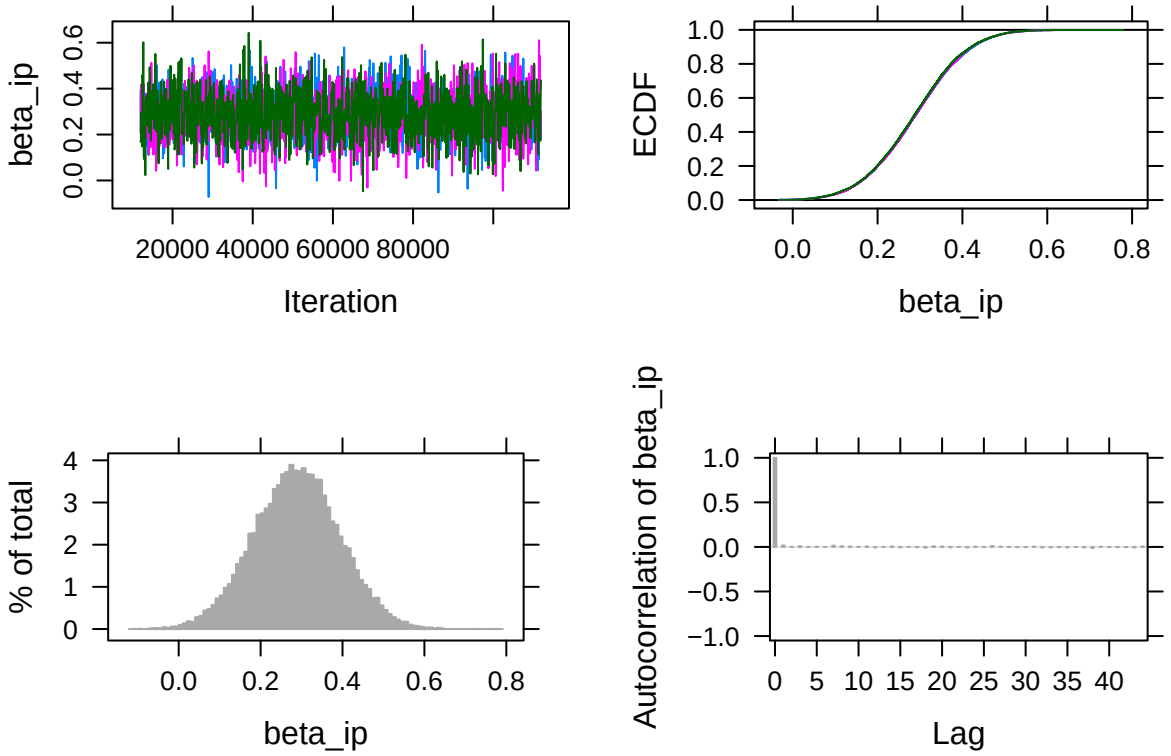

```
# Diagnostic plots for beta_inflation
plot(posterior_MLR_scaled, vars = "beta_inflation")
```

```
## Generating plots...
```



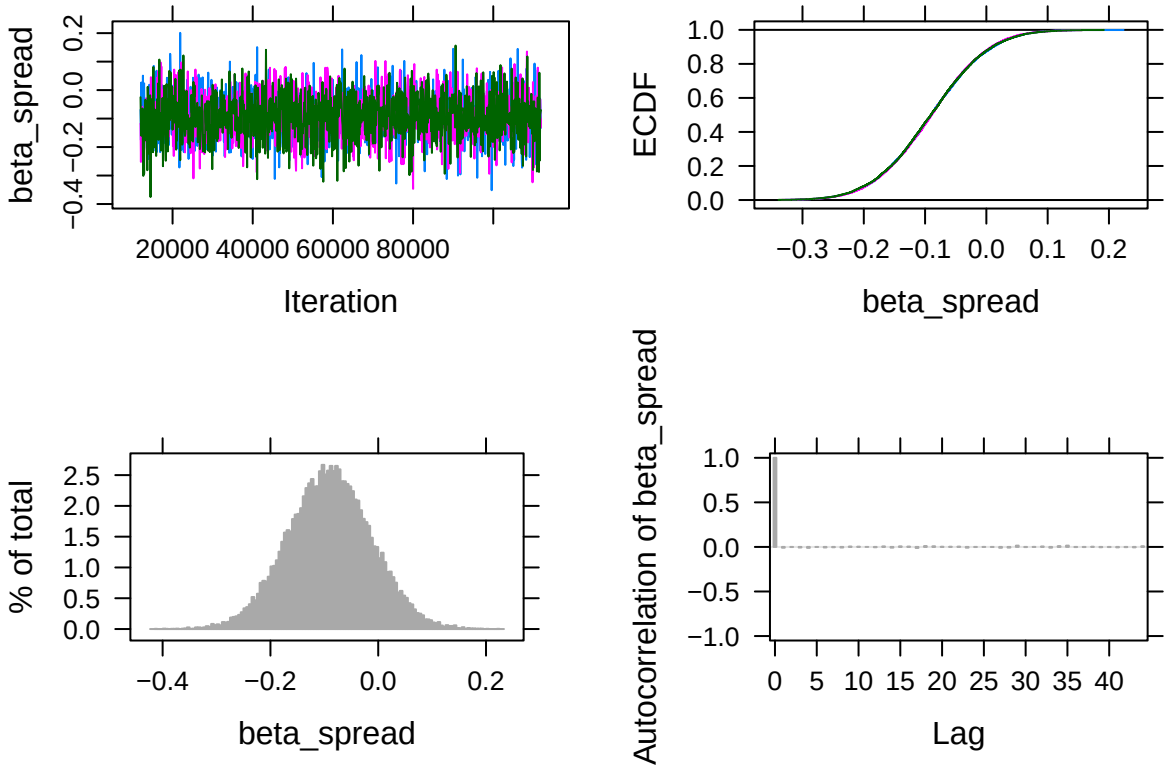

```
# Diagnostic plots for beta_ip
plot(posterior_MLR_scaled, vars = "beta_ip")
```

```
## Generating plots...
```



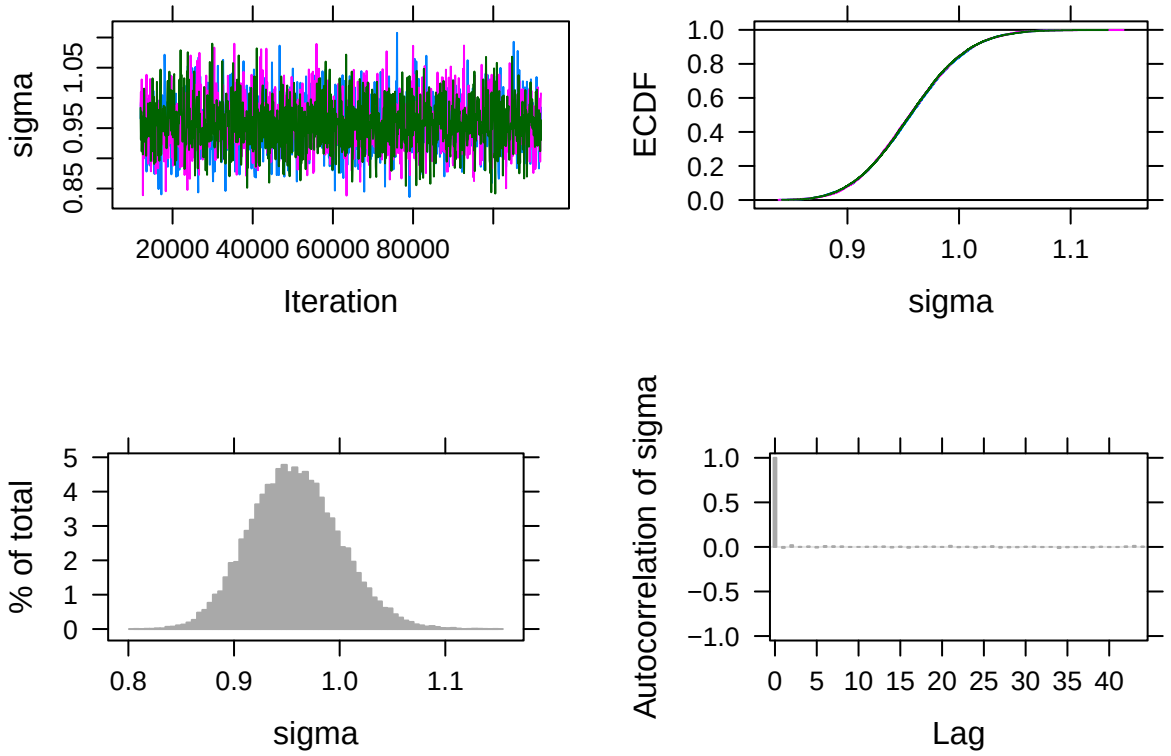

```
# Diagnostic plot for beta_spread
plot(posterior_MLR_scaled, vars = "beta_spread")
```

```
## Generating plots...
```




```
# Diagnostic plot for sigma
plot(posterior_MLR_scaled, vars = "sigma")
```

```
## Generating plots...
```




**Interpretation and inference**

For both of our JAGS simulations, each of our monitored parameters seemingly converges to a stable distribution.

## Probability of Negative GDP Growth (with unscaled data)

```
# Convert posterior results to matrix
post <- as.matrix(as.mcmc(posterior_MLR))
```

```
## Warning in as.mcmc.runjags(posterior_MLR): Combining the 3 mcmc chains together
```

```
# Extract predictor values from data
X <- model.matrix(~ lag_gdp_growth + lag_unrate + lag_inflation +
                    lag_ip_growth + lag_spread, data = df_final)

# Create vector of coeffecient draws
beta_draws <- cbind(post[, "beta_0"],
                    post[, "beta_gdp"],
                    post[, "beta_unrate"],
                    post[, "beta_inflation"],
                    post[, "beta_ip"],
```

```
                    post[, "beta_spread"])

# Calculate mu response
mu_post <- beta_draws %*% t(X)

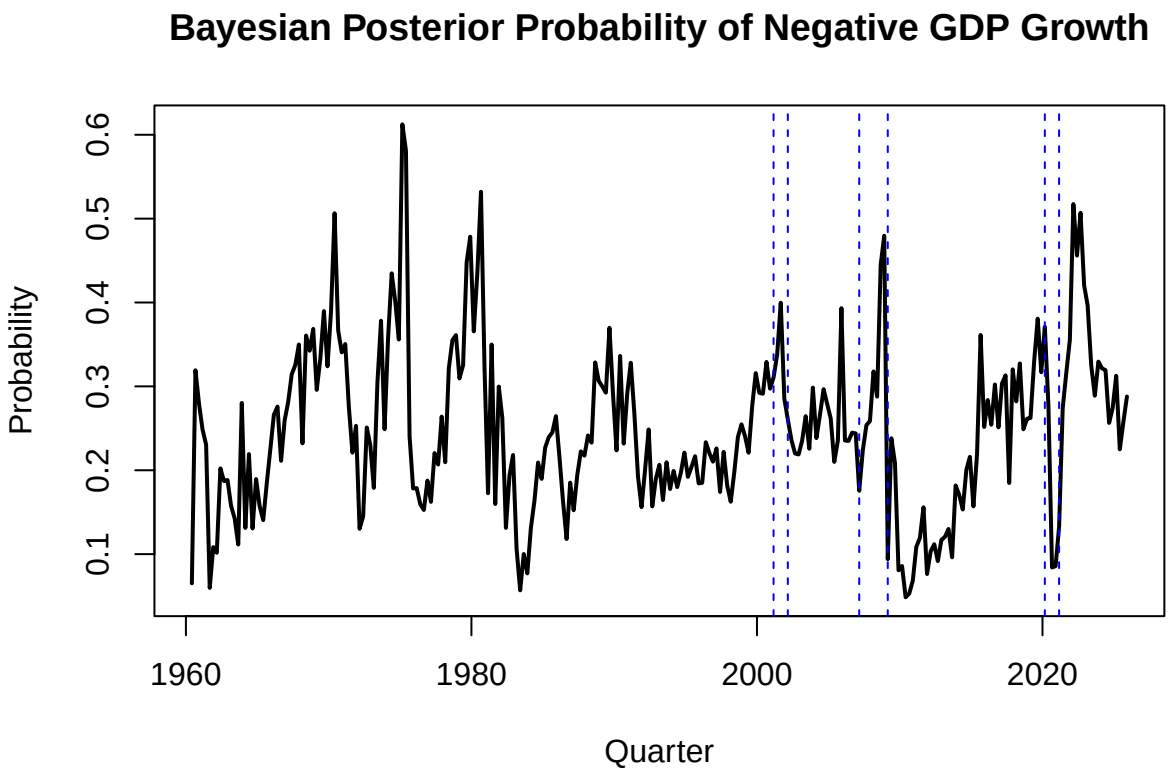
# Create posterior sigma draws vector
sigma_post <- post[, "sigma"]

# Calculate probability of negative GDP growth
bayes_prob_neg <- sapply(1:ncol(mu_post), function(j) {
  mean(pnorm(0, mean = mu_post[, j], sd = sigma_post))
})

# Store results in dataframe
df_final$bayes_prob_neg <- bayes_prob_neg
```

```
# Plot probability over time
plot(df_final$date, df_final$bayes_prob_neg, type="l", lwd=2,
     main="Bayesian Posterior Probability of Negative GDP Growth",
     xlab="Quarter", ylab="Probability")

# Add vertical lines for key quarters
abline(v = as.Date(c(
  "2001-03-01",
  "2002-03-01",
  "2007-03-01",
  "2009-03-01",
  "2020-03-01",
  "2021-03-01"
)), col = "blue", lty = 2)
```




## 5.) Full Model Comparison

### Coefficient Estimates

```
# Obtain frequentist robust results from scaled model
robust_vcov <- vcovHC(freq_mod, type = "HC1")
robust_test <- coeftest(freq_mod, vcov = robust_vcov)

# Extract estimates, se, p-values, and interval bounds
freq_est <- robust_test[, "Estimate"]
freq_se  <- robust_test[, "Std. Error"]
freq_p   <- robust_test[, "Pr(>|t|)"]
freq_lower <- freq_est - 1.96 * freq_se
freq_upper <- freq_est + 1.96 * freq_se

# Convert Bayesian posterior draws to matrix
post_scaled <- as.matrix(as.mcmc(posterior_MLR_scaled))
```

```
## Warning in as.mcmc.runjags(posterior_MLR_scaled): Combining the 3 mcmc chains
## together
```

```r
# Define vector of param names
bayes_params <- c("beta_0", "beta_gdp", "beta_unrate",
                  "beta_inflation", "beta_ip", "beta_spread")

# Calculate mean, lower, and upper bounds from Bayesian Posterior draws
bayes_mean  <- apply(post_scaled[, bayes_params], 2, mean)
bayes_lower <- apply(post_scaled[, bayes_params], 2, quantile, probs = 0.025)
bayes_upper <- apply(post_scaled[, bayes_params], 2, quantile, probs = 0.975)

# Construct table comparing frequentist and Bayesian results
comparison_table <- data.frame(
  Variable = c("(Intercept)", "lag_gdp_growth", "lag_unrate",
               "lag_inflation", "lag_ip_growth", "lag_spread"),
  Freq_Estimate = as.numeric(freq_est),
  Freq_Robust_SE = as.numeric(freq_se),
  Freq_95CI_Lower = as.numeric(freq_lower),
  Freq_95CI_Upper = as.numeric(freq_upper),
  Freq_pvalue = as.numeric(freq_p),
  Bayes_Mean = c(bayes_mean["beta_0"],
                 bayes_mean["beta_gdp"],
                 bayes_mean["beta_unrate"],
                 bayes_mean["beta_inflation"],
                 bayes_mean["beta_ip"],
                 bayes_mean["beta_spread"]),
  Bayes_95CrI_Lower = c(bayes_lower["beta_0"],
                        bayes_lower["beta_gdp"],
                        bayes_lower["beta_unrate"],
                        bayes_lower["beta_inflation"],
                        bayes_lower["beta_ip"],
                        bayes_lower["beta_spread"]),
  Bayes_95CrI_Upper = c(bayes_upper["beta_0"],
                        bayes_upper["beta_gdp"],
                        bayes_upper["beta_unrate"],
                        bayes_upper["beta_inflation"],
                        bayes_upper["beta_ip"],
                        bayes_upper["beta_spread"])
)

# Display table
knitr::kable(comparison_table,
             digits = 4,
             caption = "Frequentist and Bayesian coefficient comparison (scaled data)")
```

Table 1: Frequentist and Bayesian coefficient comparison (scaled data)

| | Variable | Freq_Estimate | Freq_Robust_SE | Freq_95CI_Lower | Freq_95CI_Upper | Freq_pvalue | Bayes_Mean | Bayes_95CrI_Lower | Bayes_95CrI_Upper |
|---|---|---|---|---|---|---|---|---|---|
| beta_0 | (Intercept) | 0.0000 | 0.0589 | -0.1154 | 0.1154 | 1.0000 | -0.0001 | -0.1148 | 0.1144 |
| beta_gdp | lag_gdp_growth | -0.1662 | 0.1813 | -0.5216 | 0.1892 | 0.3603 | -0.1609 | -0.3662 | 0.0438 |
| beta_unrate | lag_unrate | 0.2611 | 0.1464 | -0.0259 | 0.5481 | 0.0757 | 0.2589 | 0.1100 | 0.4074 |

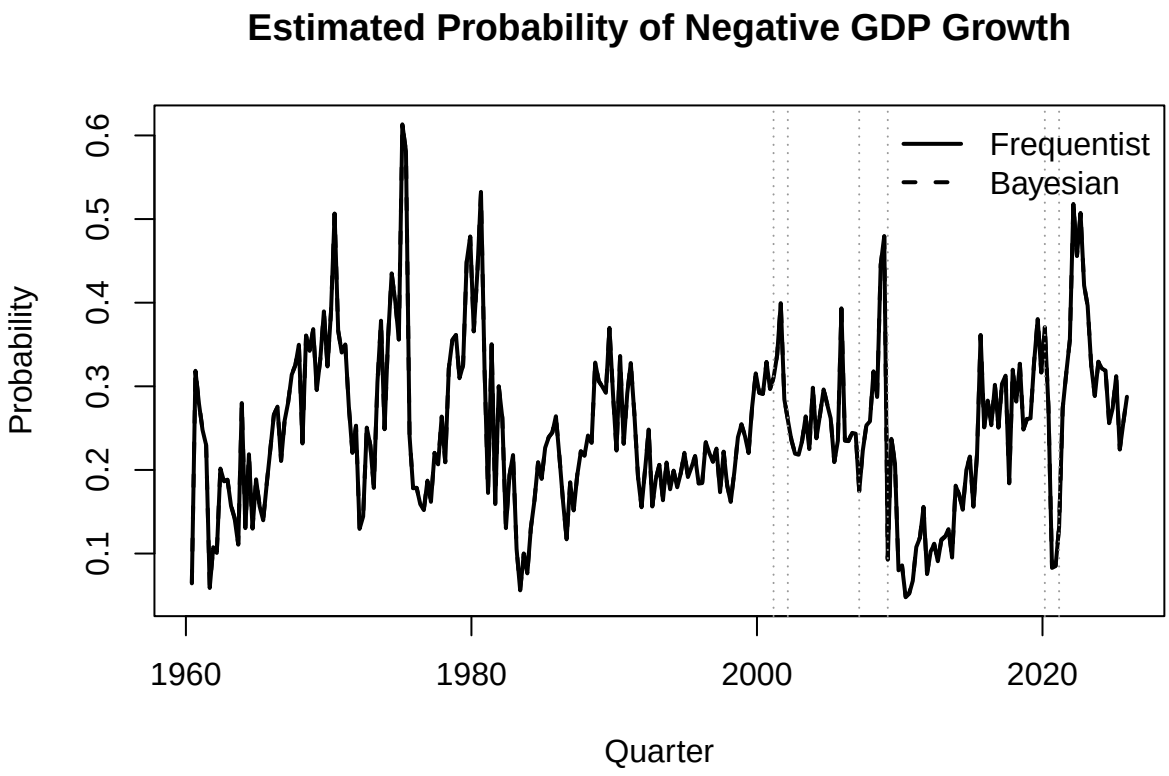
| | Variable | Freq_Estimate | Freq_Robust_SE | Freq_95CI_Lower | Freq_95CI_Upper | Freq_p_value | Bayes_Mean | Bayes_95CrI_Lower | Bayes_95CrI_Upper |
|---|---|---|---|---|---|---|---|---|---|
| beta_inflation | lag_inflation | -0.2160 | 0.1079 | -0.4275 | -0.0046 | 0.0463 | -0.2143 | -0.3377 | -0.0901 |
| beta_ip | lag_ip_growth | 0.2941 | 0.1067 | 0.0851 | 0.5032 | 0.0062 | 0.2884 | 0.0828 | 0.4933 |
| beta_spread | lag_spread | -0.0924 | 0.1203 | -0.3282 | 0.1433 | 0.4430 | -0.0902 | -0.2427 | 0.0618 |

## Recession Probabilities

```
# Plot frequentist vs. Bayesian recession probabilities
plot(df_final$date, df_final$recession_prob,
     type = "l", lwd = 2,
     ylim = range(c(df_final$recession_prob, df_final$bayes_prob_neg), na.rm = TRUE),
     main = "Estimated Probability of Negative GDP Growth",
     xlab = "Quarter",
     ylab = "Probability")

# Insert vertical lines corresponding to recession periods
lines(df_final$date, df_final$bayes_prob_neg, lwd = 2, lty = 2)
abline(v = as.Date(c("2001-03-01", "2002-03-01",
                     "2007-03-01", "2009-03-01",
                     "2020-03-01", "2021-03-01")),
       col = "gray60", lty = 3)

# Define legend
legend("topright",
       legend = c("Frequentist", "Bayesian"),
       lwd = 2,
       lty = c(1, 2),
       bty = "n")
```




```r
# Compute correlation between frequentist and Bayesian probabilities
prob_correlation <- cor(df_final$recession_prob, df_final$bayes_prob_neg, use = "complete.obs")
mean_abs_diff <- mean(abs(df_final$recession_prob - df_final$bayes_prob_neg), na.rm = TRUE)

# Create summary table with correlation and MAD
prob_summary <- data.frame(
  Metric = c("Correlation", "Mean Absolute Difference"),
  Value = c(prob_correlation, mean_abs_diff)
)

# Display table
knitr::kable(prob_summary,
             digits = 4,
             caption = "Comparison of frequentist and Bayesian recession probabilities")
```

Table 2: Comparison of frequentist and Bayesian recession probabilities

| Metric | Value |
|---|---|
| Correlation | 1e+00 |
| Mean Absolute Difference | 5e-04 |

### Predictive Fit

```
# Compute frequentist fit metrics
freq_rmse <- sqrt(mean((df_final$gdp_growth - df_final$pred_gdp_growth)^2, na.rm = TRUE))
freq_mae  <- mean(abs(df_final$gdp_growth - df_final$pred_gdp_growth), na.rm = TRUE)

# Compute Bayesian posterior mean fitted values
post_unscaled <- as.matrix(as.mcmc(posterior_MLR))
```

```
## Warning in as.mcmc.runjags(posterior_MLR): Combining the 3 mcmc chains together
```

```
# Extract unscaled predictor variables
X_unscaled <- model.matrix(~ lag_gdp_growth + lag_unrate + lag_inflation +
                             lag_ip_growth + lag_spread,
                           data = df_final)

# Put together unscaled draws
beta_draws_unscaled <- cbind(
  post_unscaled[, "beta_0"],
  post_unscaled[, "beta_gdp"],
  post_unscaled[, "beta_unrate"],
  post_unscaled[, "beta_inflation"],
  post_unscaled[, "beta_ip"],
  post_unscaled[, "beta_spread"]
)

# Compute mu and predicted means
mu_post_unscaled <- beta_draws_unscaled %*% t(X_unscaled)
bayes_pred_mean <- colMeans(mu_post_unscaled)

# Insert results into data
df_final$bayes_pred_gdp_growth <- bayes_pred_mean

# Calculate Bayesian fit metrics
bayes_rmse <- sqrt(mean((df_final$gdp_growth - df_final$bayes_pred_gdp_growth)^2, na.rm = TRUE))
bayes_mae  <- mean(abs(df_final$gdp_growth - df_final$bayes_pred_gdp_growth), na.rm = TRUE)

# Create comparison table for metrics
fit_comparison <- data.frame(
  Model = c("Frequentist", "Bayesian"),
  RMSE = c(freq_rmse, bayes_rmse),
  MAE  = c(freq_mae, bayes_mae)
)

# Display table
knitr::kable(fit_comparison,
             digits = 5,
             caption = "Predictive fit comparison on unscaled GDP growth")
```

Table 3: Predictive fit comparison on unscaled GDP growth

| Model | RMSE | MAE |
|---|---|---|
| Frequentist | 0.00994 | 0.00604 |
| Bayesian | 0.00994 | 0.00603 |

# 6.) Other Bayesian Models

## t-distributed error model

```
# Model string for t distributed errors
modelString_t <- "
model {

  for (i in 1:N) {
    mu[i] <- beta_0 +
             beta_gdp * lag_gdp[i] +
             beta_unrate * lag_unrate[i] +
             beta_inflation * lag_inflation[i] +
             beta_ip * lag_ip[i] +
             beta_spread * lag_spread[i]

    y[i] ~ dt(mu[i], tau, nu)
  }

  # Priors for coefficients
  beta_0         ~ dnorm(0, 1)
  beta_gdp       ~ dnorm(0, 1)
  beta_unrate    ~ dnorm(0, 1)
  beta_inflation ~ dnorm(0, 1)
  beta_ip        ~ dnorm(0, 1)
  beta_spread    ~ dnorm(0, 1)

  # Prior for sigma
  sigma ~ dunif(0, 10)
  tau <- pow(sigma, -2)

  # Prior for degrees of freedom
  nu_minus_two ~ dexp(1/30)
  nu <- nu_minus_two + 2
}
"

# Pass data and hyperparameter values
y <- as.vector(df_final_scaled$gdp_growth)
lag_gdp <- as.vector(df_final_scaled$lag_gdp_growth)
lag_unrate <- as.vector(df_final_scaled$lag_unrate)
lag_inflation <- as.vector(df_final_scaled$lag_inflation)
lag_ip <- as.vector(df_final_scaled$lag_ip_growth)
lag_spread <- as.vector(df_final_scaled$lag_spread)
```

```
N <- length(y)

the_data_t <- list(
  y = y,
  lag_gdp = lag_gdp,
  lag_unrate = lag_unrate,
  lag_inflation = lag_inflation,
  lag_ip = lag_ip,
  lag_spread = lag_spread,
  N = N
)

initsfunction_t <- function(chain) {
  list(
    .RNG.seed = 200 + chain,
    .RNG.name = "base::Mersenne-Twister",
    sigma = 1,
    nu_minus_two = 10
  )
}

posterior_t <- run.jags(
  model = modelString_t,
  n.chains = 3,
  data = the_data_t,
  monitor = c("beta_0", "beta_gdp", "beta_unrate",
              "beta_inflation", "beta_ip", "beta_spread",
              "sigma", "nu"),
  adapt = 2000,
  burnin = 10000,
  sample = 10000,
  thin = 10,
  inits = initsfunction_t
)
```

```
## Compiling rjags model...
## Calling the simulation using the rjags method...
## Adapting the model for 2000 iterations...
## Burning in the model for 10000 iterations...
## Running the model for 100000 iterations...
## Simulation complete
## Calculating summary statistics...
## Calculating the Gelman-Rubin statistic for 8 variables....
## Finished running the simulation
```

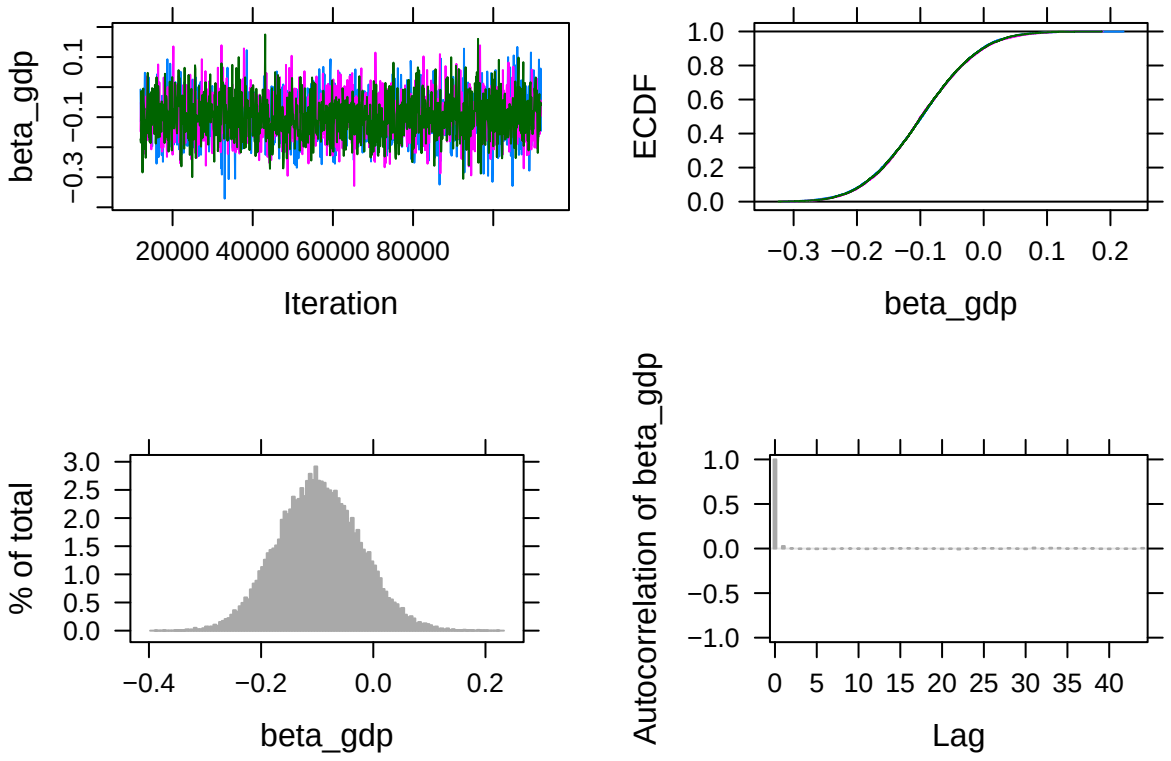
```
summary(posterior_t)
```

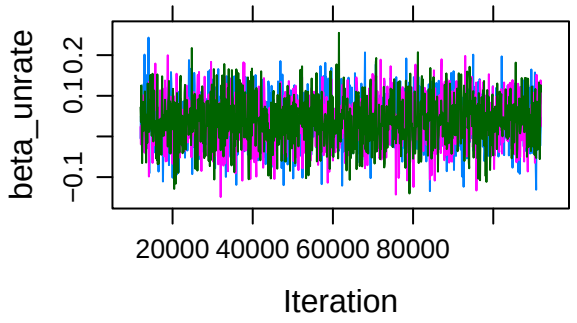
```
##                    Lower95       Median     Upper95         Mean         SD
## beta_0         -0.07085809  0.005176421  0.07803302  0.005194925 0.03773514
## beta_gdp       -0.24098505 -0.098208483  0.04816551 -0.096898446 0.07410371
## beta_unrate    -0.07426995  0.037280387  0.15358244  0.037192602 0.05826241
## beta_inflation -0.21880233 -0.125474766 -0.03677848 -0.125644936 0.04651114
```

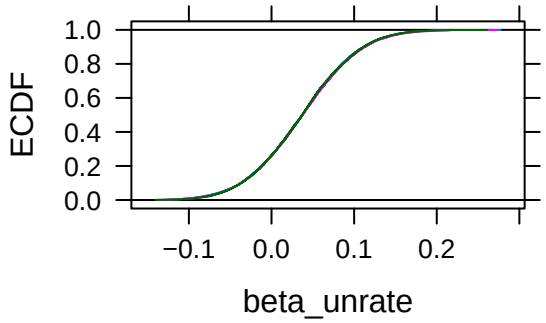
```
## beta_ip          0.26048543  0.406768515  0.55560199  0.406690310 0.07513526
## beta_spread     -0.09221274  0.010194810  0.11460766  0.010176747 0.05255627
## sigma            0.40849741  0.479288798  0.55325512  0.480595114 0.03732101
## nu               2.00149779  2.834115991  3.91638373  2.914141474 0.54547539
##                 Mode         MCerr MC%ofSD SSeff         AC.100      psrf
## beta_0            NA 0.0002178640     0.6 30000  4.293006e-03 1.0000279
## beta_gdp          NA 0.0004400973     0.6 28352 -1.585320e-03 0.9999896
## beta_unrate       NA 0.0003504973     0.6 27632  3.489386e-03 1.0000307
## beta_inflation    NA 0.0002682042     0.6 30073  3.193909e-03 1.0000153
## beta_ip           NA 0.0004452252     0.6 28479  9.216657e-03 0.9999805
## beta_spread       NA 0.0003140931     0.6 27998 -7.218522e-03 1.0000464
## sigma             NA 0.0002205900     0.6 28624 -9.527162e-06 1.0000593
## nu                NA 0.0031440634     0.6 30100  7.375415e-04 0.9999648
```

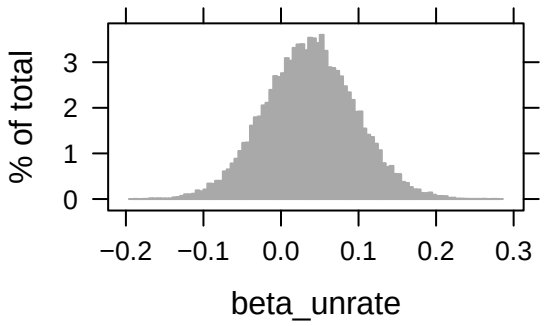
```r
# Plot gianostic plots
plot(posterior_t, vars = c("beta_gdp", "beta_unrate", "beta_inflation",
                           "beta_ip", "beta_spread", "sigma", "nu"))
```

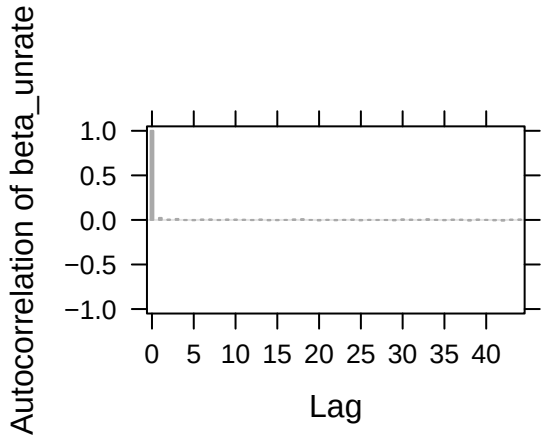
```
## Generating plots...
```

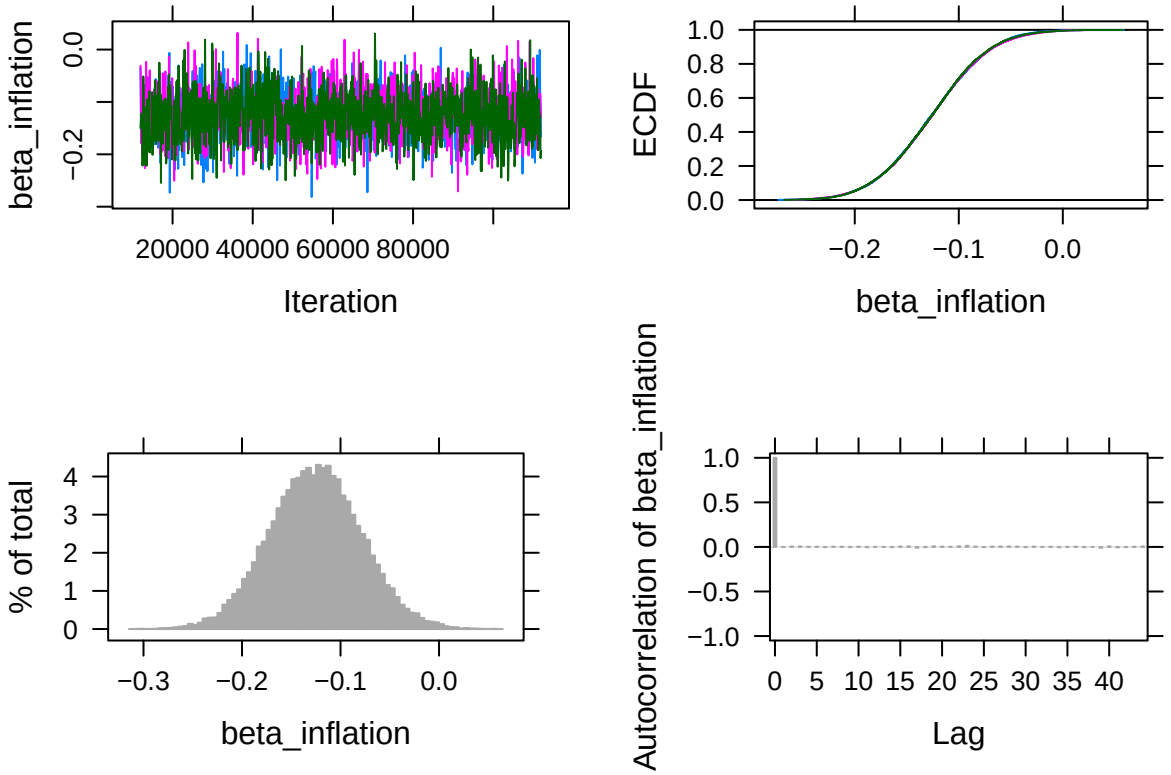


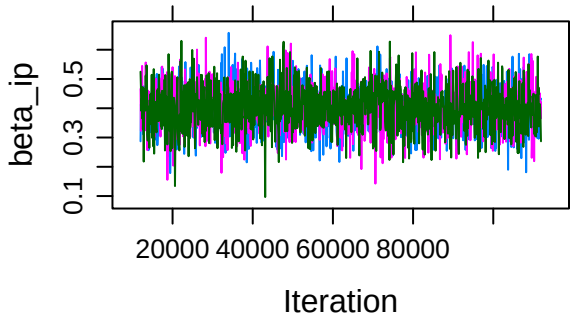

beta_unrate
−0.1
0.1
0.2
20000 40000 60000 80000
Iteration

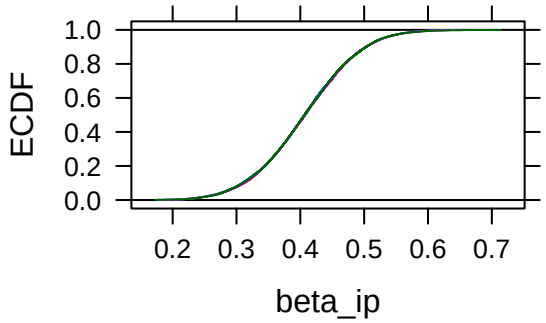

ECDF
0.0
0.2
0.4
0.6
0.8
1.0
−0.1
0.0
0.1
0.2
beta_unrate

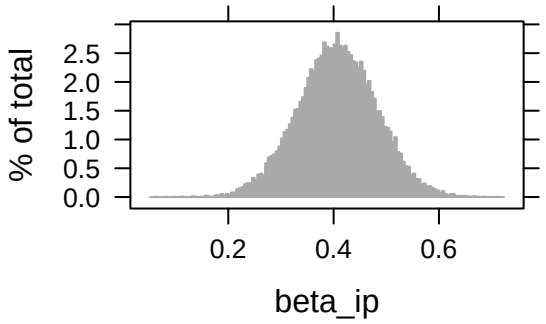

% of total
0
1
2
3
−0.2
−0.1
0.0
0.1
0.2
0.3
beta_unrate

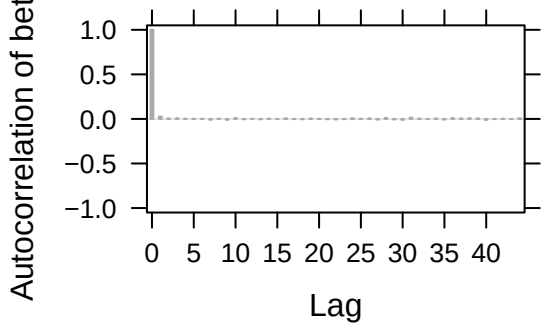

Autocorrelation of beta_unrate
−1.0
−0.5
0.0
0.5
1.0
0
5
10
15
20
25
30
35
40
Lag

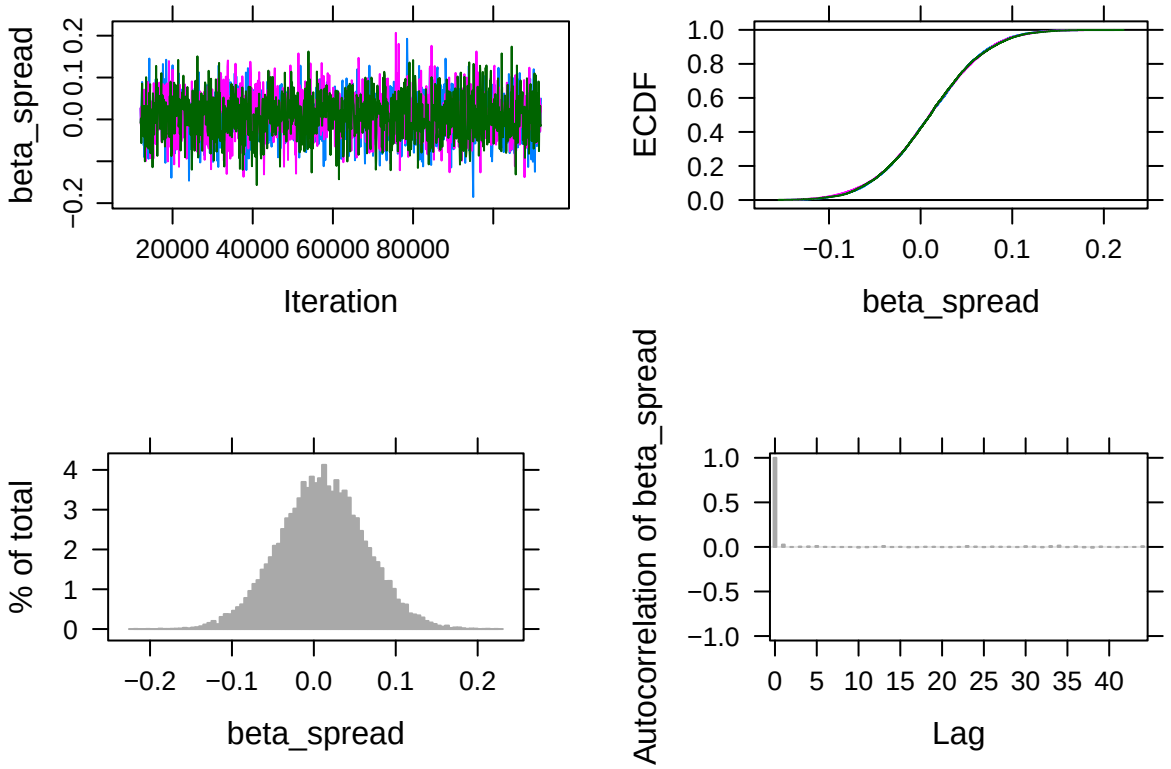

beta_inflation
0.0
−0.2
20000 40000 60000 80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.1
0.0
beta_inflation
% of total
4
3
2
1
0
−0.3
−0.2
−0.1
0.0
beta_inflation
Autocorrelation of beta_inflation
1.0
0.5
0.0
−0.5
−1.0
0
5
10
15
20
25
30
35
40
Lag

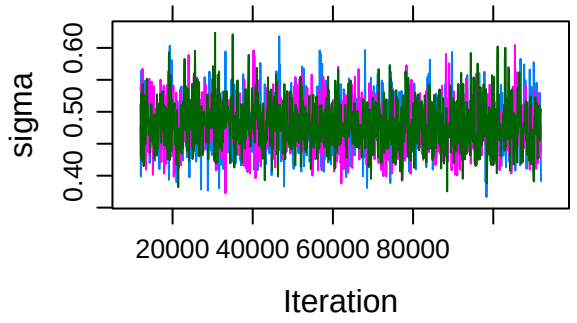

beta_ip
0.1
0.3
0.5
20000 40000 60000 80000
Iteration

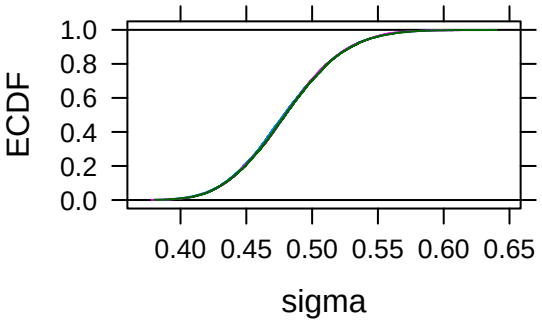

ECDF
0.0
0.2
0.4
0.6
0.8
1.0
0.2 0.3 0.4 0.5 0.6 0.7
beta_ip

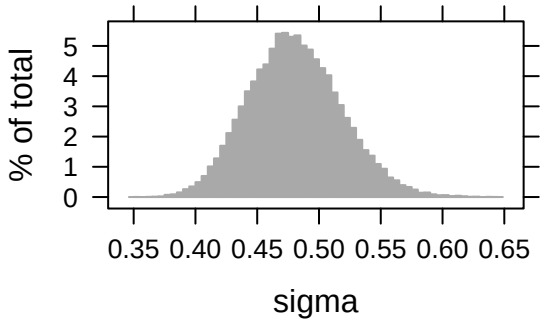

% of total
0.0
0.5
1.0
1.5
2.0
2.5
0.2 0.4 0.6
beta_ip

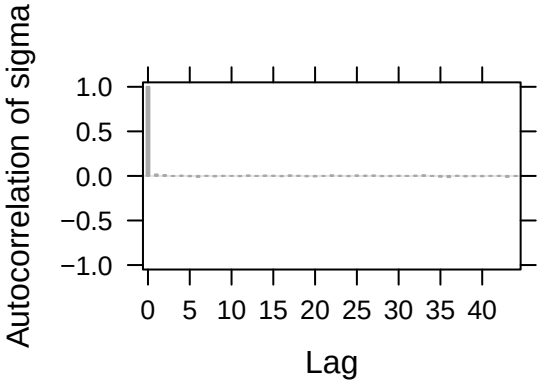

Autocorrelation of beta_ip
−1.0
−0.5
0.0
0.5
1.0
0 5 10 15 20 25 30 35 40
Lag

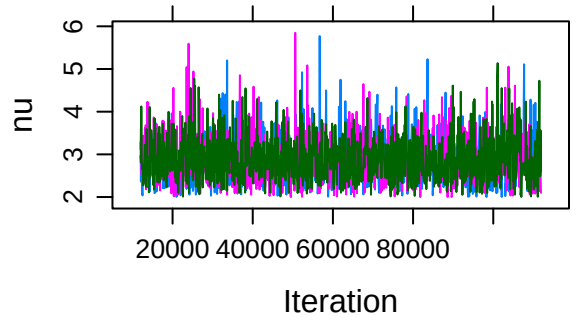

beta_spread
−0.2
0.0
0.1
0.2
20000
40000
60000
80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
−0.1
0.0
0.1
0.2
beta_spread
% of total
4
3
2
1
0
−0.2
−0.1
0.0
0.1
0.2
beta_spread
Autocorrelation of beta_spread
1.0
0.5
0.0
−0.5
−1.0
0
5
10
15
20
25
30
35
40
Lag

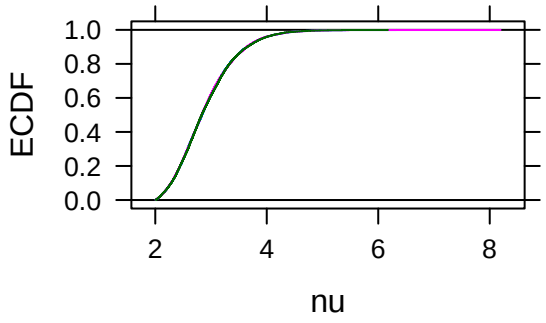

sigma
0.40 0.50 0.60
20000 40000 60000 80000
Iteration

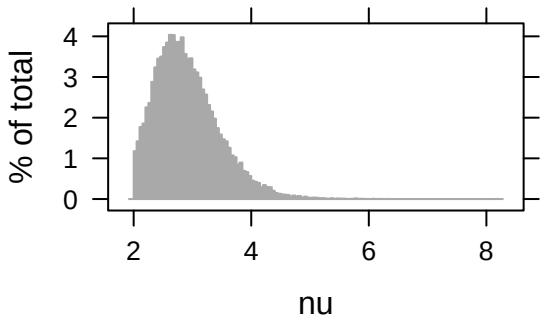

ECDF
1.0
0.8
0.6
0.4
0.2
0.0
0.40 0.45 0.50 0.55 0.60 0.65
sigma

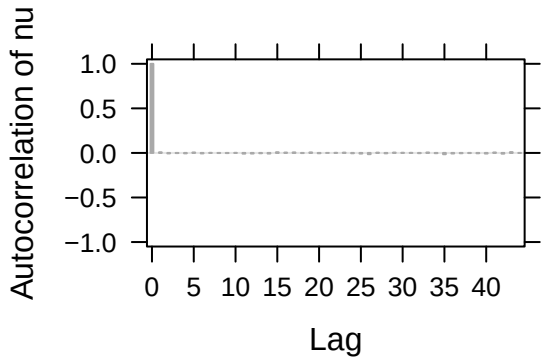

% of total
5
4
3
2
1
0
0.35 0.40 0.45 0.50 0.55 0.60 0.65
sigma

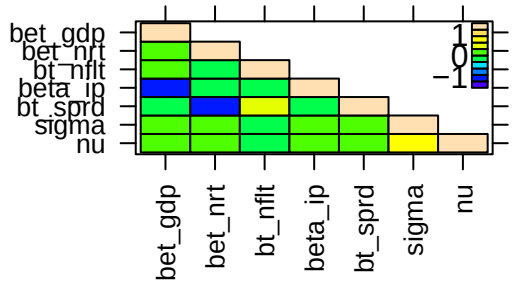

Autocorrelation of sigma
1.0
0.5
0.0
−0.5
−1.0
0 5 10 15 20 25 30 35 40
Lag


nu
2
3
4
5
6
20000 40000 60000 80000
Iteration


ECDF
0.0
0.2
0.4
0.6
0.8
1.0
2
4
6
8
nu


% of total
0
1
2
3
4
2
4
6
8
nu


Autocorrelation of nu
−1.0
−0.5
0.0
0.5
1.0
0 5 10 15 20 25 30 35 40
Lag




```
# Convert posterior draws to matrix
post_t <- as.matrix(as.mcmc(posterior_t))
```

```
## Warning in as.mcmc.runjags(posterior_t): Combining the 3 mcmc chains together
```

```
# Define vector of param names
t_params <- c("beta_0", "beta_gdp", "beta_unrate",
              "beta_inflation", "beta_ip", "beta_spread", "sigma", "nu")

# Compute mean, upper and lower bounds
t_mean  <- apply(post_t[, t_params], 2, mean)
t_lower <- apply(post_t[, t_params], 2, quantile, probs = 0.025)
t_upper <- apply(post_t[, t_params], 2, quantile, probs = 0.975)

# Create summary table
t_summary_table <- data.frame(
  Parameter = t_params,
  Mean = t_mean,
  CrI_2.5 = t_lower,
  CrI_97.5 = t_upper
)

# Display table
knitr::kable(
  t_summary_table,
```

```
  digits = 4,
  caption = "Posterior summary for Bayesian regression with Student-t errors"
)
```

Table 4: Posterior summary for Bayesian regression with Student-t errors

| | Parameter | Mean | CrI_2.5 | CrI_97.5 |
|---|---|---|---|---|
| beta_0 | beta_0 | 0.0052 | -0.0695 | 0.0798 |
| beta_gdp | beta_gdp | -0.0969 | -0.2386 | 0.0507 |
| beta_unrate | beta_unrate | 0.0372 | -0.0766 | 0.1522 |
| beta_inflation | beta_inflation | -0.1256 | -0.2162 | -0.0334 |
| beta_ip | beta_ip | 0.4067 | 0.2575 | 0.5537 |
| beta_spread | beta_spread | 0.0102 | -0.0937 | 0.1133 |
| sigma | sigma | 0.4806 | 0.4118 | 0.5571 |
| nu | nu | 2.9141 | 2.0974 | 4.1946 |

```
# Design matrix for scaled predictors
X_scaled <- model.matrix(
  ~ lag_gdp_growth + lag_unrate + lag_inflation + lag_ip_growth + lag_spread,
  data = df_final_scaled
)

# Combine coeffecient draws
beta_draws_t <- cbind(
  post_t[, "beta_0"],
  post_t[, "beta_gdp"],
  post_t[, "beta_unrate"],
  post_t[, "beta_inflation"],
  post_t[, "beta_ip"],
  post_t[, "beta_spread"]
)

# Compute mu and draw mean
mu_post_t <- beta_draws_t %*% t(X_scaled)
t_pred_mean <- colMeans(mu_post_t)

# Store results
df_final$t_bayes_pred_scaled <- t_pred_mean
```

```
# Define model string for unscaled t model
modelString_t_unscaled <- "
model {

  for (i in 1:N) {
    mu[i] <- beta_0 +
             beta_gdp * lag_gdp[i] +
             beta_unrate * lag_unrate[i] +
             beta_inflation * lag_inflation[i] +
             beta_ip * lag_ip[i] +
             beta_spread * lag_spread[i]
```

```
    y[i] ~ dt(mu[i], tau, nu)
  }

  beta_0         ~ dnorm(0, 1/25)
  beta_gdp       ~ dnorm(0, 1/25)
  beta_unrate    ~ dnorm(0, 1/25)
  beta_inflation ~ dnorm(0, 1/25)
  beta_ip        ~ dnorm(0, 1/25)
  beta_spread    ~ dnorm(0, 1/25)

  sigma ~ dunif(0, 0.1)
  tau <- pow(sigma, -2)

  nu_minus_two ~ dexp(1/30)
  nu <- nu_minus_two + 2
}
"

# Pass data and hyperparameter values
y <- as.vector(df_final$gdp_growth)
lag_gdp <- as.vector(df_final$lag_gdp_growth)
lag_unrate <- as.vector(df_final$lag_unrate)
lag_inflation <- as.vector(df_final$lag_inflation)
lag_ip <- as.vector(df_final$lag_ip_growth)
lag_spread <- as.vector(df_final$lag_spread)

N <- length(y)

the_data_t_unscaled <- list(
  y = y,
  lag_gdp = lag_gdp,
  lag_unrate = lag_unrate,
  lag_inflation = lag_inflation,
  lag_ip = lag_ip,
  lag_spread = lag_spread,
  N = N
)

initsfunction_t_unscaled <- function(chain) {
  list(
    .RNG.seed = 200 + chain,
    .RNG.name = "base::Mersenne-Twister",
    sigma = 0.01,
    nu_minus_two = 10
  )
}

posterior_t_unscaled <- run.jags(
  model = modelString_t_unscaled,
  n.chains = 3,
  data = the_data_t_unscaled,
  monitor = c("beta_0", "beta_gdp", "beta_unrate",
              "beta_inflation", "beta_ip", "beta_spread",
```

```
              "sigma", "nu"),
  adapt = 2000,
  burnin = 10000,
  sample = 10000,
  thin = 10,
  inits = initsfunction_t_unscaled,
  method = "parallel"
)
```

```
## Calling 3 simulations using the parallel method...
## Following the progress of chain 1 (the program will wait for all chains
## to finish before continuing):
## Welcome to JAGS 4.3.2 on Mon Apr 20 21:10:08 2026
## JAGS is free software and comes with ABSOLUTELY NO WARRANTY
## Loading module: basemod: ok
## Loading module: bugs: ok
## . . Reading data file data.txt
## . Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 263
##    Unobserved stochastic nodes: 8
##    Total graph size: 3037
## . Reading parameter file inits1.txt
## . Initializing model
## . Adapting 2000
## -------------------------------------------------| 2000
## ++++++++++++++++++++++++++++++++++++++++++++++++++ 100%
## Adaptation successful
## . Updating 10000
## -------------------------------------------------| 10000
## ************************************************** 100%
## . . . . . . . . . Updating 100000
## -------------------------------------------------| 100000
## ************************************************** 100%
## . . . . Updating 0
## . Deleting model
## .
## All chains have finished
## Simulation complete.  Reading coda files...
## Coda files loaded successfully
## Calculating summary statistics...
## Calculating the Gelman-Rubin statistic for 8 variables....
## Finished running the simulation
```

```
# Give summary of draws
summary(posterior_t_unscaled)
```

```
##                    Lower95        Median      Upper95          Mean
## beta_0          0.003257300  6.923615e-03  0.010698900  6.929179e-03
## beta_gdp       -0.249846000 -1.001550e-01  0.041368700 -9.971509e-02
## beta_unrate    -0.000497228  2.274815e-04  0.000941601  2.269404e-04
```

```
## beta_inflation -0.306329000 -1.771735e-01 -0.052666000 -1.779247e-01
## beta_ip         0.151559000  2.358375e-01  0.320864000  2.359489e-01
## beta_spread    -0.000774903  8.747695e-05  0.001005810  8.793501e-05
## sigma           0.004317050  5.049240e-03  0.005868450  5.066691e-03
## nu              2.000120000  2.840585e+00  3.914510000  2.918684e+00
##                          SD Mode        MCerr MC%ofSD SSeff        AC.100
## beta_0         0.0018958623   NA 2.996468e-05     1.6  4003  0.0752018240
## beta_gdp       0.0745197552   NA 5.796493e-04     0.8 16528  0.0040817120
## beta_unrate    0.0003663691   NA 6.017781e-06     1.6  3707  0.0717904593
## beta_inflation 0.0653055926   NA 4.389914e-04     0.7 22130 -0.0129699458
## beta_ip        0.0434914735   NA 2.840110e-04     0.7 23450 -0.0048007115
## beta_spread    0.0004533109   NA 5.052671e-06     1.1  8049  0.0250894878
## sigma          0.0003992869   NA 2.361433e-06     0.6 28590  0.0002906496
## nu             0.5458374176   NA 3.189611e-03     0.6 29285  0.0002535235
##                    psrf
## beta_0         1.000484
## beta_gdp       1.000180
## beta_unrate    1.000376
## beta_inflation 1.000003
## beta_ip        1.000092
## beta_spread    1.000255
## sigma          1.000024
## nu             1.000143
```

```
# Convert results to matrix form
post_t_unscaled <- as.matrix(as.mcmc(posterior_t_unscaled))
```

```
## Warning in as.mcmc.runjags(posterior_t_unscaled): Combining the 3 mcmc chains
## together
```

```
# Create design matrix for unscaled data
X_unscaled <- model.matrix(
  ~ lag_gdp_growth + lag_unrate + lag_inflation + lag_ip_growth + lag_spread,
  data = df_final
)

# Combine coefficient draws
beta_draws_t_unscaled <- cbind(
  post_t_unscaled[, "beta_0"],
  post_t_unscaled[, "beta_gdp"],
  post_t_unscaled[, "beta_unrate"],
  post_t_unscaled[, "beta_inflation"],
  post_t_unscaled[, "beta_ip"],
  post_t_unscaled[, "beta_spread"]
)

# Compute and extract desired params
mu_draws_t <- beta_draws_t_unscaled %*% t(X_unscaled)
sigma_draws_t <- post_t_unscaled[, "sigma"]
nu_draws_t <- post_t_unscaled[, "nu"]

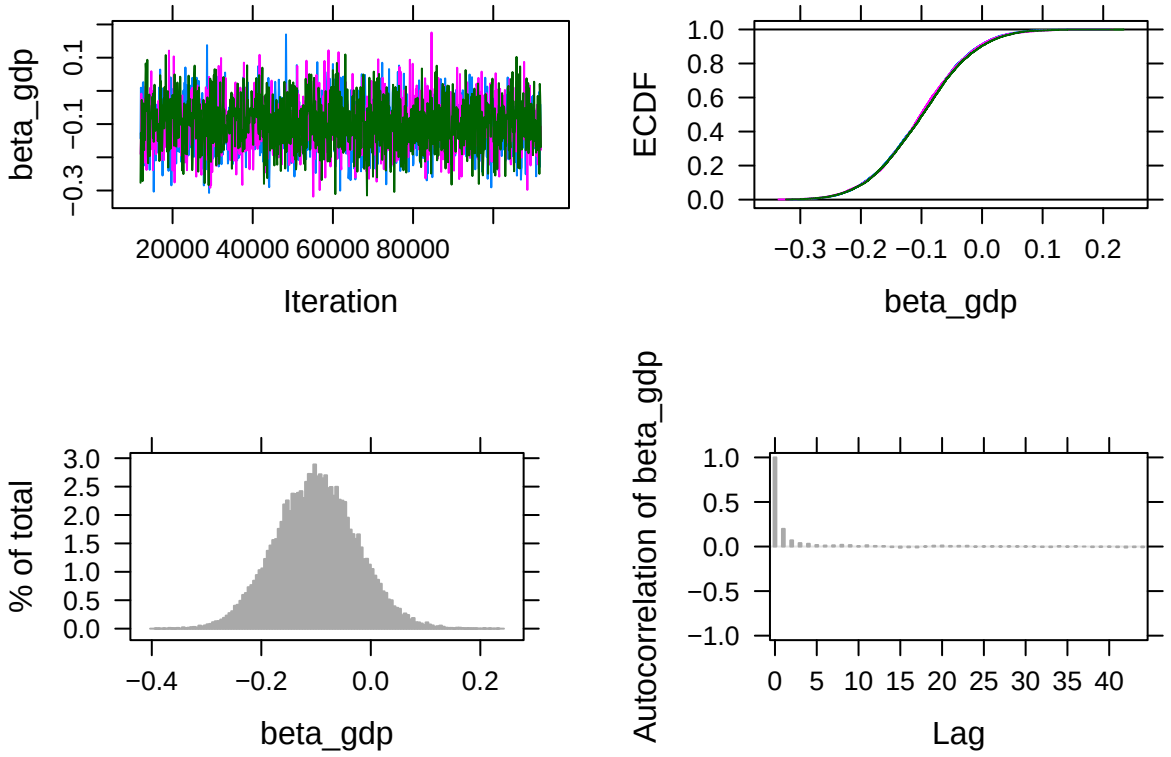
# PCompute posterior probability of negative GDP growth for each quarter
bayes_t_prob_neg <- sapply(1:ncol(mu_draws_t), function(j) {
```

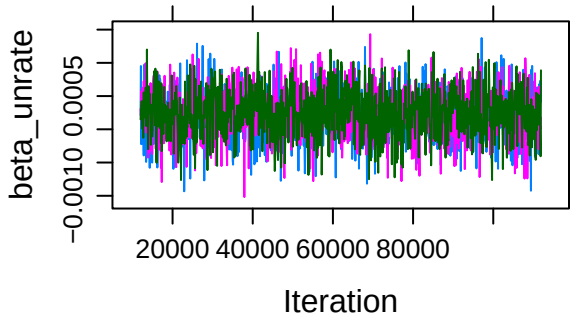
```
  mean(pt((0 - mu_draws_t[, j]) / sigma_draws_t, df = nu_draws_t))
})

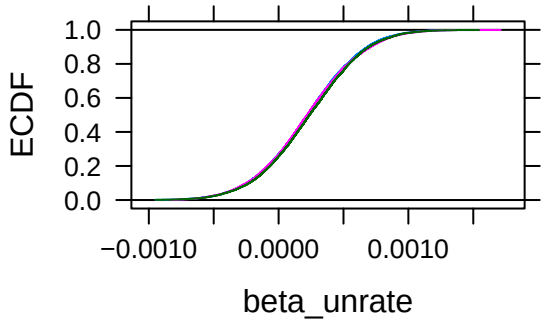
# Store results
df_final$bayes_t_prob_neg <- bayes_t_prob_neg
```

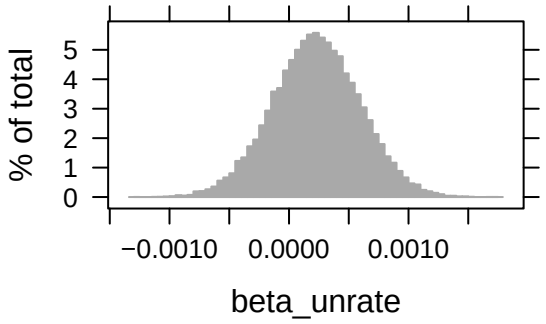
```
# Plot diagnostic plots of simulation
plot(posterior_t_unscaled, vars = c("beta_gdp", "beta_unrate", "beta_inflation",
                            "beta_ip", "beta_spread", "sigma", "nu"))
```

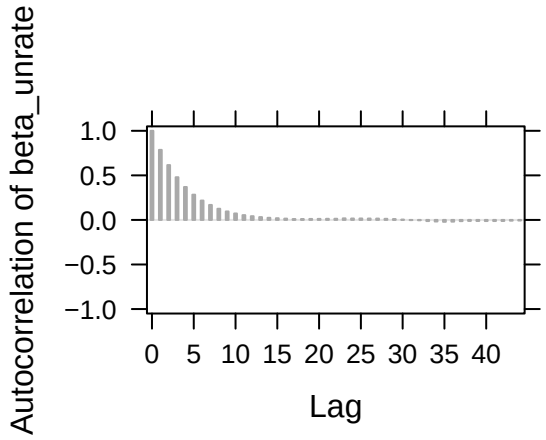
```
## Generating plots...
```

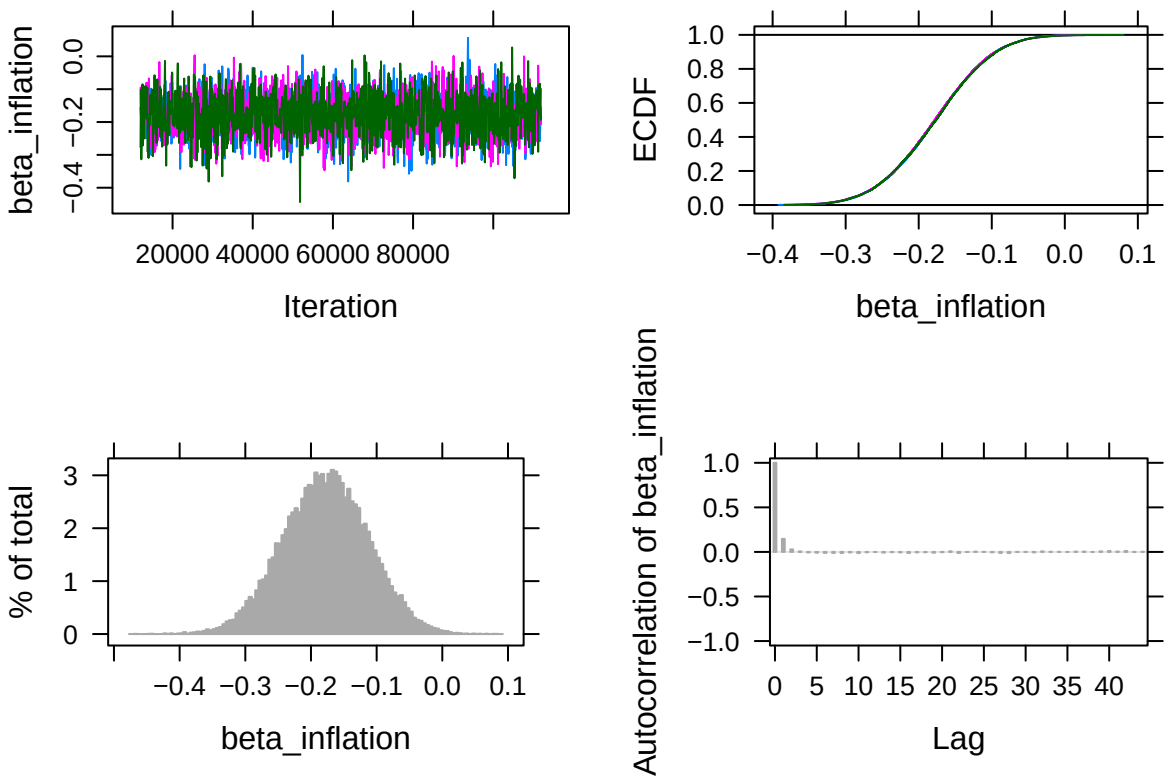


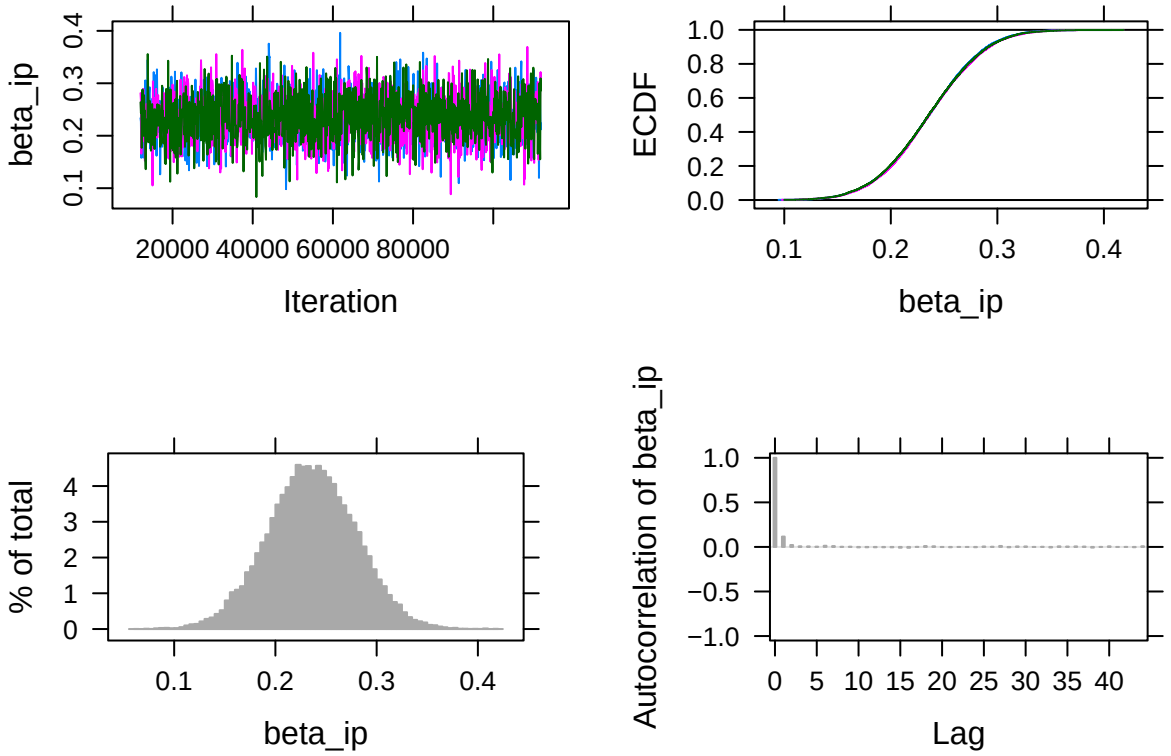

beta_unrate
−0.0010 0.0005
20000 40000 60000 80000
Iteration

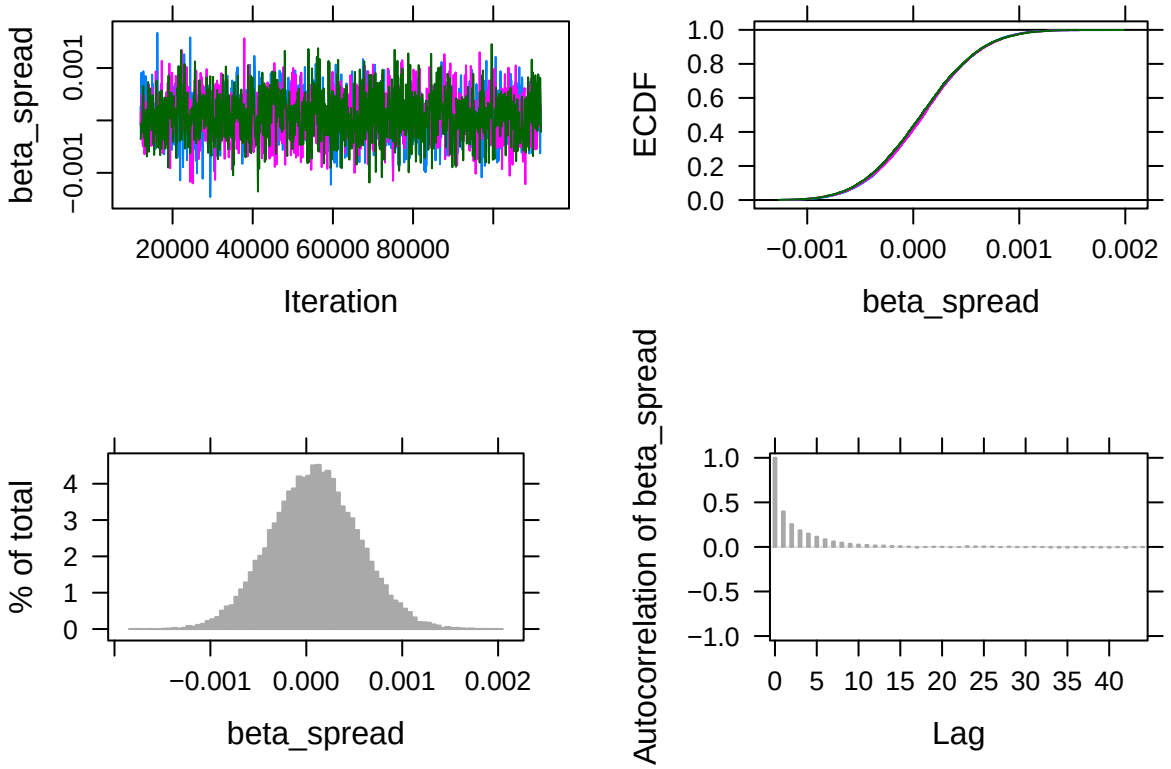

ECDF
1.0
0.8
0.6
0.4
0.2
0.0
−0.0010
0.0000
0.0010
beta_unrate

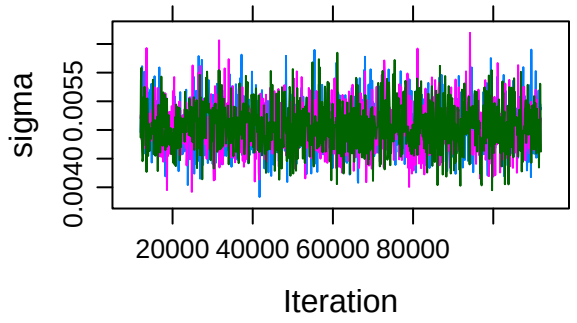

% of total
5
4
3
2
1
0
−0.0010
0.0000
0.0010
beta_unrate

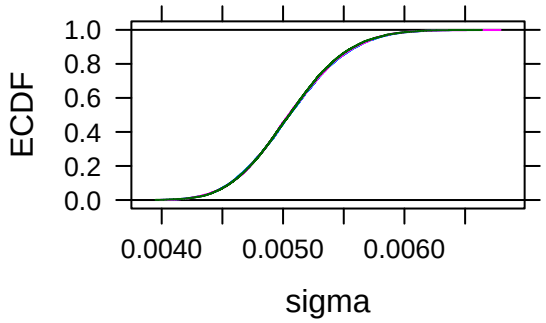

Autocorrelation of beta_unrate
1.0
0.5
0.0
−0.5
−1.0
0
5
10
15
20
25
30
35
40
Lag

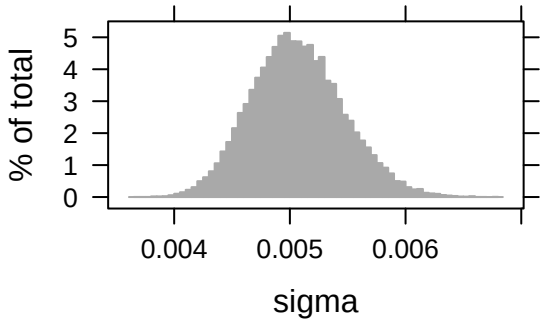

beta_inflation
−0.4 −0.2 0.0
20000 40000 60000 80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
−0.4 −0.3 −0.2 −0.1 0.0 0.1
beta_inflation
% of total
3
2
1
0
−0.4 −0.3 −0.2 −0.1 0.0 0.1
beta_inflation
Autocorrelation of beta_inflation
1.0
0.5
0.0
−0.5
−1.0
0 5 10 15 20 25 30 35 40
Lag

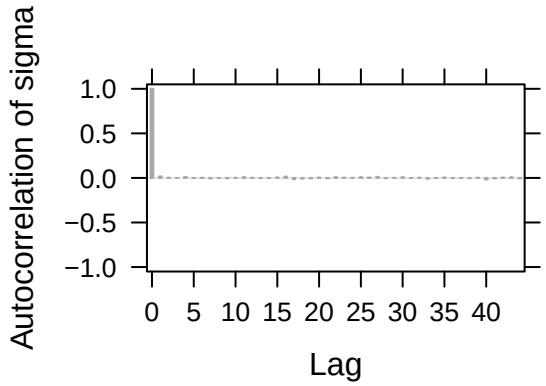

beta_ip
0.1 0.2 0.3 0.4
20000 40000 60000 80000
Iteration
ECDF
1.0 0.8 0.6 0.4 0.2 0.0
0.1 0.2 0.3 0.4
beta_ip
% of total
4 3 2 1 0
0.1 0.2 0.3 0.4
beta_ip
Autocorrelation of beta_ip
1.0 0.5 0.0 −0.5 −1.0
0 5 10 15 20 25 30 35 40
Lag

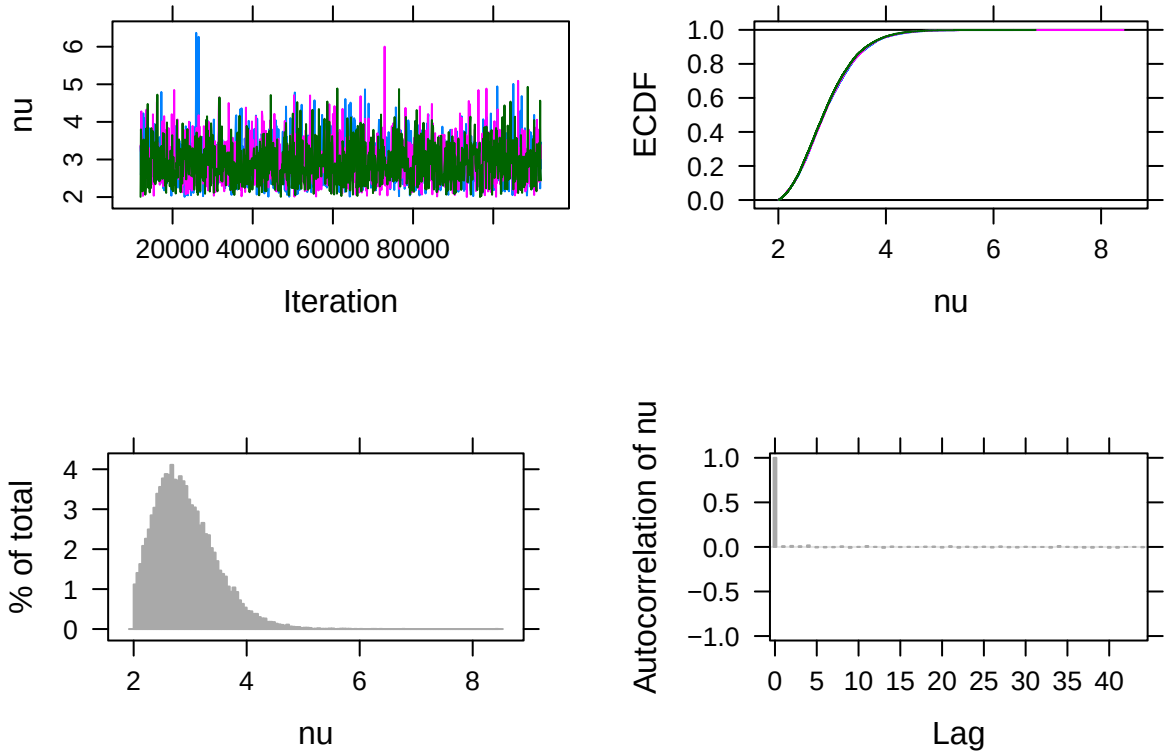

beta_spread
0.001
−0.001
20000 40000 60000 80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
−0.001
0.000
0.001
0.002
beta_spread
% of total
4
3
2
1
0
−0.001
0.000
0.001
0.002
beta_spread
Autocorrelation of beta_spread
1.0
0.5
0.0
−0.5
−1.0
0
5
10
15
20
25
30
35
40
Lag

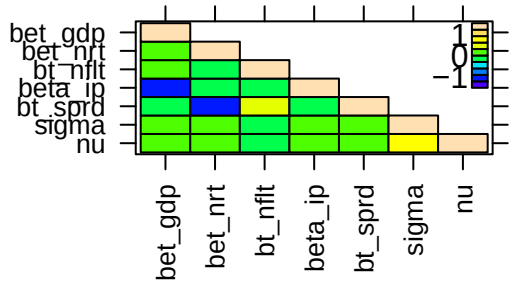

sigma
0.0040 0.0055
20000 40000 60000 80000
Iteration


ECDF
1.0
0.8
0.6
0.4
0.2
0.0
0.0040
0.0050
0.0060
sigma


% of total
5
4
3
2
1
0
0.004
0.005
0.006
sigma


Autocorrelation of sigma
1.0
0.5
0.0
−0.5
−1.0
0 5 10 15 20 25 30 35 40
Lag


nu
2 3 4 5 6
20000 40000 60000 80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
2 4 6 8
nu
% of total
4 3 2 1 0
2 4 6 8
nu
Autocorrelation of nu
1.0 0.5 0.0 −0.5 −1.0
0 5 10 15 20 25 30 35 40
Lag




```
# Convert results to matrix form
post_t_scaled <- as.matrix(as.mcmc(posterior_t))
```

```
## Warning in as.mcmc.runjags(posterior_t): Combining the 3 mcmc chains together
```

```
# Define vector of params
t_params <- c("beta_0", "beta_gdp", "beta_unrate",
              "beta_inflation", "beta_ip", "beta_spread")

# Compute mean and upper and lower bounds
t_mean  <- apply(post_t_scaled[, t_params], 2, mean)
t_lower <- apply(post_t_scaled[, t_params], 2, quantile, 0.025)
t_upper <- apply(post_t_scaled[, t_params], 2, quantile, 0.975)

# Create table
t_table <- data.frame(
  Parameter = t_params,
  Mean = t_mean,
  CrI_2.5 = t_lower,
  CrI_97.5 = t_upper
)

# Display table
knitr::kable(t_table,
             digits = 4,
             caption = "Posterior summary for t-error Bayesian model (scaled data)")
```

Table 5: Posterior summary for t-error Bayesian model (scaled data)

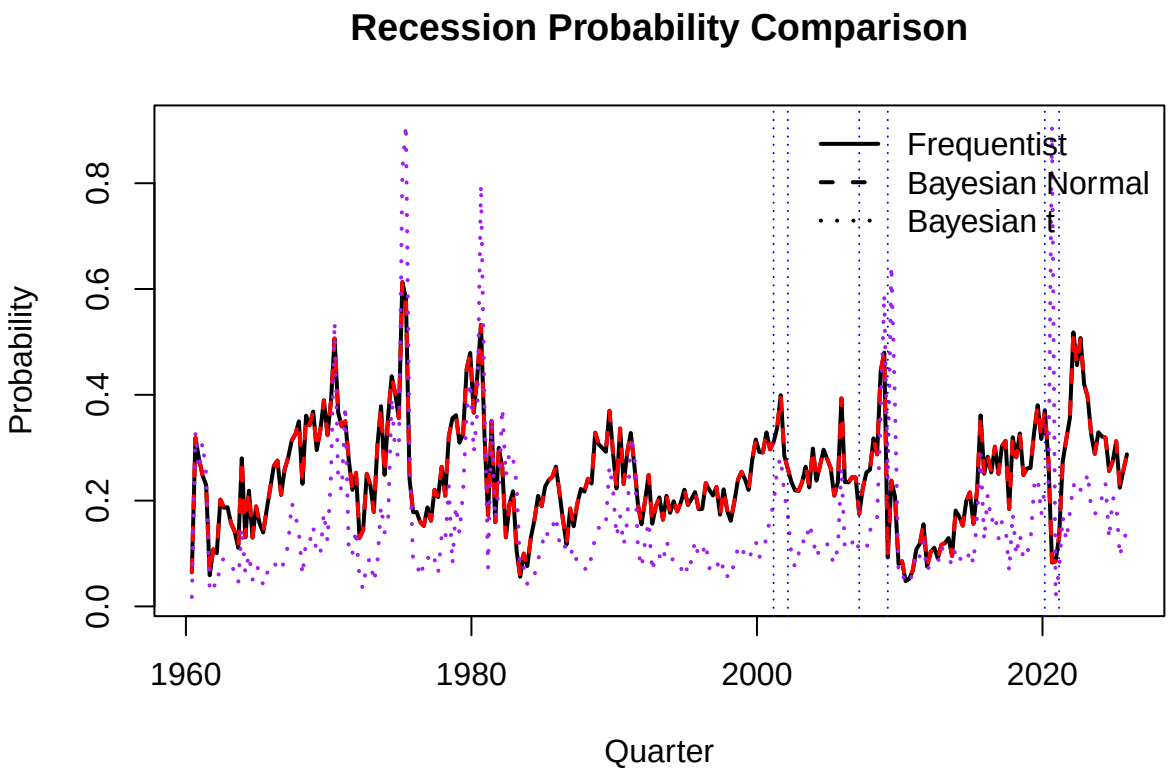
| | Parameter | Mean | CrI_2.5 | CrI_97.5 |
|---|---|---|---|---|
| beta_0 | beta_0 | 0.0052 | -0.0695 | 0.0798 |
| beta_gdp | beta_gdp | -0.0969 | -0.2386 | 0.0507 |
| beta_unrate | beta_unrate | 0.0372 | -0.0766 | 0.1522 |
| beta_inflation | beta_inflation | -0.1256 | -0.2162 | -0.0334 |
| beta_ip | beta_ip | 0.4067 | 0.2575 | 0.5537 |
| beta_spread | beta_spread | 0.0102 | -0.0937 | 0.1133 |

```r
# Plot recession probabilities via frequentist, normal bayesian, t bayesian
plot(df_final$date, df_final$recession_prob,
     type = "l", lwd = 2,
     ylim = range(c(df_final$recession_prob,
                    df_final$bayes_prob_neg,
                    df_final$bayes_t_prob_neg), na.rm = TRUE),
     main = "Recession Probability Comparison",
     xlab = "Quarter",
     ylab = "Probability")

lines(df_final$date, df_final$bayes_prob_neg, lwd = 2, lty = 2, col = "red")
lines(df_final$date, df_final$bayes_t_prob_neg, lwd = 2, lty = 3, col = "purple")

# Insert vertical lines corresponding to recession periods
abline(v = as.Date(c("2001-03-01", "2002-03-01",
                     "2007-03-01", "2009-03-01",
                     "2020-03-01", "2021-03-01")),
       col = "blue", lty = 3)

legend("topright",
       legend = c("Frequentist", "Bayesian Normal", "Bayesian t"),
       lwd = 2,
       lty = c(1,2,3),
       bty = "n")
```




```
# t-model predictions and accuracy
t_rmse <- sqrt(mean((df_final$gdp_growth - df_final$t_bayes_pred)^2, na.rm = TRUE))
t_mae  <- mean(abs(df_final$gdp_growth - df_final$t_bayes_pred), na.rm = TRUE)

# Create table
fit_table <- data.frame(
  Model = c("Frequentist", "Bayesian Normal", "Bayesian t"),
  RMSE = c(freq_rmse, bayes_rmse, t_rmse),
  MAE  = c(freq_mae, bayes_mae, t_mae)
)

# Display table
knitr::kable(fit_table,
             digits = 5,
             caption = "Predictive fit comparison across models")
```

Table 6: Predictive fit comparison across models

| Model | RMSE | MAE |
|---|---|---|
| Frequentist | 0.00994 | 0.00604 |
| Bayesian Normal | 0.00994 | 0.00603 |
| Bayesian t | 0.34565 | 0.24248 |

### Reduced model

```
# Define reduced model predictors via robust regression
reduced_vars <- c("lag_inflation", "lag_ip_growth")
```

```
# Reduced formula on scaled data
reduced_formula <- as.formula(
  paste("gdp_growth ~", paste(reduced_vars, collapse = " + "))
)

# Fit reduced frequentist model on scaled data
freq_mod_reduced <- lm(reduced_formula, data = df_final_scaled)

# Display summary
summary(freq_mod_reduced)
```

```
##
## Call:
## lm(formula = reduced_formula, data = df_final_scaled)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -8.4702 -0.3540  0.0051  0.3492  6.8451
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -1.860e-17  6.041e-02   0.000  1.00000
## lag_inflation -1.877e-01  6.060e-02  -3.097  0.00217 **
## lag_ip_growth  1.199e-01  6.060e-02   1.979  0.04886 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9798 on 260 degrees of freedom
## Multiple R-squared:  0.04741,    Adjusted R-squared:  0.04008
## F-statistic:  6.47 on 2 and 260 DF,  p-value: 0.00181
```

```
# Robust SE results
robust_vcov_reduced <- vcovHC(freq_mod_reduced, type = "HC1")
robust_test_reduced <- coeftest(freq_mod_reduced, vcov = robust_vcov_reduced)

# Display results
robust_test_reduced
```

```
##
## t test of coefficients:
##
##                  Estimate  Std. Error t value Pr(>|t|)
## (Intercept)   -1.8597e-17  6.0414e-02  0.0000  1.00000
## lag_inflation -1.8770e-01  8.9034e-02 -2.1082  0.03597 *
## lag_ip_growth  1.1994e-01  2.0950e-01  0.5725  0.56749
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# Store reduced regression metrics
freq_red_est <- robust_test_reduced[, "Estimate"]
freq_red_se  <- robust_test_reduced[, "Std. Error"]
freq_red_p   <- robust_test_reduced[, "Pr(>|t|)"]
freq_red_lo  <- freq_red_est - 1.96 * freq_red_se
freq_red_hi  <- freq_red_est + 1.96 * freq_red_se

# Create metrics table
freq_reduced_table <- data.frame(
  Variable = rownames(robust_test_reduced),
  Estimate = as.numeric(freq_red_est),
  Robust_SE = as.numeric(freq_red_se),
  CI_Lower = as.numeric(freq_red_lo),
  CI_Upper = as.numeric(freq_red_hi),
  P_Value = as.numeric(freq_red_p)
)

# Display table
knitr::kable(
  freq_reduced_table,
  digits = 4,
  caption = "Reduced frequentist model with robust standard errors"
)
```

Table 7: Reduced frequentist model with robust standard errors

| Variable | Estimate | Robust_SE | CI_Lower | CI_Upper | P_Value |
|---|---|---|---|---|---|
| (Intercept) | 0.0000 | 0.0604 | -0.1184 | 0.1184 | 1.0000 |
| lag_inflation | -0.1877 | 0.0890 | -0.3622 | -0.0132 | 0.0360 |
| lag_ip_growth | 0.1199 | 0.2095 | -0.2907 | 0.5306 | 0.5675 |

```r
# Fit reduced frequentist model on unscaled data
freq_mod_reduced_unscaled <- lm(reduced_formula, data = df_final)

# Display summary
summary(freq_mod_reduced_unscaled)
```

```
##
## Call:
## lm(formula = reduced_formula, data = df_final)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.089228 -0.003729  0.000054  0.003679  0.072109
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)    0.009335   0.001017   9.176  < 2e-16 ***
## lag_inflation -0.264237   0.085314  -3.097  0.00217 **
## lag_ip_growth  0.069041   0.034885   1.979  0.04886 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01032 on 260 degrees of freedom
## Multiple R-squared:  0.04741,    Adjusted R-squared:  0.04008
## F-statistic:  6.47 on 2 and 260 DF,  p-value: 0.00181
```

```
# Predict with model
pred_red <- predict(freq_mod_reduced_unscaled, newdata = df_final, se.fit = TRUE)

# Calculate desired values from predictions
mu_hat_red <- pred_red$fit
se_mu_red  <- pred_red$se.fit
sigma_hat_red <- sigma(freq_mod_reduced_unscaled)
pred_sd_red <- sqrt(se_mu_red^2 + sigma_hat_red^2)

# Probability of negative GDP growth
recession_prob_red <- pnorm(0, mean = mu_hat_red, sd = pred_sd_red)

# Store in dataframe
df_final$pred_gdp_growth_red <- mu_hat_red
df_final$pred_sd_red <- pred_sd_red
df_final$recession_prob_red <- recession_prob_red
```

```
# Calculate error metrics
freq_red_rmse <- sqrt(mean((df_final$gdp_growth - df_final$pred_gdp_growth_red)^2, na.rm = TRUE))
freq_red_mae  <- mean(abs(df_final$gdp_growth - df_final$pred_gdp_growth_red), na.rm = TRUE)

# Store in table
freq_red_fit <- data.frame(
  Model = "Frequentist Reduced",
  RMSE = freq_red_rmse,
  MAE = freq_red_mae
)

# Display table
knitr::kable(
  freq_red_fit,
  digits = 5,
  caption = "Predictive fit for reduced frequentist model"
)
```

Table 8: Predictive fit for reduced frequentist model

| Model | RMSE | MAE |
|---|---|---|
| Frequentist Reduced | 0.01026 | 0.00596 |

```
# Define model string for reduced version of Bayesian normal
modelString_reduced <- "
model {

  for (i in 1:N) {
    mu[i] <- beta_0 +
```

```
            beta_inflation * lag_inflation[i] +
            beta_ip * lag_ip[i]

    y[i] ~ dnorm(mu[i], tau)
  }

  beta_0         ~ dnorm(0, 1/25)
  beta_inflation ~ dnorm(0, 1/25)
  beta_ip        ~ dnorm(0, 1/25)

  sigma ~ dt(0, 1/25, 1) T(0, 1.0E6)
  tau <- pow(sigma, -2)
}
"

# Pass data and hyperparameter values
y <- as.vector(df_final$gdp_growth)
lag_inflation <- as.vector(df_final$lag_inflation)
lag_ip <- as.vector(df_final$lag_ip_growth)

N <- length(y)

the_data_reduced <- list(
  y = y,
  lag_inflation = lag_inflation,
  lag_ip = lag_ip,
  N = N
)

initsfunction_reduced <- function(chain) {
  list(
    .RNG.seed = 300 + chain,
    .RNG.name = "base::Mersenne-Twister"
  )
}

posterior_reduced <- run.jags(
  model = modelString_reduced,
  n.chains = 3,
  data = the_data_reduced,
  monitor = c("beta_0", "beta_inflation", "beta_ip", "sigma"),
  adapt = 2000,
  burnin = 10000,
  sample = 10000,
  thin = 10,
  inits = initsfunction_reduced,
  method = "parallel"
)
```

```
## Calling 3 simulations using the parallel method...
## Following the progress of chain 1 (the program will wait for all chains
## to finish before continuing):
## Welcome to JAGS 4.3.2 on Mon Apr 20 21:31:48 2026
```

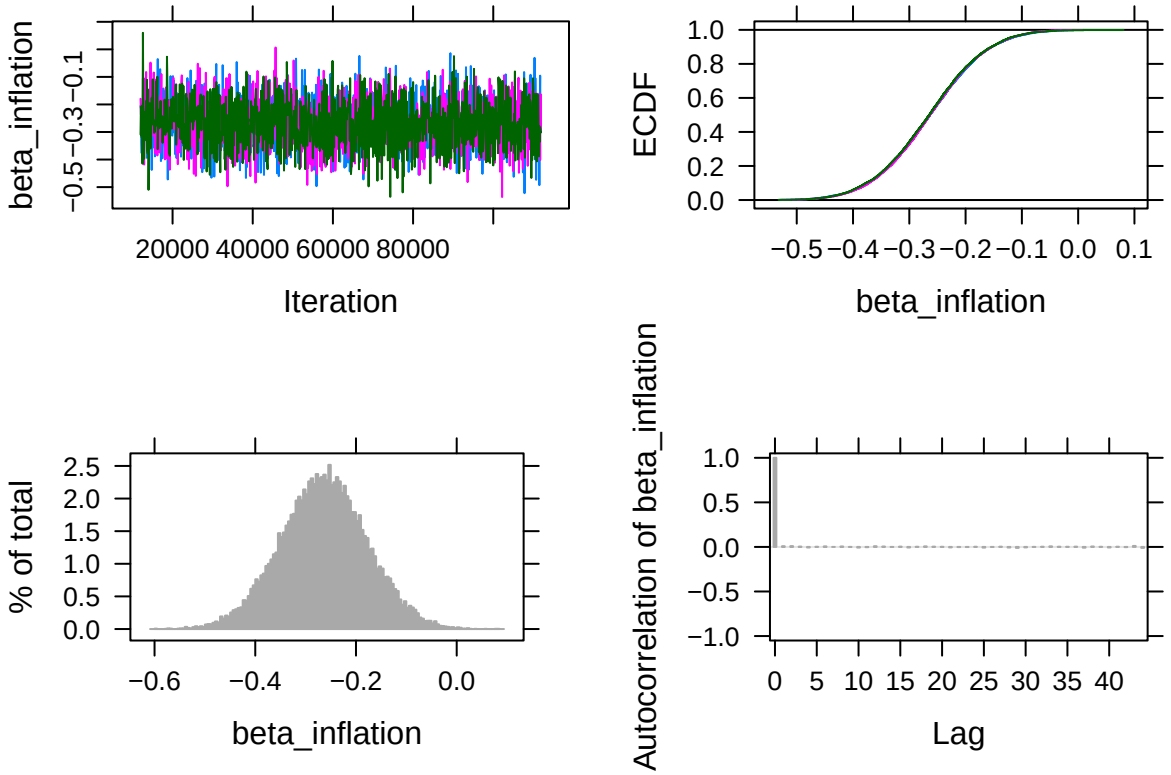
```
## JAGS is free software and comes with ABSOLUTELY NO WARRANTY
## Loading module: basemod: ok
## Loading module: bugs: ok
## . . Reading data file data.txt
## . Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 263
##    Unobserved stochastic nodes: 4
##    Total graph size: 1591
## . Reading parameter file inits1.txt
## . Initializing model
## . Adapting 2000
## -------------------------------------------------| 2000
## ++++++++++++++++++++++++++++++++++++++++++++++++++ 100%
## Adaptation successful
## . Updating 10000
## -------------------------------------------------| 10000
## ************************************************** 100%
## . . . . . Updating 100000
## -------------------------------------------------| 100000
## ************************************************** 100%
## . . . . Updating 0
## . Deleting model
## .
## All chains have finished
## Simulation complete.  Reading coda files...
## Coda files loaded successfully
## Calculating summary statistics...
## Calculating the Gelman-Rubin statistic for 4 variables....
## Finished running the simulation
```

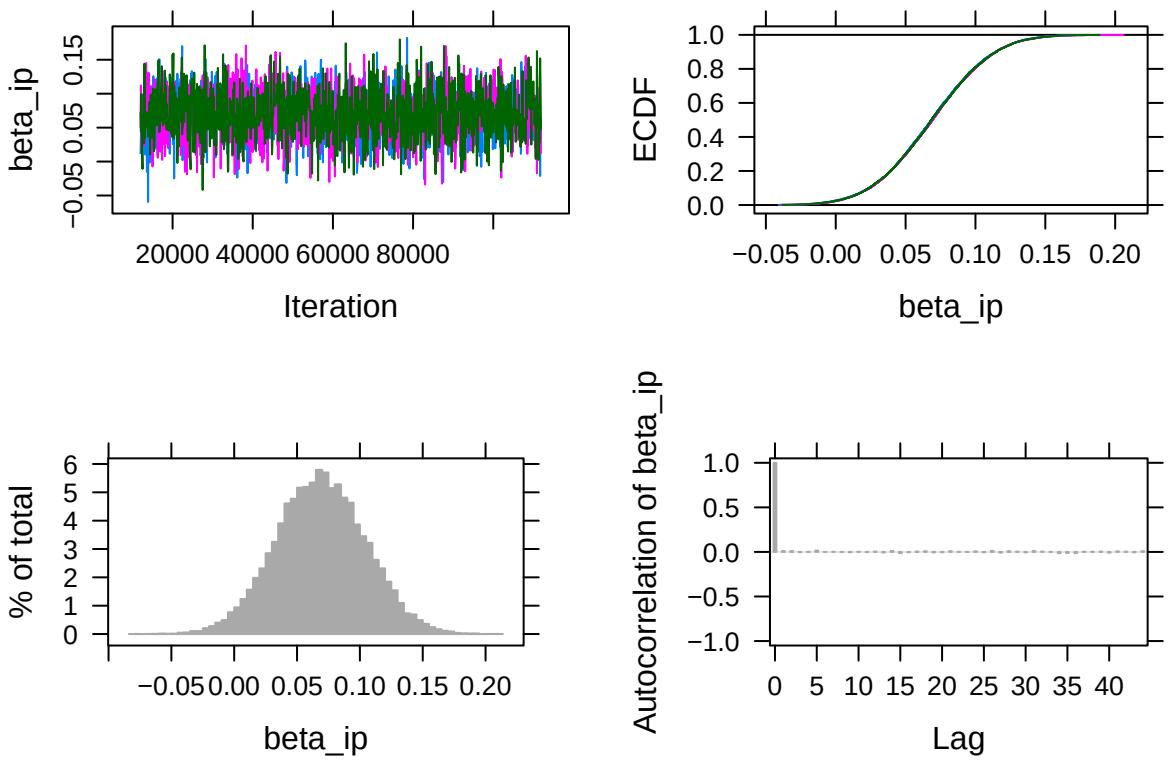
```
summary(posterior_reduced)
```

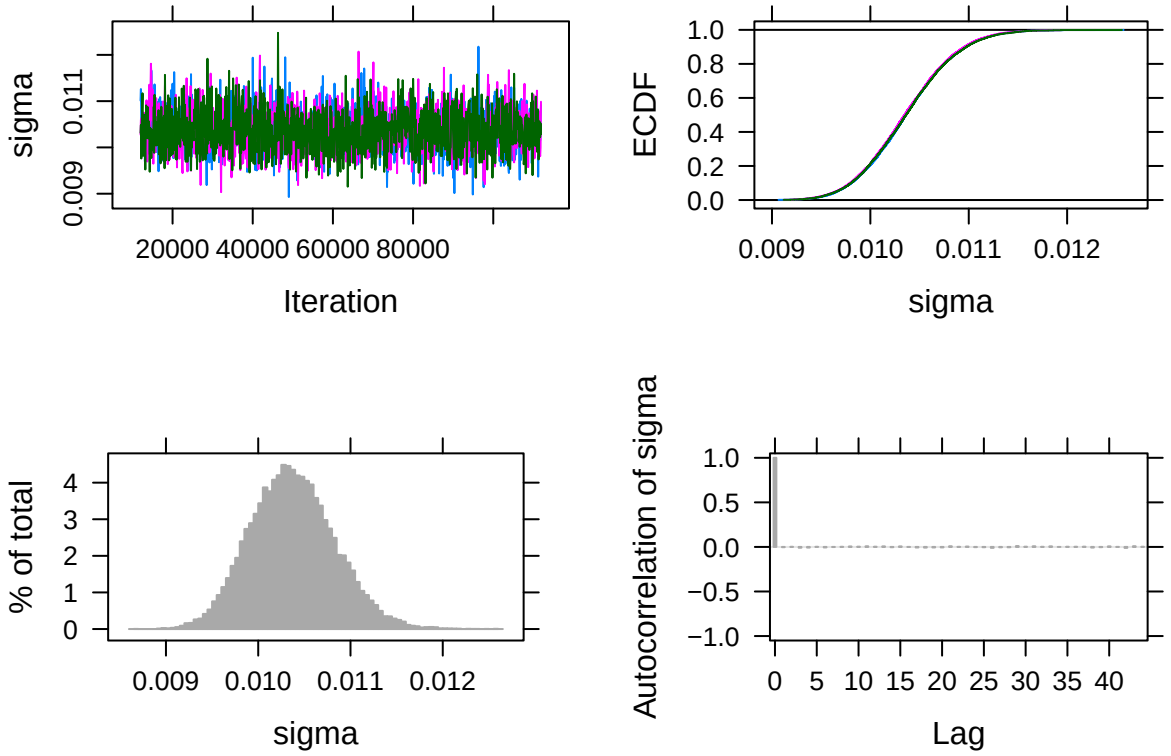
```
##                    Lower95       Median    Upper95        Mean           SD
## beta_0          0.00739572  0.009338035  0.0114003  0.00933300 0.0010214120
## beta_inflation -0.42515200 -0.263833500 -0.0864025 -0.26403138 0.0857418846
## beta_ip        -0.00191563  0.068753950  0.1359870  0.06890622 0.0353152734
## sigma           0.00948886  0.010356950  0.0112556  0.01037225 0.0004557378
##                   Mode        MCerr MC%ofSD SSeff       AC.100     psrf
## beta_0              NA 5.965916e-06     0.6 29312 -0.004771387 1.000063
## beta_inflation      NA 5.008559e-04     0.6 29306 -0.007391498 1.000151
## beta_ip             NA 2.084628e-04     0.6 28699  0.001352247 1.000012
## sigma               NA 2.610034e-06     0.6 30489  0.002140105 1.000213
```

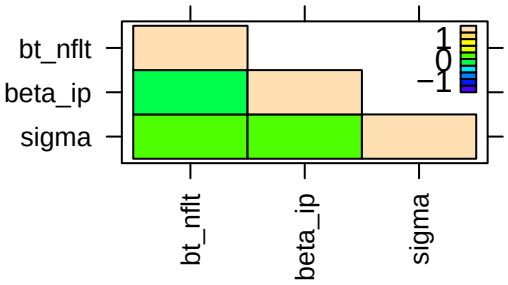
```
# Plot diagnostic plots
plot(posterior_reduced, vars = c("beta_inflation",
                          "beta_ip", "sigma"))
```

```
## Generating plots...
```


beta_inflation
−0.5 −0.3 −0.1
20000 40000 60000 80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
−0.5 −0.4 −0.3 −0.2 −0.1 0.0 0.1
beta_inflation
% of total
2.5
2.0
1.5
1.0
0.5
0.0
−0.6 −0.4 −0.2 0.0
beta_inflation
Autocorrelation of beta_inflation
1.0
0.5
0.0
−0.5
−1.0
0 5 10 15 20 25 30 35 40
Lag


beta_ip
−0.05 0.05 0.15
20000 40000 60000 80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
−0.05 0.00 0.05 0.10 0.15 0.20
beta_ip
% of total
6
5
4
3
2
1
0
−0.05 0.00 0.05 0.10 0.15 0.20
beta_ip
Autocorrelation of beta_ip
1.0
0.5
0.0
−0.5
−1.0
0 5 10 15 20 25 30 35 40
Lag


sigma
0.009
0.011
20000 40000 60000 80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
0.009 0.010 0.011 0.012
sigma
% of total
4
3
2
1
0
0.009 0.010 0.011 0.012
sigma
Autocorrelation of sigma
1.0
0.5
0.0
−0.5
−1.0
0 5 10 15 20 25 30 35 40
Lag




```
# Define model stirng for reduced normal Bayesian model on unscaled data
modelString_reduced_scaled <- "
model {

  for (i in 1:N) {
    mu[i] <- beta_0 +
             beta_inflation * lag_inflation[i] +
             beta_ip * lag_ip[i]

    y[i] ~ dnorm(mu[i], tau)
  }

  beta_0         ~ dnorm(0, 1)
  beta_inflation ~ dnorm(0, 1)
  beta_ip        ~ dnorm(0, 1)

  sigma ~ dt(0, 1, 1) T(0, 1.0E6)
  tau <- pow(sigma, -2)
}
"

# Pass data and hyperparameter values
y <- as.vector(df_final_scaled$gdp_growth)
lag_inflation <- as.vector(df_final_scaled$lag_inflation)
lag_ip <- as.vector(df_final_scaled$lag_ip_growth)
```

```r
N <- length(y)

the_data_reduced_scaled <- list(
  y = y,
  lag_inflation = lag_inflation,
  lag_ip = lag_ip,
  N = N
)

posterior_reduced_scaled <- run.jags(
  model = modelString_reduced_scaled,
  n.chains = 3,
  data = the_data_reduced_scaled,
  monitor = c("beta_0", "beta_inflation", "beta_ip", "sigma"),
  adapt = 2000,
  burnin = 10000,
  sample = 10000,
  thin = 10,
  inits = initsfunction_reduced,
  method = "parallel"
)
```

```
## Calling 3 simulations using the parallel method...
## Following the progress of chain 1 (the program will wait for all chains
## to finish before continuing):
## Welcome to JAGS 4.3.2 on Mon Apr 20 21:32:24 2026
## JAGS is free software and comes with ABSOLUTELY NO WARRANTY
## Loading module: basemod: ok
## Loading module: bugs: ok
## . . Reading data file data.txt
## . Compiling model graph
##    Resolving undeclared variables
##    Allocating nodes
## Graph information:
##    Observed stochastic nodes: 263
##    Unobserved stochastic nodes: 4
##    Total graph size: 1589
## . Reading parameter file inits1.txt
## . Initializing model
## . Adapting 2000
## -------------------------------------------------| 2000
## ++++++++++++++++++++++++++++++++++++++++++++++++++ 100%
## Adaptation successful
## . Updating 10000
## -------------------------------------------------| 10000
## ************************************************** 100%
## . . . . . Updating 100000
## -------------------------------------------------| 100000
## ************************************************** 100%
## . . . . Updating 0
## . Deleting model
## .
## All chains have finished
```

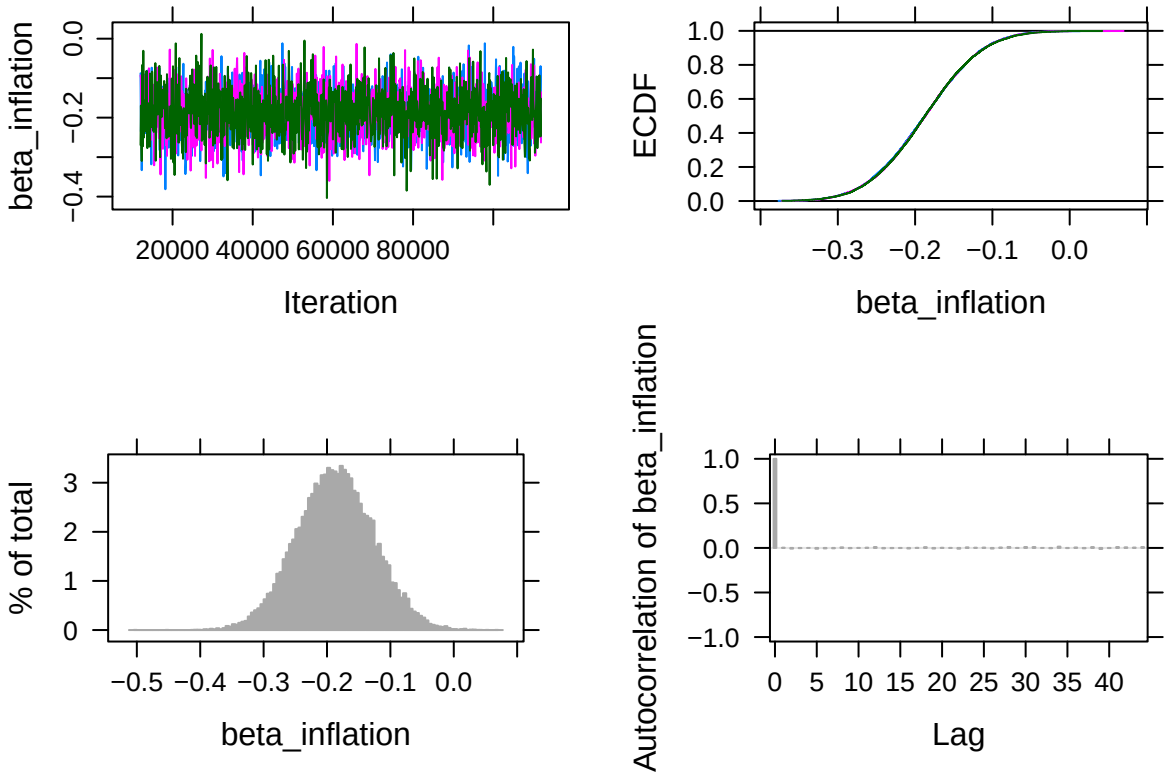
```
## Simulation complete.  Reading coda files...
## Coda files loaded successfully
## Calculating summary statistics...
## Calculating the Gelman-Rubin statistic for 4 variables....
## Finished running the simulation
```

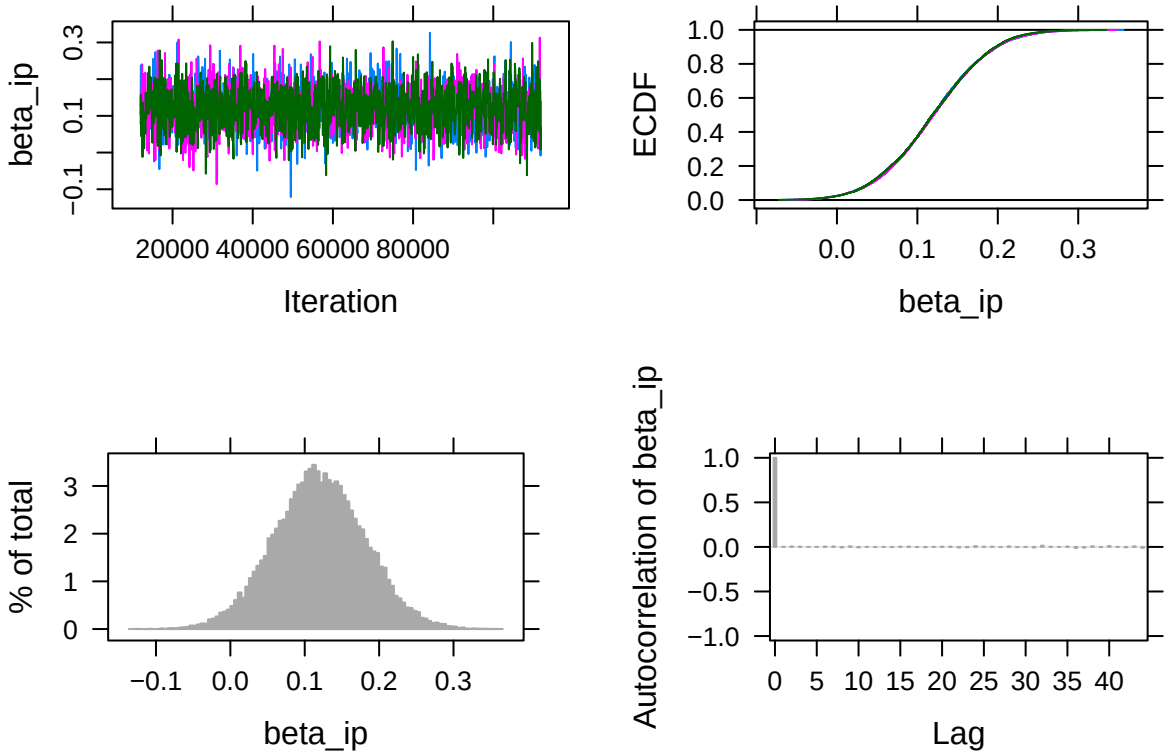
```
summary(posterior_reduced_scaled)
```

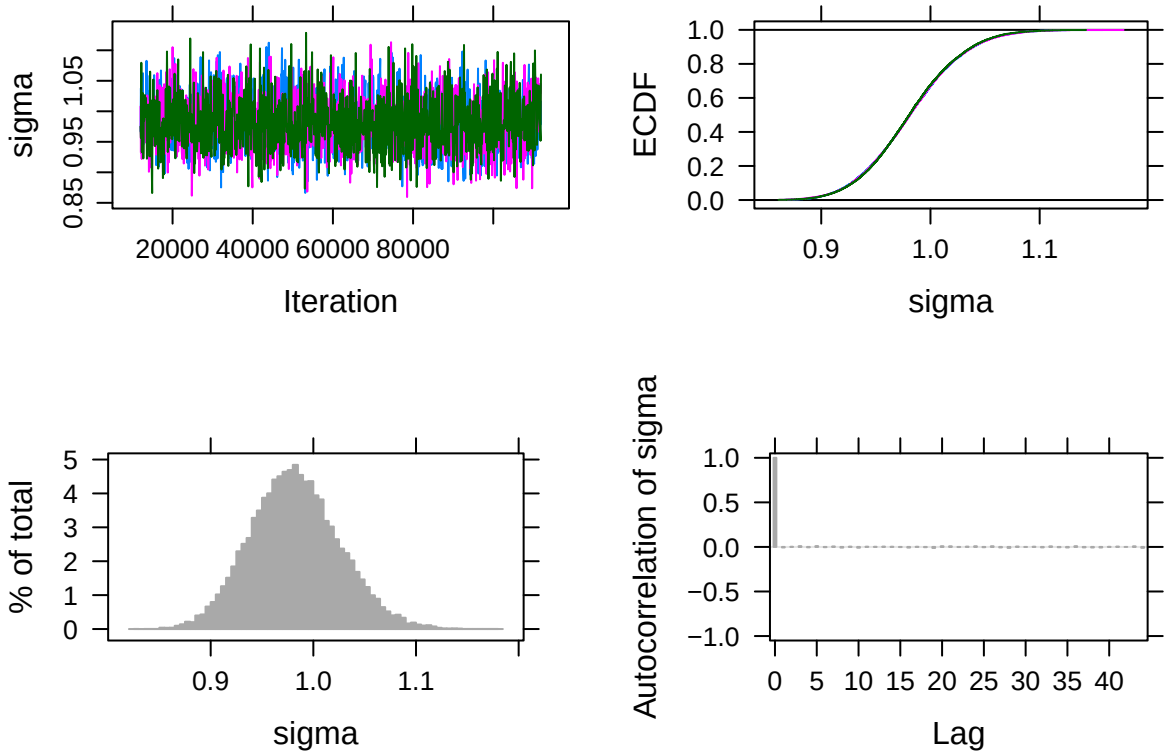
```
##                     Lower95        Median    Upper95          Mean         SD
## beta_0         -0.11949300 -0.0008760065  0.1162440 -0.0005288852 0.06026997
## beta_inflation -0.30389600 -0.1871610000 -0.0676891 -0.1872252475 0.06058635
## beta_ip         0.00310625  0.1191255000  0.2427680  0.1198368385 0.06099385
## sigma           0.89952300  0.9808905000  1.0681500  0.9822849954 0.04315518
##                   Mode        MCerr MC%ofSD SSeff        AC.100      psrf
## beta_0              NA 0.0003479688     0.6 30000 -0.0012813583 1.0001924
## beta_inflation      NA 0.0003497954     0.6 30000 -0.0003075652 0.9999796
## beta_ip             NA 0.0003521482     0.6 30000 -0.0054514449 1.0000648
## sigma               NA 0.0002491566     0.6 30000 -0.0091786612 1.0000309
```

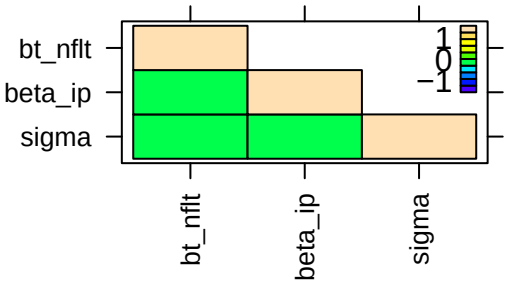
```
# Plot diagnostic plots
plot(posterior_reduced_scaled, vars = c("beta_inflation",
                          "beta_ip", "sigma"))
```

```
## Generating plots...
```




beta_ip
−0.1
0.1
0.3
20000 40000 60000 80000
Iteration
ECDF
0.0
0.2
0.4
0.6
0.8
1.0
0.0 0.1 0.2 0.3
beta_ip
% of total
0
1
2
3
−0.1 0.0 0.1 0.2 0.3
beta_ip
Autocorrelation of beta_ip
−1.0
−0.5
0.0
0.5
1.0
0 5 10 15 20 25 30 35 40
Lag


sigma
0.85 0.95 1.05
20000 40000 60000 80000
Iteration
ECDF
1.0
0.8
0.6
0.4
0.2
0.0
0.9
1.0
1.1
sigma
% of total
5
4
3
2
1
0
0.9
1.0
1.1
sigma
Autocorrelation of sigma
1.0
0.5
0.0
−0.5
−1.0
0 5 10 15 20 25 30 35 40
Lag




```
# Convert draws to matrix form
post_red_scaled <- as.matrix(as.mcmc(posterior_reduced_scaled))
```

```
## Warning in as.mcmc.runjags(posterior_reduced_scaled): Combining the 3 mcmc
## chains together
```

```
# Define vector of reduced params
red_params <- c("beta_0", "beta_inflation", "beta_ip", "sigma")

# Compute mean, upper and lower bounds
red_mean  <- apply(post_red_scaled[, red_params], 2, mean)
red_lower <- apply(post_red_scaled[, red_params], 2, quantile, probs = 0.025)
red_upper <- apply(post_red_scaled[, red_params], 2, quantile, probs = 0.975)

# Create result table
bayes_reduced_table <- data.frame(
  Parameter = red_params,
  Mean = red_mean,
  CrI_2.5 = red_lower,
  CrI_97.5 = red_upper
)

# Display table
knitr::kable(
  bayes_reduced_table,
```

```
  digits = 4,
  caption = "Posterior summary for reduced Bayesian model"
)
```

Table 9: Posterior summary for reduced Bayesian model

| | Parameter | Mean | CrI_2.5 | CrI_97.5 |
|---|---|---|---|---|
| beta_0 | beta_0 | -0.0005 | -0.1182 | 0.1176 |
| beta_inflation | beta_inflation | -0.1872 | -0.3058 | -0.0689 |
| beta_ip | beta_ip | 0.1198 | 0.0007 | 0.2407 |
| sigma | sigma | 0.9823 | 0.9016 | 1.0709 |

```
# Convert draws to matrix form
post_red_unscaled <- as.matrix(as.mcmc(posterior_reduced))
```

```
## Warning in as.mcmc.runjags(posterior_reduced): Combining the 3 mcmc chains
## together
```

```
# Define design matrix
X_red <- model.matrix(~ lag_inflation + lag_ip_growth, data = df_final)

# Combine coefficient draws
beta_draws_red <- cbind(
  post_red_unscaled[, "beta_0"],
  post_red_unscaled[, "beta_inflation"],
  post_red_unscaled[, "beta_ip"]
)

# Compute and extract desired params
mu_post_red <- beta_draws_red %*% t(X_red)
sigma_post_red <- post_red_unscaled[, "sigma"]

# Create table for results
bayes_prob_neg_red <- sapply(1:ncol(mu_post_red), function(j) {
  mean(pnorm(0, mean = mu_post_red[, j], sd = sigma_post_red))
})

# Store results in dataframe
df_final$bayes_prob_neg_red <- bayes_prob_neg_red
```

```
# Compute posterior means
bayes_pred_mean_red <- colMeans(mu_post_red)

# Store predicted GDP growth in dataframe
df_final$bayes_pred_gdp_growth_red <- bayes_pred_mean_red

# Compute error metrics
bayes_red_rmse <- sqrt(mean((df_final$gdp_growth - df_final$bayes_pred_gdp_growth_red)^2, na.rm = TRUE))
bayes_red_mae  <- mean(abs(df_final$gdp_growth - df_final$bayes_pred_gdp_growth_red), na.rm = TRUE)

# Create table
```

```r
bayes_red_fit <- data.frame(
  Model = "Bayesian Reduced",
  RMSE = bayes_red_rmse,
  MAE = bayes_red_mae
)

# Display table
knitr::kable(
  bayes_red_fit,
  digits = 5,
  caption = "Predictive fit for reduced Bayesian model"
)
```

Table 10: Predictive fit for reduced Bayesian model

| Model | RMSE | MAE |
|---|---|---|
| Bayesian Reduced | 0.01026 | 0.00596 |

```r
# Create table
reduced_fit_comparison <- data.frame(
  Model = c("Frequentist Reduced", "Bayesian Reduced"),
  RMSE = c(freq_red_rmse, bayes_red_rmse),
  MAE  = c(freq_red_mae, bayes_red_mae)
)

# Display table
knitr::kable(
  reduced_fit_comparison,
  digits = 5,
  caption = "Reduced-model predictive fit comparison"
)
```

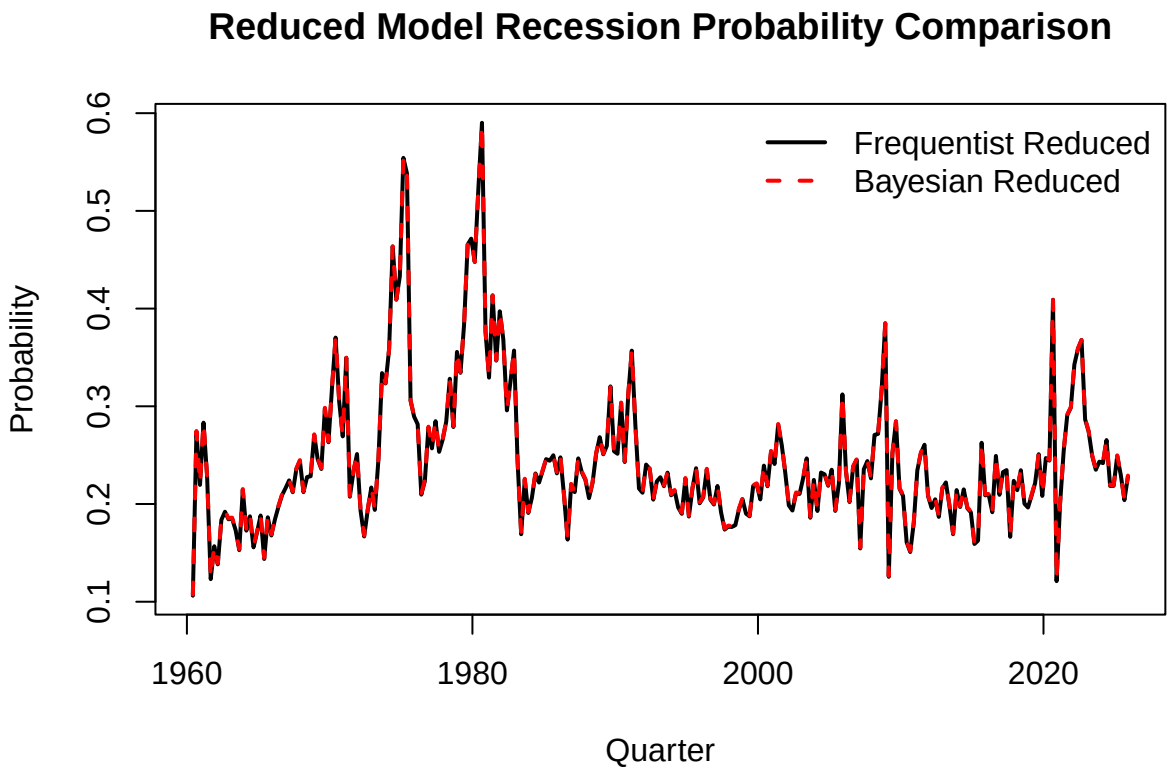
Table 11: Reduced-model predictive fit comparison

| Model | RMSE | MAE |
|---|---|---|
| Frequentist Reduced | 0.01026 | 0.00596 |
| Bayesian Reduced | 0.01026 | 0.00596 |

```r
# Plot probability of negative GDP growth for frequentist vs. Bayesian for reduced models
plot(df_final$date, df_final$recession_prob_red,
     type = "l", lwd = 2,
     col = "black",
     ylim = range(c(df_final$recession_prob_red, df_final$bayes_prob_neg_red), na.rm = TRUE),
     main = "Reduced Model Recession Probability Comparison",
     xlab = "Quarter",
     ylab = "Probability")

lines(df_final$date, df_final$bayes_prob_neg_red,
      lwd = 2, lty = 2, col = "red")
```

```
legend("topright",
       legend = c("Frequentist Reduced", "Bayesian Reduced"),
       col = c("black", "red"),
       lwd = 2,
       lty = c(1, 2),
       bty = "n")
```




### Comparison of ALL Models

```
# Plot probabilities of negative GDP growth of all models
plot(df_final$date, df_final$recession_prob,
     type = "l", lwd = 2,
     col = "black",
     ylim = range(c(
       df_final$recession_prob,
       df_final$bayes_prob_neg,
       df_final$bayes_t_prob_neg,
       df_final$recession_prob_red,
       df_final$bayes_prob_neg_red
     ), na.rm = TRUE),
     main = "Recession Probability Comparison Across All Models",
     xlab = "Quarter",
     ylab = "Probability")

# Full models
lines(df_final$date, df_final$bayes_prob_neg,
```

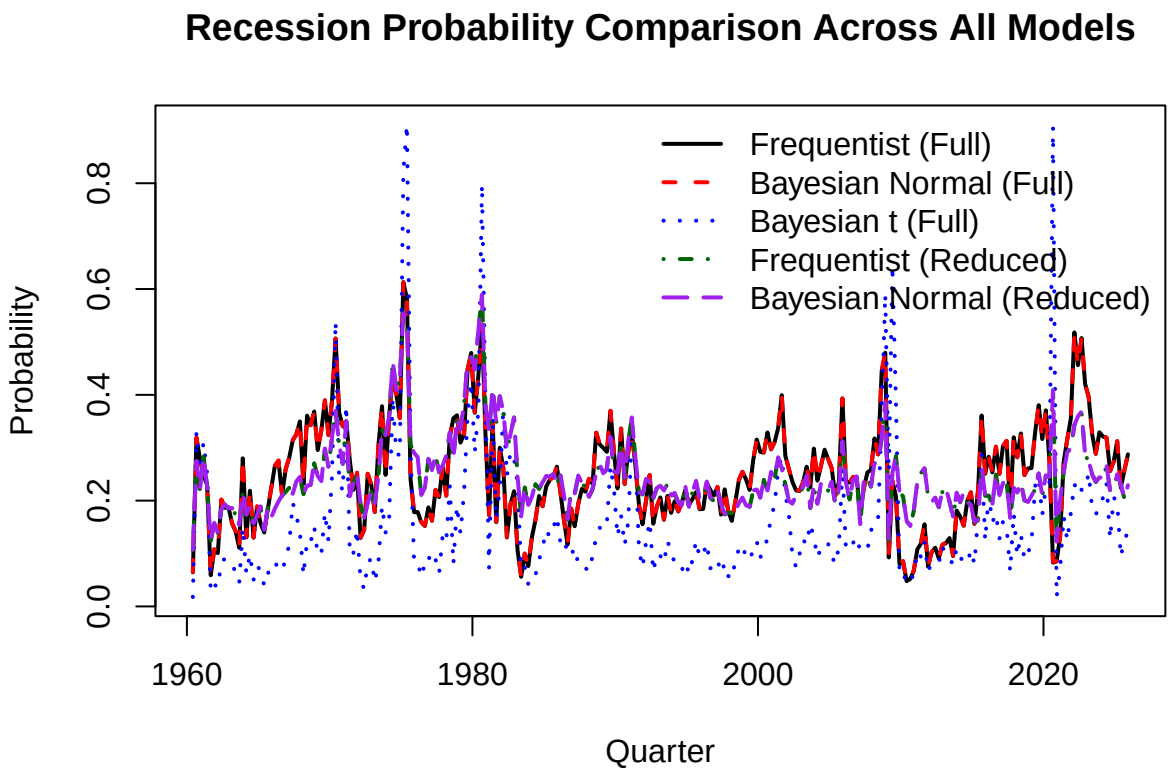
```
      col = "red", lwd = 2, lty = 2)

lines(df_final$date, df_final$bayes_t_prob_neg,
      col = "blue", lwd = 2, lty = 3)

# Reduced models
lines(df_final$date, df_final$recession_prob_red,
      col = "darkgreen", lwd = 2, lty = 4)

lines(df_final$date, df_final$bayes_prob_neg_red,
      col = "purple", lwd = 2, lty = 5)

legend("topright",
       legend = c(
         "Frequentist (Full)",
         "Bayesian Normal (Full)",
         "Bayesian t (Full)",
         "Frequentist (Reduced)",
         "Bayesian Normal (Reduced)"
       ),
       col = c("black", "red", "blue", "darkgreen", "purple"),
       lwd = 2,
       lty = c(1,2,3,4,5),
       bty = "n")
```




## Rolling Bayesian Model (shifting window)

```r
# Define constants for simulation
window_size <- 80
n_total <- nrow(df_final)
total_windows <- n_total - window_size + 1

# Initialize dataframe for results
rolling_results <- data.frame()

# Define progress bar
pb <- txtProgressBar(min = 0, max = total_windows, style = 3)
```

```
##   |                                                                      |
```

```r
# Loop through all windows and perform JAGS simulation
for (start_idx in 1:total_windows) {

  # Track progress through bar
  setTxtProgressBar(pb, start_idx)
  cat(sprintf("  Window %d of %d\r", start_idx, total_windows))

  # Define end index
  end_idx <- start_idx + window_size - 1
  df_window <- df_final[start_idx:end_idx, ]

  # Extract windowed data for each predictor and response
  y_win <- as.vector(df_window$gdp_growth)
  lag_gdp_win <- as.vector(df_window$lag_gdp_growth)
  lag_unrate_win <- as.vector(df_window$lag_unrate)
  lag_inflation_win <- as.vector(df_window$lag_inflation)
  lag_ip_win <- as.vector(df_window$lag_ip_growth)
  lag_spread_win <- as.vector(df_window$lag_spread)

  the_data_win <- list(
    y = y_win,
    lag_gdp = lag_gdp_win,
    lag_unrate = lag_unrate_win,
    lag_inflation = lag_inflation_win,
    lag_ip = lag_ip_win,
    lag_spread = lag_spread_win,
    N = length(y_win),
    mu0 = 0, g0 = 1/25,
    mu1 = 0, g1 = 1/25,
    mu2 = 0, g2 = 1/25,
    mu3 = 0, g3 = 1/25,
    mu4 = 0, g4 = 1/25,
    mu5 = 0, g5 = 1/25,
    mu_sigma = 0,
    tau_sigma = 1/25,
    nu_sigma = 1,
    upper_sigma = 1e6
  )

  fit_win <- NULL
```

```
# Try catch to remedy invalid model run
tryCatch(
  {
    capture.output(
      fit_win <- run.jags(
        model = modelString,
        n.chains = 2,
        data = the_data_win,
        monitor = c("beta_gdp", "beta_unrate", "beta_inflation", "beta_ip", "beta_spread"),
        adapt = 1000,
        burnin = 3000,
        sample = 3000,
        thin = 5,
        inits = initsfunction,
        method = "rjags",
        summarise = TRUE
      ),
      file = NULL
    )
  },
  error = function(e) {
    message(sprintf("\nWindow %d failed: %s", start_idx, e$message))
  }
)

if (is.null(fit_win)) {
  next
}

# Store summary
summ <- summary(fit_win)

# Format results in dataframe
rolling_results <- rbind(
  rolling_results,
  data.frame(
    end_date = df_window$date[nrow(df_window)],

    beta_gdp_mean = summ["beta_gdp", "Mean"],
    beta_gdp_low = summ["beta_gdp", "Lower95"],
    beta_gdp_high = summ["beta_gdp", "Upper95"],

    beta_unrate_mean = summ["beta_unrate", "Mean"],
    beta_unrate_low = summ["beta_unrate", "Lower95"],
    beta_unrate_high = summ["beta_unrate", "Upper95"],

    beta_inflation_mean = summ["beta_inflation", "Mean"],
    beta_inflation_low = summ["beta_inflation", "Lower95"],
    beta_inflation_high = summ["beta_inflation", "Upper95"],

    beta_ip_mean = summ["beta_ip", "Mean"],
    beta_ip_low = summ["beta_ip", "Lower95"],
    beta_ip_high = summ["beta_ip", "Upper95"],
```

```
      beta_spread_mean = summ["beta_spread", "Mean"],
      beta_spread_low = summ["beta_spread", "Lower95"],
      beta_spread_high = summ["beta_spread", "Upper95"]
    )
  )
}
```

```
##   |                                                                      |
```

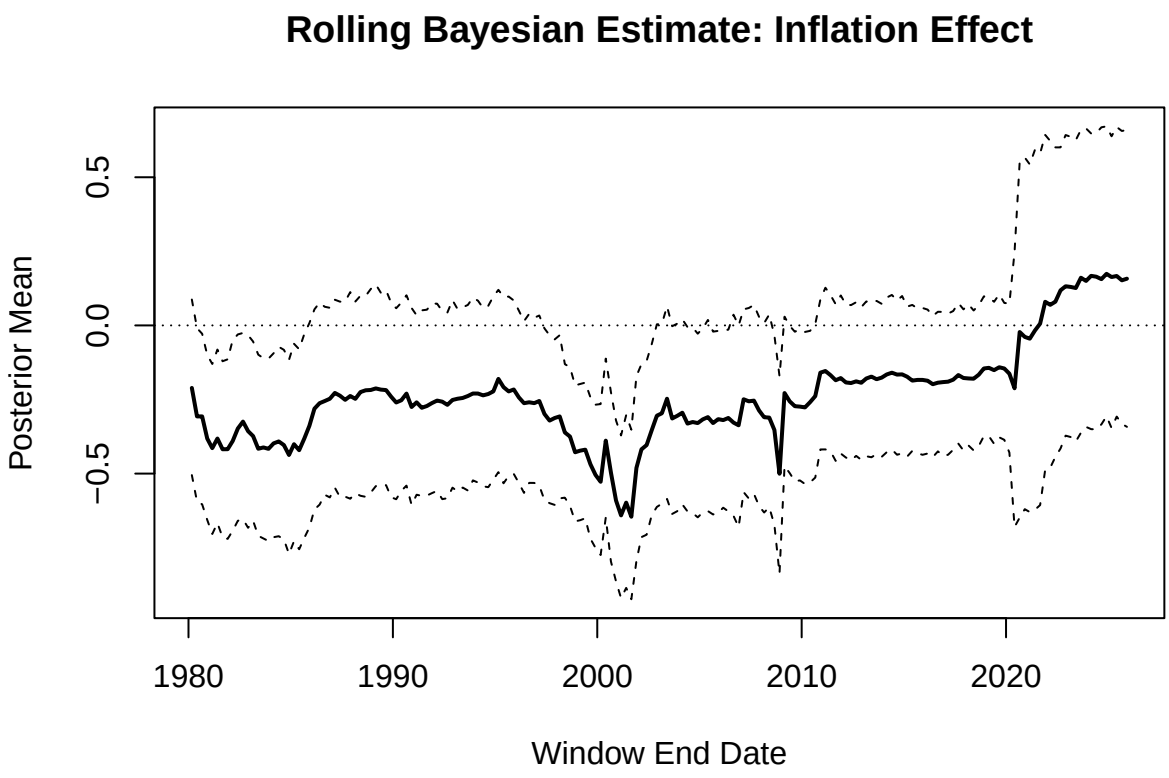
```
close(pb)
```

```
cat("\nDone.\n")
```

```
##
## Done.
```

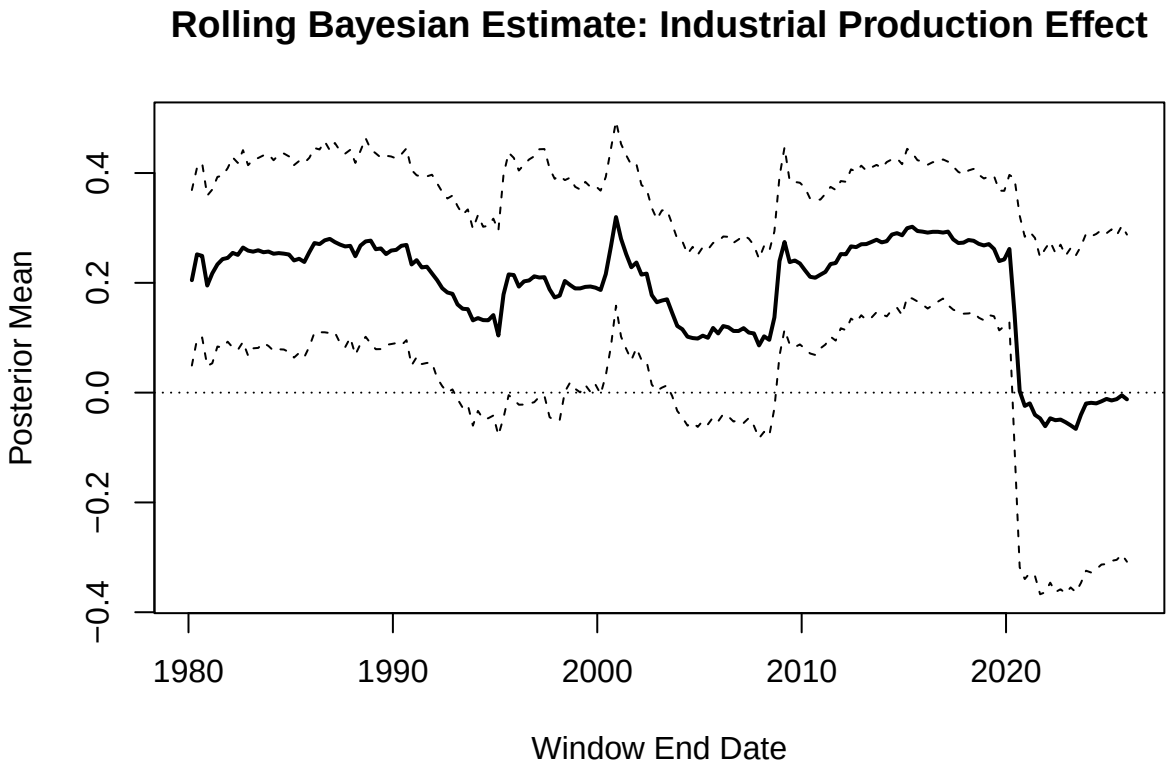
```
# Plot rolling coefficient value plot for Inflation
plot(rolling_results$end_date, rolling_results$beta_inflation_mean,
     type = "l", lwd = 2,
     ylim = range(c(rolling_results$beta_inflation_low,
                    rolling_results$beta_inflation_high)),
     main = "Rolling Bayesian Estimate: Inflation Effect",
     xlab = "Window End Date",
     ylab = "Posterior Mean")

lines(rolling_results$end_date, rolling_results$beta_inflation_low, lty = 2)
lines(rolling_results$end_date, rolling_results$beta_inflation_high, lty = 2)
abline(h = 0, lty = 3)
```



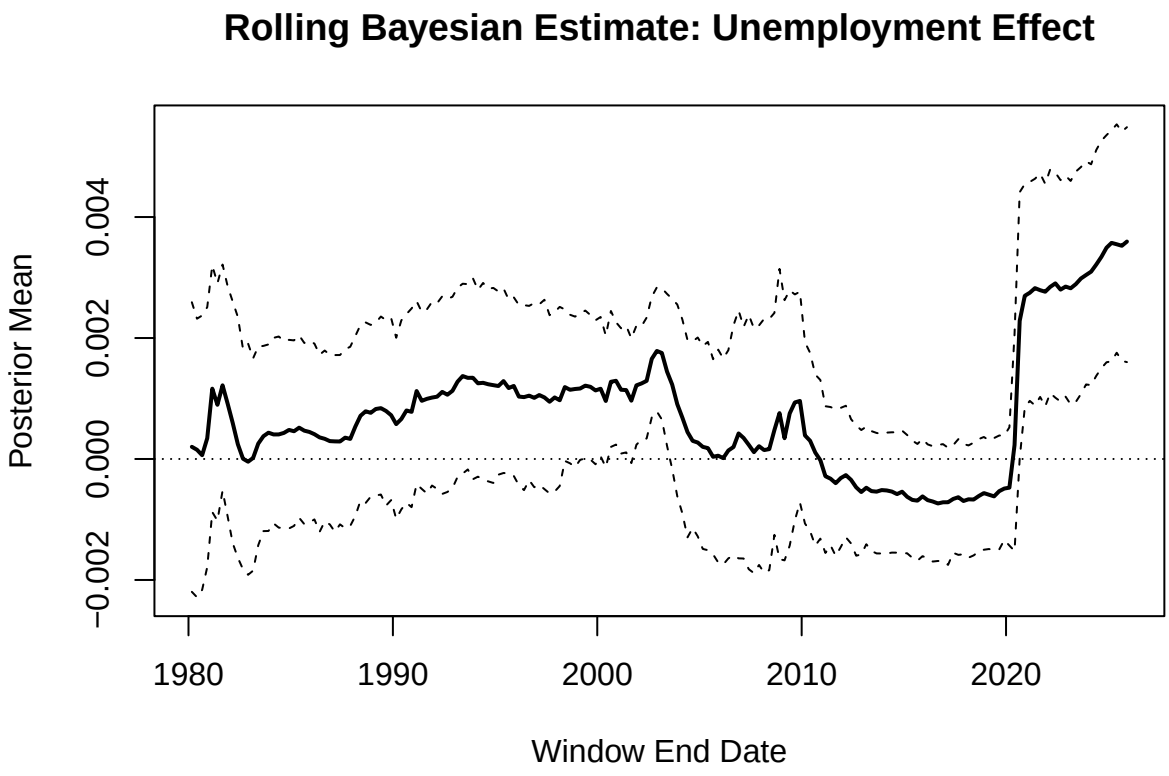

```
# Plot rolling coefficient value plot for IP
plot(rolling_results$end_date, rolling_results$beta_ip_mean,
     type = "l", lwd = 2,
     ylim = range(c(rolling_results$beta_ip_low,
                    rolling_results$beta_ip_high)),
     main = "Rolling Bayesian Estimate: Industrial Production Effect",
     xlab = "Window End Date",
     ylab = "Posterior Mean")

lines(rolling_results$end_date, rolling_results$beta_ip_low, lty = 2)
lines(rolling_results$end_date, rolling_results$beta_ip_high, lty = 2)
abline(h = 0, lty = 3)
```



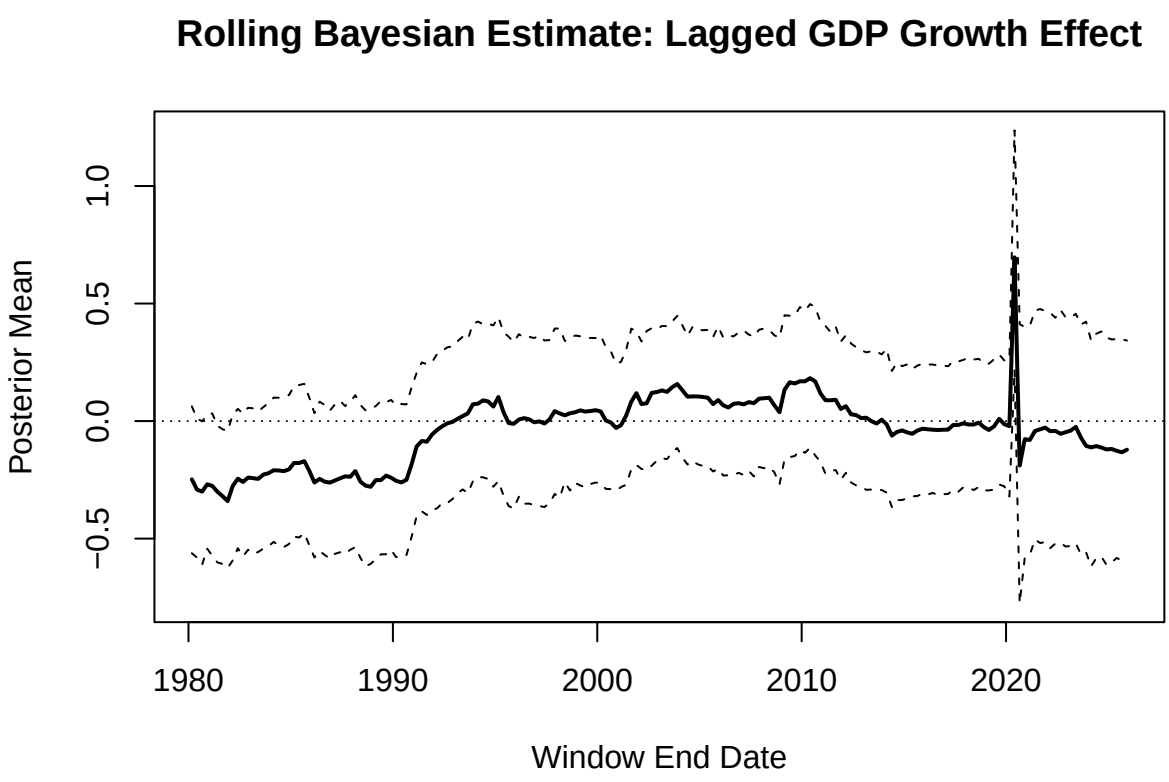

```r
# Plot rolling coefficient value plot for Unemployment
plot(rolling_results$end_date, rolling_results$beta_unrate_mean,
     type = "l", lwd = 2,
     ylim = range(c(rolling_results$beta_unrate_low,
                    rolling_results$beta_unrate_high), na.rm = TRUE),
     main = "Rolling Bayesian Estimate: Unemployment Effect",
     xlab = "Window End Date",
     ylab = "Posterior Mean")

lines(rolling_results$end_date, rolling_results$beta_unrate_low, lty = 2)
lines(rolling_results$end_date, rolling_results$beta_unrate_high, lty = 2)
abline(h = 0, lty = 3)
```



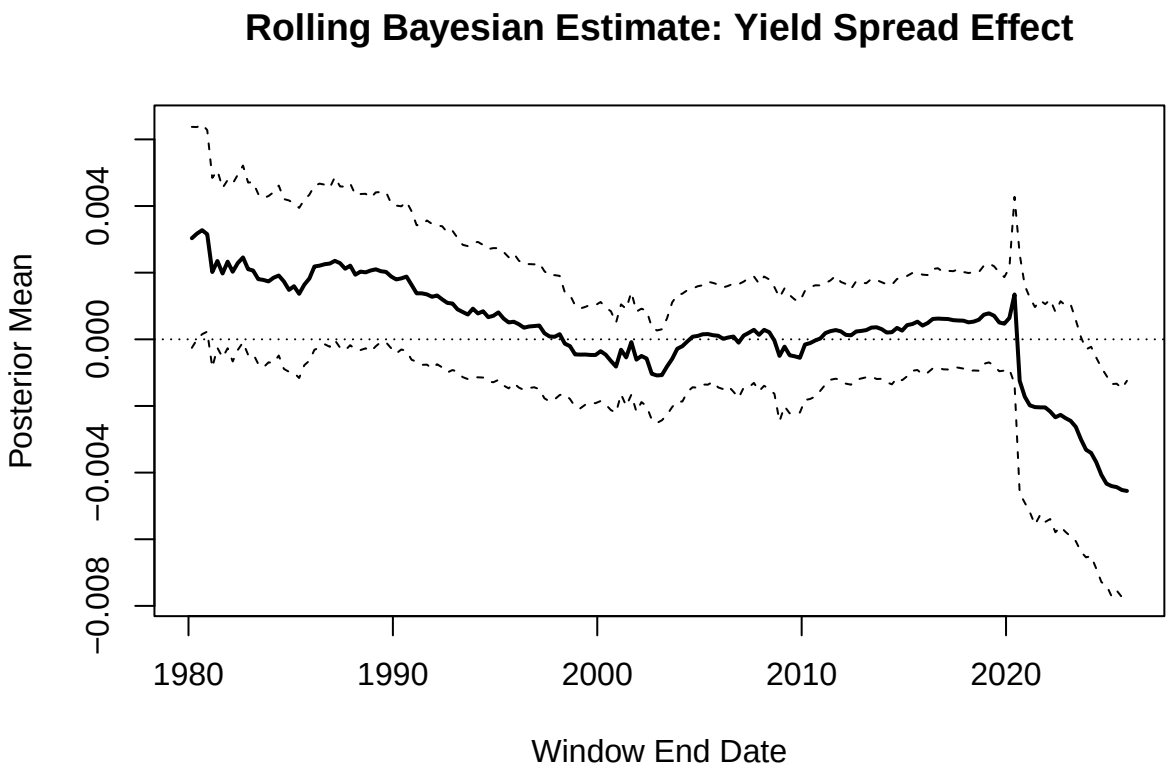

```r
# Plot rolling coefficient value plot for Lagged GDP growth
plot(rolling_results$end_date, rolling_results$beta_gdp_mean,
     type = "l", lwd = 2,
     ylim = range(c(rolling_results$beta_gdp_low,
                    rolling_results$beta_gdp_high), na.rm = TRUE),
     main = "Rolling Bayesian Estimate: Lagged GDP Growth Effect",
     xlab = "Window End Date",
     ylab = "Posterior Mean")

lines(rolling_results$end_date, rolling_results$beta_gdp_low, lty = 2)
lines(rolling_results$end_date, rolling_results$beta_gdp_high, lty = 2)
abline(h = 0, lty = 3)
```




```r
# Plot rolling coefficient value plot for Yield Spread
plot(rolling_results$end_date, rolling_results$beta_spread_mean,
     type = "l", lwd = 2,
     ylim = range(c(rolling_results$beta_spread_low,
                    rolling_results$beta_spread_high), na.rm = TRUE),
     main = "Rolling Bayesian Estimate: Yield Spread Effect",
     xlab = "Window End Date",
     ylab = "Posterior Mean")

lines(rolling_results$end_date, rolling_results$beta_spread_low, lty = 2)
lines(rolling_results$end_date, rolling_results$beta_spread_high, lty = 2)
abline(h = 0, lty = 3)
```




# 7.) PPC

## Normal PPC

```
# Posterior draws from normal Bayesian model
post_norm <- as.matrix(as.mcmc(posterior_MLR))
```

```
## Warning in as.mcmc.runjags(posterior_MLR): Combining the 3 mcmc chains together
```

```
# Design matrix
X_unscaled <- model.matrix(
  ~ lag_gdp_growth + lag_unrate + lag_inflation + lag_ip_growth + lag_spread,
  data = df_final
)

# Coefficient draws
beta_draws_norm <- cbind(
  post_norm[, "beta_0"],
  post_norm[, "beta_gdp"],
  post_norm[, "beta_unrate"],
  post_norm[, "beta_inflation"],
  post_norm[, "beta_ip"],
```

```
  post_norm[, "beta_spread"]
)

# Posterior means for each observation
mu_post_norm <- beta_draws_norm %*% t(X_unscaled)
sigma_post_norm <- post_norm[, "sigma"]

# Simulate replicated datasets
set.seed(123)
n_draws <- nrow(mu_post_norm)

yrep_norm <- matrix(NA, nrow = n_draws, ncol = ncol(mu_post_norm))

for (s in 1:n_draws) {
  yrep_norm[s, ] <- rnorm(
    n = ncol(mu_post_norm),
    mean = mu_post_norm[s, ],
    sd = sigma_post_norm[s]
  )
}
```

## T Bayesian PPC

```
# Define constants for PPC sim
set.seed(123)
n_draws_t <- nrow(mu_post_t)

# Posterior draws from t Bayesian model
post_t <- as.matrix(as.mcmc(posterior_t_unscaled))
```

```
## Warning in as.mcmc.runjags(posterior_t_unscaled): Combining the 3 mcmc chains
## together
```

```
# Combine draws
beta_draws_t <- cbind(
  post_t[, "beta_0"],
  post_t[, "beta_gdp"],
  post_t[, "beta_unrate"],
  post_t[, "beta_inflation"],
  post_t[, "beta_ip"],
  post_t[, "beta_spread"]
)

# Compute and extract desired params
mu_post_t <- beta_draws_t %*% t(X_unscaled)
sigma_post_t <- post_t[, "sigma"]
nu_post_t <- post_t[, "nu"]

# Initialize predicted value storage
yrep_t <- matrix(NA, nrow = n_draws_t, ncol = ncol(mu_post_t))
```

```r
# Populate PPC data matrix
for (s in 1:n_draws_t) {
  yrep_t[s, ] <- mu_post_t[s, ] + sigma_post_t[s] * rt(ncol(mu_post_t), df = nu_post_t[s])
}
```

```r
# Observed data
y_obs <- df_final$gdp_growth

# Define function that combines PPC metrics into summaries
ppc_stats <- function(y) {
  c(
    mean = mean(y),
    sd = sd(y),
    min = min(y),
    prop_negative = mean(y < 0),
    n_extreme_neg = sum(y < -0.02)
  )
}

# Summary for observed vals
obs_stats <- ppc_stats(y_obs)

# Summary for replications from normal model
rep_stats_norm <- t(apply(yrep_norm, 1, ppc_stats))

# Summary for replicated from t model
rep_stats_t <- t(apply(yrep_t, 1, ppc_stats))
```

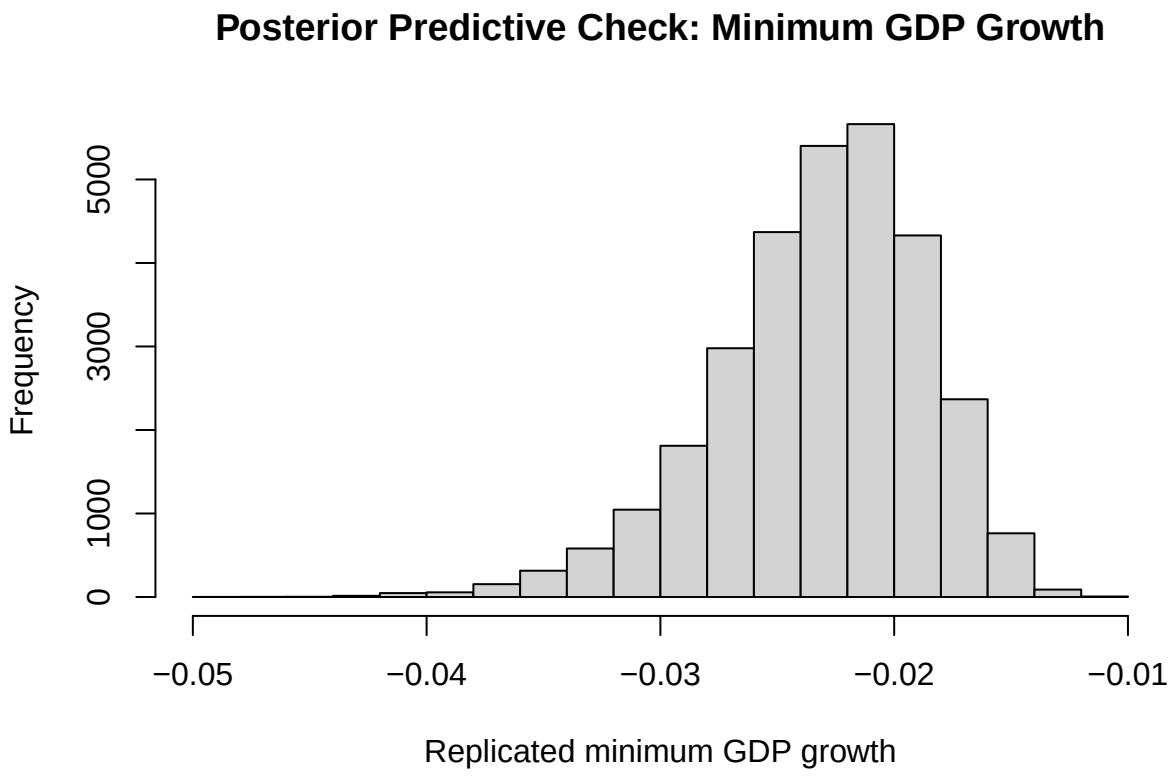
```r
# Create table
ppp_table <- data.frame(
  Statistic = names(obs_stats),
  Observed = as.numeric(obs_stats),
  PPP_Normal = sapply(1:length(obs_stats), function(j) mean(rep_stats_norm[, j] >= obs_stats[j])),
  PPP_t = sapply(1:length(obs_stats), function(j) mean(rep_stats_t[, j] >= obs_stats[j]))
)

# DIsplay table
knitr::kable(
  ppp_table,
  digits = 4,
  caption = "Posterior predictive checks for Bayesian normal and Bayesian t models"
)
```

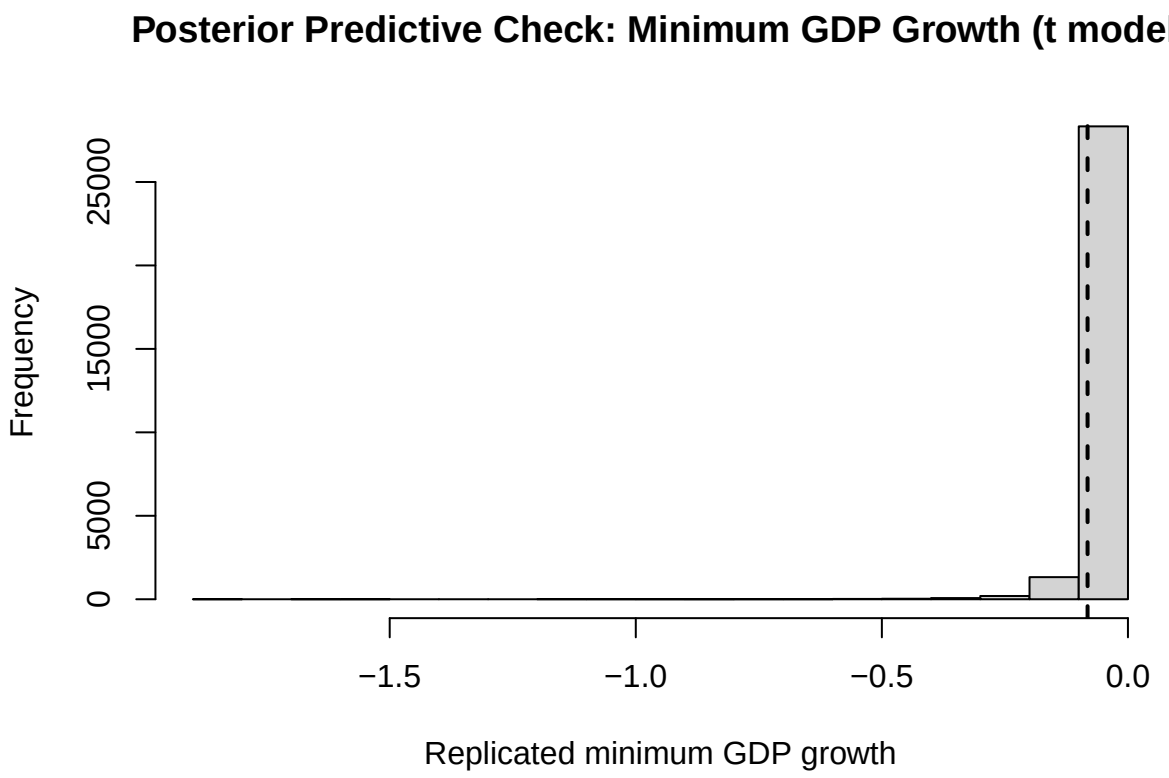
Table 12: Posterior predictive checks for Bayesian normal and Bayesian t models

| Statistic | Observed | PPP_Normal | PPP_t |
|---|---|---|---|
| mean | 0.0073 | 0.5007 | 0.5329 |
| sd | 0.0105 | 0.6136 | 0.2502 |
| min | -0.0820 | 1.0000 | 0.9129 |
| prop_negative | 0.1217 | 1.0000 | 0.9159 |
| n_extreme_neg | 3.0000 | 0.2004 | 0.4105 |

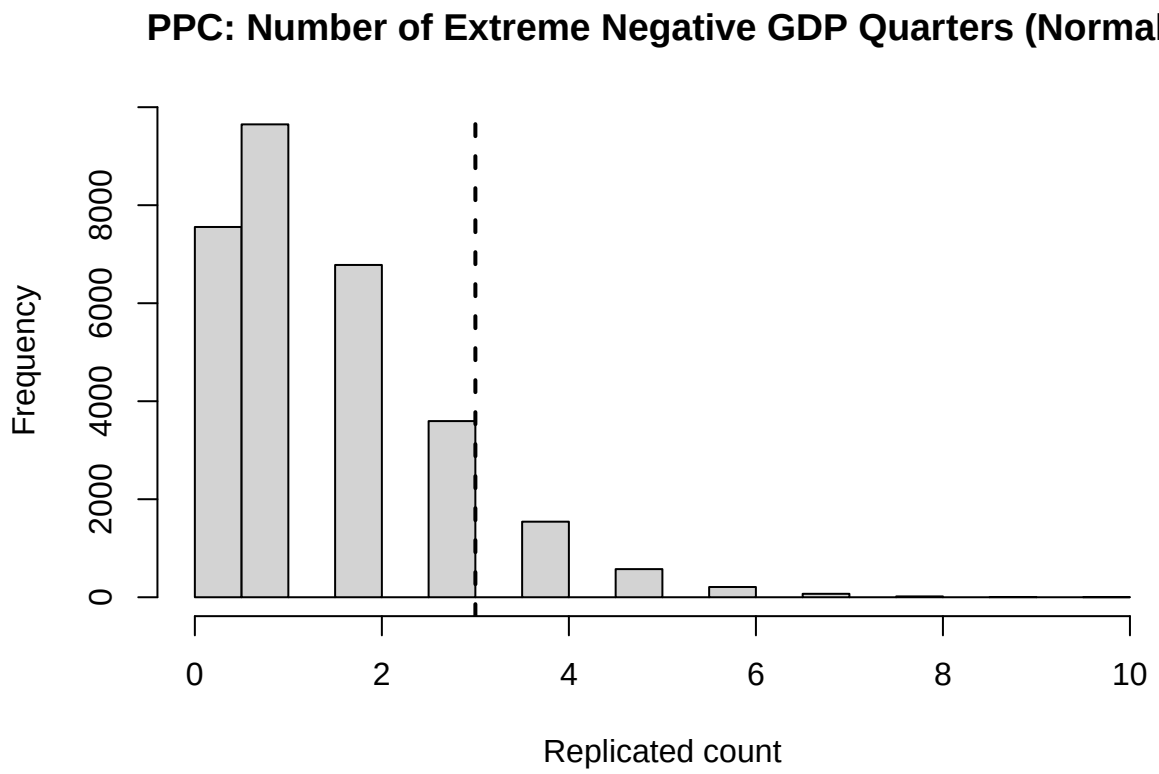
```r
# Plot histogram of minimum GDP growth for normal model
hist(rep_stats_norm[, "min"],
     main = "Posterior Predictive Check: Minimum GDP Growth",
     xlab = "Replicated minimum GDP growth")
abline(v = obs_stats["min"], lwd = 2, lty = 2)
```



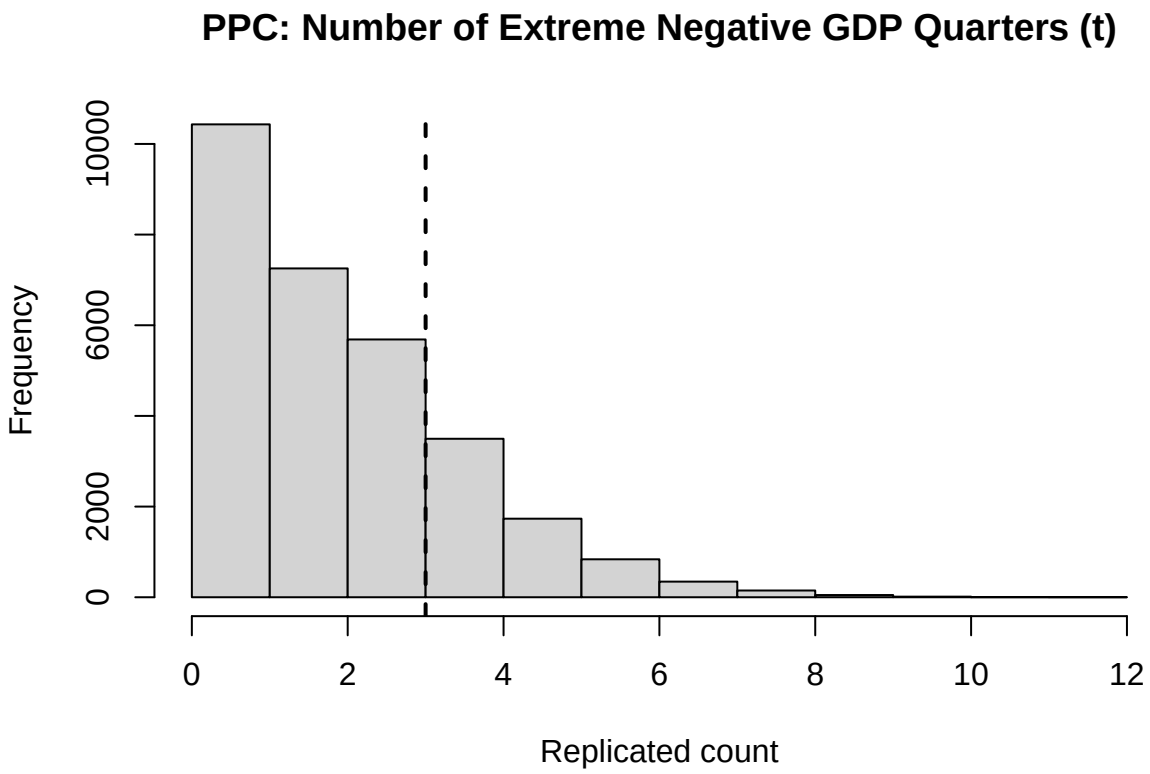

```r
# Plot histogram of minimum GDP growth for t model
hist(rep_stats_t[, "min"],
     main = "Posterior Predictive Check: Minimum GDP Growth (t model)",
     xlab = "Replicated minimum GDP growth")
abline(v = obs_stats["min"], lwd = 2, lty = 2)
```




```
# Plot histogram for number of extreme negative GDP Quarters for normal model
hist(rep_stats_norm[, "n_extreme_neg"],
     main = "PPC: Number of Extreme Negative GDP Quarters (Normal)",
     xlab = "Replicated count")
abline(v = obs_stats["n_extreme_neg"], lwd = 2, lty = 2)
```




```r
# Plot histogram for number of extreme negative GDP Quarters for t model
hist(rep_stats_t[, "n_extreme_neg"],
     main = "PPC: Number of Extreme Negative GDP Quarters (t)",
     xlab = "Replicated count")
abline(v = obs_stats["n_extreme_neg"], lwd = 2, lty = 2)
```




## 8.) Remarks and Conclusion